

数学

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	v
第一章 快速幂	1
1.1 无符号 64 位模乘与模幂	1
1.2 模幂与几何级数同时求值	1
第二章 数论	2
2.1 数论基础	2
2.1.1 幂的约数和	2
2.2 模算术简介	2
2.2.1 动态模整数与合数模逆元	2
2.2.2 静态模整数	5
2.3 素数	6
2.3.1 确定性 64 位 Miller-Rabin 判素	6
2.4 欧拉函数	7
2.4.1 单个数的欧拉函数	7
2.5 筛法	8
2.5.1 线性筛、欧拉函数与 Möbius 函数	8
2.5.2 区间筛素数	9
2.6 分解质因数	10
2.6.1 Pollard-Rho 因数分解	10
2.7 裴蜀定理 & 一次不定方程	11
2.7.1 扩展欧几里得与 Bézout 系数	11
2.7.2 二元一次不定方程	12
2.8 费马小定理 & 欧拉定理	12
2.8.1 Carmichael: 模乘法群的公共指数	12
2.9 模逆元	13
2.9.1 模整数类型的批量逆元	13
2.9.2 任意正模数的单个逆元	13
2.9.3 任意数组的批量逆元	14
2.9.4 连续整数的线性逆元表	14
2.10 线性同余方程	15
2.10.1 线性同余方程	15
2.11 中国剩余定理	15
2.11.1 广义 CRT: 逐项合并同余	15
2.11.2 Garner: 大周期 CRT 与混合进制	16
2.12 卢卡斯定理	17
2.12.1 Lucas: 小素数模的大下标组合数	17
2.12.2 合数模数组合数 (扩展 Lucas)	18
2.13 二次剩余	20
2.13.1 素数模平方根 (Cipolla)	20
2.14 阶 & 原根	21
2.14.1 乘法阶与原根	21

2.15	离散对数	24
2.15.1	扩展 BSGS	24
2.16	高次剩余 & 单位根	25
2.16.1	素数模高次剩余：全部根的紧凑表示	25
2.16.2	素数幂模高次剩余：含非互素被开方数	26
2.16.3	合数求根准备：分解模数与素数幂原根	29
2.16.4	一般合数模高次剩余：CRT 组合与按索引取根	30
2.17	莫比乌斯反演	32
2.17.1	矩形 GCD 计数	32
2.18	狄利克雷双曲线法 & 杜教筛	33
2.18.1	杜教筛	33
2.19	类欧几里德算法	34
2.19.1	有符号整除和：类欧几里德	34
2.19.2	整除和、带权和与平方和	35
第三章	多项式与生成函数	36
3.1	快速数论变换	36
3.1.1	可指定素数与原根的 NTT 卷积	36
3.1.2	精确有符号整数卷积	38
3.2	快速沃尔什变换	39
3.2.1	按位 OR、AND、XOR 卷积	39
3.2.2	不相交子集卷积	40
3.3	Chirp Z 变换	42
3.3.1	Chirp Z：等比点求值	42
3.4	多项式牛顿迭代	43
3.4.1	形式幂级数求逆	43
3.5	多项式初等函数	44
3.5.1	形式幂级数微积分、对数与指数	44
3.6	常系数齐次线性递推	46
3.6.1	特征多项式求线性递推第 k 项	46
3.6.2	快速递推与有理生成函数单项	47
3.7	多项式平移 连续点值平移	48
3.7.1	多项式系数平移	48
第四章	组合数学	49
4.1	排列组合	49
4.1.1	阶乘组合数	49
4.2	斯特林数	50
4.2.1	第二类斯特林数：同一行前缀	50
4.2.2	无符号第一类斯特林数：同一行	51
4.3	分拆数	52
4.3.1	整数分拆与重数上限	52
第五章	线性代数	53
5.1	矩阵	53
5.1.1	模矩阵乘法与幂	53
5.1.2	Max-plus 矩阵幂	54
5.2	行列式	55
5.2.1	素数模行列式	55

5.2.2	任意正模数行列式：欧几里得消元	56
5.3	线性基	57
5.3.1	64 位异或线性基	57
5.3.2	带位置限制的异或线性基	59
5.3.3	异或线性空间求交	61
5.3.4	Zassenhaus：同时求和空间与交空间	62
5.3.5	无向图最大异或行走	62
第六章	数值算法	65
6.1	插值	65
6.1.1	拉格朗日插值与连续点优化	65
6.2	高斯消元	67
6.2.1	素数模高斯消元与全部解	67
第七章	Berlekamp–Massey 算法	69
7.1	Berlekamp–Massey：最短线性递推	69
第八章	配套知识与验证范围	70
8.1	整除、卷积与筛法	70
8.2	杜教筛：预筛与缓存都不可省略	70
8.3	64 位判素与质因数分解	70
8.4	单个数的欧拉函数	71
8.5	线性筛：三张表的约定	71
8.6	区间筛：只保存目标区间	72
8.7	矩形 GCD 计数：缩放后容斥	72
8.8	整数分拆：重数限制的差一问题	72
8.9	幂的约数和：不除以 $p - 1$	73
8.10	类欧几里德：有符号整除求和	73
8.11	类欧几里德的三个求和量	74
8.12	模运算的适用条件	74
8.13	任意数组的批量逆元	74
8.14	连续整数的线性逆元表	75
8.15	一次不定方程：特解与正整数解范围	75
8.16	线性同余：最小解与最小周期	76
8.17	扩展 CRT：逐式合并	76
8.18	Garner：总周期可以超过整数范围	76
8.19	素数模平方根	77
8.20	乘法阶与原根	77
8.21	Carmichael：最小公共指数	77
8.22	Lucas：按素数进制逐位计算	78
8.23	合数模数组合数与插值	78
8.24	阶乘表：组合数与排列数	78
8.25	组合计数	79
8.26	群作用计数	79
8.27	生成函数与递推	79
8.28	线性代数与生成树	79
8.29	模线性方程组：工厂工期	80
8.30	概率、期望与博弈	80
8.31	线性递推：系数方向与外推条件	80

8.32 素数模高次剩余与单位根	81
8.33 素数幂模高次剩余与根的提升	82
8.34 一般合数模高次剩余：CRT 组合	83
8.35 自动准备模数分解与原根	84
8.36 带权矩阵树定理与方向约定	85
8.37 任意正模数行列式	86
8.38 任意正模数的矩阵树计数	87
8.39 斯特林数的快速整行计算	88
8.40 参数化 NTT 与模数适配	89
8.41 Bostan–Mori 与快速递推单项	90
8.42 集合变换与不相交子集卷积	91
8.43 Taylor 平移与等比点求值	92
第一部分 跨类依赖	93
附录 A 树状数组	94
A.1 树状数组与第 k 小	94
附录 B 矩阵树定理	95
B.1 带权矩阵树：无向树与内外向树形图	95
B.2 任意正模数带权矩阵树	96

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 快速幂

1.1 无符号 64 位模乘与模幂

Mod64::mul(a,b,m) 与 power(a,e,m), a、b、e 支持完整 unsigned long long, m 为非零无符号 64 位模数。乘积用 uint128 后取模, 不通过浮点近似; m=1 返回 0, e=0 返回 1 mod m。mul 为 $O(1)$, power 为 $O(\log(e+1))$ 次模乘、 $O(1)$ 空间。它也是 64 位判素的底层组件。

```
1 struct Mod64
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5
6     static ull mul(ull a, ull b, ull m) { return u128(a) * b % m; }
7
8     static ull power(ull a, ull b, ull m)
9     {
10         assert(m);
11         ull r = 1 % m;
12         a %= m;
13         for (; b >>= 1, a = mul(a, a, m))
14             if (b & 1) r = mul(r, a, m);
15         return r;
16     }
17 };
```

1.2 模幂与几何级数同时求值

power_sum(a,n,mod) 返回 $\{a^n \bmod \text{mod}, \sum_{i=0..n-1} a^i \bmod \text{mod}\}$ 。a 为完整 unsigned long long, n 为 uint128 项数, mod 为正 unsigned long long。n=0 返回 $\{1 \% \text{mod}, 0\}$; a=0 且 n>0 时和为 $1 \% \text{mod}$ 。无需逆元, 合数模数也可用。时间与递归空间 $O(\log(n+1))$, 至多 128 次折半。只抄此函数需 utility、cassert。

```
1 inline pair<unsigned long long, unsigned long long>
2 power_sum(unsigned long long a, __uint128_t n, unsigned long long mod)
3 {
4     assert(mod >= 1);
5     if (!n) return {1 % mod, 0};
6     auto [p, s] = power_sum(a, n / 2, mod);
7     unsigned long long square = __uint128_t(p) * p % mod;
8     unsigned long long sum = __uint128_t(s) * (p + 1) % mod;
9     if (n % 2)
10         return {__uint128_t(square) * a % mod, (__uint128_t(sum) + square) % mod};
11     return {square, sum};
12 }
```

第二章 数论

2.1 数论基础

相关应用：约数和公式应用；需核对正文定位，导航无独立幂约数和页。

2.1.1 幂的约数和

传入 a 、 b 、 mod 和可复用的 PollardRho 分解器 solver ，返回 a^b 的正约数和取模。 a 、 b 、 mod 均为 unsigned long long, $a \geq 1$ 、 $\text{mod} \geq 1$ 。 $a=1$ 或 $b=0$ 时结果为 $1 \% \text{mod}$ 。依赖 power_sum 与分解器；重数 e 对应 $e*b+1$ 项，长度用 uint128。时间由分解及每个不同质因子的 $O(\log(e*b+2))$ 求和组成。

```
1 inline unsigned long long divisor_sum_power(unsigned long long a,
2                                             unsigned long long b,
3                                             unsigned long long mod,
4                                             PollardRho &solver)
5 {
6     assert(a >= 1 && mod >= 1);
7     if (!b) return 1 % mod;
8     auto factors = solver.factor(a);
9     unsigned long long answer = 1 % mod;
10    for (int i = 0, j; i < (int)factors.size(); i = j)
11    {
12        j = i + 1;
13        while (j < (int)factors.size() && factors[j] == factors[i]) j++;
14        __uint128_t length = __uint128_t(j - i) * b + 1;
15        auto sum = power_sum(factors[i], length, mod).second;
16        answer = __uint128_t(answer) * sum % mod;
17    }
18    return answer;
19 }
```

2.2 模算术简介

2.2.1 动态模整数与合数模逆元

$\text{mint}<\text{tag}>$ 按 tag 独立保存运行时模数，默认 998244353。先 $\text{set_mod}(p)$, $1 \leq p \leq \text{INT_MAX}$ ，再构造该模数下的所有对象；更换同 tag 的模数后，旧对象、容器元素和缓存全部失效，必须重建，不能跨模混算。构造支持 long long 全范围， v 为规范余数；加减乘及复合赋值、相等比较为 $O(1)$ ，乘法提升到 long long，不依赖素数。 $\text{pow}(k)$ 的 k 为 uint64_t 非负指数，0 次幂为 $1 \bmod p$ 。 $\text{try_inv}()$ 以扩展欧几里得求逆， $\text{gcd}(v,p) \neq 1$ 返回 nullopt ； $\text{inv}()$ 与除法要求可逆，否则违反断言前提。 $p=1$ 时唯一余数为 0，乘法单位元也为 0， try_inv 返回 0。幂和求逆为 $O(\log p)$ 或 $O(\log k)$ ，除法包含求逆。可与 Fenwick 区间和组合，不能用于模数和的 k th。所有中间乘法受 int 模数上界保护；不支持超过 INT_MAX 的模数，也不提供线程间安全切换模数。

```
1 template <int tag = 0> struct mint
2 {
3     static inline int mod = 998244353;
4     int v;
5
6     static void set_mod(int p)
```

```
7      {
8          assert(p >= 1);
9          mod = p;
10     }
11
12     mint(long long x = 0) : v((x % mod + mod) % mod) {}
13
14     mint operator+(mint b) const { return mint((long long)v + b.v); }
15
16     mint operator-(mint b) const { return mint((long long)v - b.v); }
17
18     mint operator*(mint b) const { return mint(1LL * v * b.v); }
19
20     mint &operator+=(mint b) { return *this = *this + b; }
21
22     mint &operator-=(mint b) { return *this = *this - b; }
23
24     mint &operator*=(mint b) { return *this = *this * b; }
25
26     mint pow(uint64_t k) const
27     {
28         mint a = *this, r = 1;
29         for (; k >>= 1, a *= a)
30             if (k & 1) r *= a;
31         return r;
32     }
```

```
34 optional<mint> try_inv() const
35 {
36     long long a = v, b = mod, x = 1, y = 0;
37     while (b)
38     {
39         long long q = a / b;
40         tie(a, b) = pair{b, a - q * b};
41         tie(x, y) = pair{y, x - q * y};
42     }
43     if (a != 1) return nullopt;
44     return mint(x);
45 }
46
47 mint inv() const
48 {
49     auto x = try_inv();
50     assert(x.has_value());
51     return *x;
52 }
53
54 mint operator/(mint b) const { return *this * b.inv(); }
55
56 mint &operator/=(mint b) { return *this = *this / b; }
57
58 bool operator==(const mint &) const = default;
59 };
```

2.2.2 静态模整数

模板参数必须为不超过 `INT_MAX` 的素数且 ≥ 2 。加减乘、`pow`、`inv`、除法；求逆要求非零，`pow` 指数非负。复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。支持 `+=`、`-=`、`*=`、`/=`，返回自身引用，允许自赋值和链式运算；可作为 Fenwick 区间和的值类型。复合除法沿用非零元素、素数模数前提。

```

1 template <int mod> struct ModInt
2 {
3     int v;
4
5     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
6
7     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
8
9     ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
10
11    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
12
13    ModInt pow(long long e) const
14    {
15        assert(e >= 0);
16        ModInt a = *this, r = 1;
17        for (; e >>= 1, a = a * a)
18            if (e & 1) r = r * a;
19        return r;
20    }
21
22    ModInt inv() const
23    {
24        assert(v);
25        return pow(mod - 2);
26    } // prime modulus
27
28    ModInt operator/(ModInt b) const { return *this * b.inv(); }
29
30    ModInt &operator+=(ModInt b) { return *this = *this + b; }
31
32    ModInt &operator-=(ModInt b) { return *this = *this - b; }
33
34    ModInt &operator*=(ModInt b) { return *this = *this * b; }
35
36    ModInt &operator/=(ModInt b) { return *this = *this / b; }
37 };

```

2.3 素数

2.3.1 确定性 64 位 Miller–Rabin 判素

Prime64::prime(n) 接收完整 unsigned long long, $n < 2^{64}$ 返回 false。先试除小素数, 再使用 2、325、9375、28178、450775、9780504、1795265022 七个底数, 结果在 64 位输入范围内确定。继承 Mod64, 模乘用 uint128; $O(\log n)$ 次模乘, $O(1)$ 空间, 不包含因数分解。

```

1 struct Prime64 : Mod64
2 {
3     static bool prime(ull n)
4     {
5         if (n < 2) return false;
6         for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
7             if (n % p == 0) return n == p;
8         ull d = n - 1;
9         int s = 0;
10        while (!(d & 1))
11        {
12            d >>= 1;
13            ++s;
14        }
15        for (ull a :
16            {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL})
17        {
18            if (a % n == 0) continue;
19            ull x = power(a, d, n);
20            if (x == 1 || x == n - 1) continue;
21            bool ok = false;
22            for (int j = 1; j < s; j++)
23            {
24                x = mul(x, x, n);
25                if (x == n - 1)
26                {
27                    ok = true;
28                    break;
29                }
30            }
31            if (!ok) return false;
32        }
33        return true;
34    }
35 };

```

2.4 欧拉函数

2.4.1 单个数的欧拉函数

`euler_phi(n)` 使用试除, $O(\sqrt{n})$ 时间、 $O(1)$ 额外空间; `euler_phi(n,solver)` 复用 PollardRho, 时间由随机分解主导, 分解结果与递归占 $O(\log n)$ 空间。 $1 \leq n \leq \text{ULLONG_MAX}$, $\phi(1)=1$ 。大素数或平衡大半素数应选分解器入口, 不要用试除穷扫平方根; 批量求 $1..N$ 用 LinearSieve。先除质因子再乘 $p-1$, 结果及中间乘积均不超过原 n 。只抄试除函数时不需要 Prime64/PollardRho。

```
1 inline unsigned long long euler_phi(unsigned long long n)
2 {
3     assert(n >= 1);
4     unsigned long long answer = n;
5     for (unsigned long long p = 2; p <= n / p; p++)
6         if (n % p == 0)
7         {
8             answer = answer / p * (p - 1);
9             while (n % p == 0) n /= p;
10        }
11    if (n > 1) answer = answer / n * (n - 1);
12    return answer;
13 }
14
15 inline unsigned long long euler_phi(unsigned long long n, PollardRho &solver)
16 {
17     assert(n >= 1);
18     auto factors = solver.factor(n);
19     unsigned long long answer = n;
20     unsigned long long previous = 0;
21     for (auto p : factors)
22         if (p != previous)
23         {
24             answer = answer / p * (p - 1);
25             previous = p;
26         }
27     return answer;
28 }
```

2.5 筛法

2.5.1 线性筛、欧拉函数与 Möbius 函数

构造参数 $0 \leq n < \text{INT_MAX}$, 且数组须能分配内存。prime 为递增质数表; lp/phi/mu 为 $0..n$ 数组。 $x \geq 2$ 时 lp[x] 是最小质因子, $\text{lp}[x] == x$ 等价于 x 为素数; 反复除以 lp[x] 可分解质因数。lp[0]=lp[1]=0, 不能直接用 $\text{lp}[x] == x$ 判断 0。 $n \geq 1$ 时 $\text{phi}[1] = \text{mu}[1] = 1$; 第 0 项均为占位 0。 $O(n)$ 时间和空间, 分解单个表内正整数耗时 $O(\log x)$ 。 $n=0$ 不可访问下标 1。

```

1 struct LinearSieve
2 {
3     vector<int> prime, lp, phi, mu;
4
5     LinearSieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
6     {
7         if (n) phi[1] = mu[1] = 1;
8         for (int i = 2; i <= n; i++)
9         {
10             if (!lp[i])
11             {
12                 lp[i] = i;
13                 prime.push_back(i);
14                 phi[i] = i - 1;
15                 mu[i] = -1;
16             }
17             for (int p : prime)
18             {
19                 if (p > n / i) break;
20                 int j = i * p;
21                 lp[j] = p;
22                 if (i % p == 0)
23                 {
24                     phi[j] = phi[i] * p;
25                     mu[j] = 0;
26                     break;
27                 }
28                 phi[j] = phi[i] * (p - 1);
29                 mu[j] = -mu[i];
30             }
31         }
32     }
33 };

```

2.5.2 区间筛素数

`segmented_primes(l,r)` 返回闭区间内递增的全部素数, $0 \leq l \leq r \leq \text{INT_MAX}$; 区间长度须能分配内存。独立函数, 每次重建底筛; 不依赖预存素数表。令 $W=r-l+1$, 时间 $O((\sqrt{r+W})\log \log(r+3))$, 空间 $O(\sqrt{r+W})$ 。倍数起点、递增变量均用 `long long`, 避免 `INT_MAX` 端点溢出; 0 和 1 不进入结果。

```

1 inline vector<int> segmented_primes(int l, int r)
2 {
3     assert(0 <= l && l <= r);
4     int limit = 0;
5     while (1LL * (limit + 1) * (limit + 1) <= r) limit++;
6     vector<char> composite(limit + 1);
7     vector<char> removed(1LL * r - l + 1);
8     for (int p = 2; p <= limit; p++)
9         if (!composite[p])
10            {
11                for (long long j = 1LL * p * p; j <= limit; j += p) composite[j] = true;
12                long long start = max(1LL * p * p, (l + 1LL * p - 1) / p * p);
13                for (long long j = start; j <= r; j += p) removed[j - l] = true;
14            }
15     vector<int> primes;
16     for (long long x = max(l, 2); x <= r; x++)
17         if (!removed[x - l]) primes.push_back(x);
18     return primes;
19 }
```

2.6 分解质因数

2.6.1 Pollard-Rho 因数分解

`factor(n)` 返回升序质因子（含重数）， $1 \leq n \leq \text{ULLONG_MAX}$ 。n=1 返回空数组； $n \geq 2$ 时 `size() == 1` 等价于 `n` 为素数，否则 `front()/back()` 分别取最小/最大质因子。依赖 `Mod64` 与 `Prime64`。公开使用 `factor`；`rho/split` 为内部辅助，`rho` 只接受大于 1 的合数，不能直接传素数。Las Vegas 随机运行时间，固定默认种子方便复现，可由构造参数更换；无确定性最坏时间保证。每次 `rho` 尝试最多 200000 轮，失败后换随机参数重试，不将失败当作素数。递归仅沿真因子拆分，深度 $O(\log n)$ 。

```

1 struct PollardRho
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     mt19937_64 rng;
6
7     PollardRho(ull seed = 712367821) : rng(seed) {}
8
9     ull rho(ull n)
10    {
11        if (n % 2 == 0) return 2;
12        for (;;)
13        {
14            ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
15            auto f = [&](ull v)
16            {
17                return (Mod64::mul(v, v, n) + u128(c)) % n;
18            };
19            // Retry bounded attempts; Las Vegas: only exact divisors returned.
20            for (int i = 0; i < 200000 && d == 1; i++)
21            {
22                x = f(x);
23                y = f(f(y));
24                d = gcd(x > y ? x - y : y - x, n);
25            }
26            if (1 < d && d < n) return d;
27        }
28    }
29
30    void split(ull n, vector<ull> &a)
31    {
32        if (n == 1) return;
33        if (Prime64::prime(n))
34        {
35            a.push_back(n);
36            return;
37        }
38        ull d = rho(n);
39        split(d, a);
40        split(n / d, a);
41    }

```

```

42
43     vector<ull> factor(ull n)
44     {
45         assert(n >= 1);
46         vector<ull> a;
47         split(n, a);
48         sort(a.begin(), a.end());
49         return a;
50     }
51 };

```

2.7 裴蜀定理 & 一次不定方程

2.7.1 扩展欧几里得与 Bézout 系数

`extended_gcd(a,b,x,y)` 约定 $0 \leq a, b \leq 2^{63}$, 参数和输出系数为 `int128`, 返回非负 `gcd` 并满足 $ax+by=\gcd(a,b)$ 。
`a=b=0` 返回 0, `x=1, y=0`; 有符号线性方程用 `linear_equation`。递归实现, 中间乘加为 `int128`; $O(\log(\min(a,b)+1))$ 时间与递归空间。

```

1  inline __int128_t extended_gcd(__int128_t a, __int128_t b, __int128_t &x, __int128_t &y)
2  {
3      using i128 = __int128_t;
4      if (!b)
5      {
6          x = 1;
7          y = 0;
8          return a;
9      }
10     i128 d = extended_gcd(b, a % b, y, x);
11     y -= a / b * x;
12     return d;
13 }

```

2.7.2 二元一次不定方程

`linear_equation(a,b,c,x,y)` 求 $ax+by=c$ 的任意整数特解。a、b、c 支持完整 long long, x、y 必须为 int128。无解返回 false 且 $x=y=0$; $a=b=c=0$ 返回 true、 $x=y=0$, 此时所有整数对都是解。其他有解情况, 令 $g=\gcd(|a|,|b|)$, 通解为 $(x+(b/g)t, y-(a/g)t)$ 。依赖 `extended_gcd`; 只需抄递归 `exgcd` 与本函数。时间、递归空间 $O(\log M)$, 其中 $M=\max(|a|,|b|)+2$; 不承诺特解最小或为正。

```

1 inline bool
2 linear_equation(long long a, long long b, long long c, __int128_t &x, __int128_t &y)
3 {
4     using i128 = __int128_t;
5     x = 0;
6     y = 0;
7     if (!a && !b) return c == 0;
8     i128 aa = a < 0 ? -i128(a) : i128(a);
9     i128 bb = b < 0 ? -i128(b) : i128(b);
10    i128 u, v;
11    i128 g = extended_gcd(aa, bb, u, v);
12    if (c % g) return false;
13    x = u * (c / g);
14    y = v * (c / g);
15    if (a < 0) x = -x;
16    if (b < 0) y = -y;
17    return true;
18 }
```

2.8 费马小定理 & 欧拉定理

相关应用: 公共指数与欧拉定理相关; 导航无独立 Carmichael 页, 不伪造叶节点。

2.8.1 Carmichael: 模乘法群的公共指数

`carmichael(n)` 返回 $\lambda(n)$, 即使所有与 n 互素的 a 都满足 a 的 $\lambda(n)$ 次幂模 n 为 1 的最小正指数; 约定 $\lambda(1)=1$ 。要求 $1 \leq n \leq \text{INT_MAX}$ 。试除分解后将各素数幂的指数取 lcm, 特别处理 2 的三次及以上幂。时间 $O(\sqrt{n})$, 额外空间 $O(1)$; 所有中间值不超过有符号 64 位范围, 结果不超过 n。该指数只适用于可逆底数, 不能据此直接对非互素底数的幂降指数。函数不依赖原根模块。

```

1 inline int carmichael(int n)
2 {
3     assert(n >= 1);
4     long long answer = 1;
5     for (int p = 2; p <= n / p; p++)
6         if (n % p == 0)
7             {
8                 int power = 1;
9                 while (n % p == 0)
10                     {
11                         n /= p;
```

```

12         power *= p;
13     }
14     int value = power / p * (p - 1);
15     if (p == 2 && power >= 8) value /= 2;
16     answer = answer / gcd(answer, (long long)value) * value;
17 }
18 if (n > 1) answer = answer / gcd(answer, (long long)(n - 1)) * (n - 1);
19 return answer;
20 }

```

2.9 模逆元

2.9.1 模整数类型的批量逆元

`batch_units(a)` 返回 `optional<vector<Z>>`，保持输入顺序；任一元素不可逆返回 `nullopt`，空输入返回空数组。`Z` 须可从 1 构造，支持乘法、复合乘法及 `try_inv`，且元素来自同一交换模整数环；目前可与 `mint<tag>` 直接组合。先累计前缀乘积，只对总乘积求一次逆，再倒序求各项逆元。总乘积可逆当且仅当所有因子可逆，因此合数模数也可使用。 $O(n)$ 次乘法加一次求逆，空间 $O(n)$ ，要求大小可表示为 `int`；建议传 `move(a)` 避免保留额外输入副本。返回前不能切换动态模数。现有 `long long` 版 `batch_inverse` 保留独立接口与历史证据，不由本函数替换。

```

1 template <class Z> optional<vector<Z>> batch_units(vector<Z> a)
2 {
3     int n = a.size();
4     vector<Z> pre(n + 1, Z(1));
5     for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * a[i];
6     auto x = pre[n].try_inv();
7     if (!x) return nullopt;
8     Z inv = *x;
9     for (int i = n - 1; i >= 0; i--)
10     {
11         Z old = a[i];
12         a[i] = inv * pre[i];
13         inv *= old;
14     }
15     return a;
16 }

```

2.9.2 任意正模数的单个逆元

`mod_inverse(a,m)` 支持完整 `long long a` 和正 `long long m`，将 `a` 规范化后用 `extended_gcd` 求逆。不可逆返回 -1，可逆返回 $0..m-1$ 内的代表；模数 1 返回 0。允许合数模数，不使用费马小定理。 $O(\log m)$ 时间与递归空间。

```

1 inline long long mod_inverse(long long a, long long m)
2 {
3     using i128 = __int128_t;
4     assert(m > 0);
5     a %= m;
6     if (a < 0) a += m;
7     i128 x, y;
8     if (extended_gcd(a, m, x, y) != 1) return -1;
9     return (x % m + m) % m;
10 }

```

2.9.3 任意数组的批量逆元

`batch_inverse(a,mod)` 返回与 `a` 同序的逆元向量。要求 `mod` 为正 `long long`，每个输入可为完整 `long long`；只要某一项与 `mod` 不互素，整个结果返回 `nullopt`，不输出部分逆元。空输入返回有效的空向量；模数 1 约定返回全零，与单个逆元接口一致。模数可以是合数。函数按值接收数组并归一化；不再需要原数组时传 `move(a)` 可避免输入副本。依赖 `mod_inverse` 和 `extended_gcd`。仅求一次逆元，时间 $O(n+\log \text{mod})$ ，空间 $O(n)$ ；128 位中间乘法防止溢出。

```

1 inline optional<vector<long long>> batch_inverse(vector<long long> a, long long mod)
2 {
3     assert(mod > 0);
4     int n = a.size();
5     vector<long long> prefix(n + 1, 1 % mod);
6     for (int i = 0; i < n; i++)
7     {
8         a[i] %= mod;
9         if (a[i] < 0) a[i] += mod;
10        prefix[i + 1] = __int128_t(prefix[i]) * a[i] % mod;
11    }
12    long long inverse = mod_inverse(prefix[n], mod);
13    if (inverse == -1) return nullopt;
14    for (int i = n - 1; i >= 0; i--)
15    {
16        long long value = a[i];
17        a[i] = __int128_t(inverse) * prefix[i] % mod;
18        inverse = __int128_t(inverse) * value % mod;
19    }
20    return a;
21 }
```

2.9.4 连续整数的线性逆元表

`inverse_table(n,p)` 返回长度 `n+1` 的 `int` 向量，位置 `i` 保存 `i` 模 `p` 的逆元， $1 \leq i \leq n$ 。位置 0 固定为 0，仅为占位，不表示零有逆元；`n=0` 返回单个占位项。要求 `p` 为素数、 $2 \leq p \leq \text{INT_MAX}$ 且 $0 \leq n < p$ ，不检测素性。时间和空间均 $O(n)$ ，中间乘法用 `long long`；独立可抄，不依赖其他模板。不能将同一递推直接用于合数模数，即使当前 `i` 与模数互素；一般模数使用扩展欧几里得或任意数组批量逆元。

```

1 inline vector<int> inverse_table(int n, int p)
2 {
3     assert(p >= 2 && 0 <= n && n < p);
4     vector<int> inverse(n + 1);
5     if (n) inverse[1] = 1;
6     for (int i = 2; i <= n; i++) inverse[i] = p - 1LL * (p / i) * inverse[p % i] % p;
7     return inverse;
8 }
```

2.10 线性同余方程

2.10.1 线性同余方程

`linear_congruence(a,b,m)` 求 $a*x+b\equiv 0 \pmod m$, 返回 {最小非负解, 最小正周期}, 无解返回 {-1,-1}。m 为正 long long, a、b 支持完整 long long。若 $g=\gcd(a,m)$ 整除 b, 周期为 m/g ; 包括 $a=0$ 和 $m=1$, 所有整数均满足时返回 {0,1}。先归一化 a, 并在 int128 中对 b 取相反数和乘法。依赖 `extended_gcd`; 递归时间、空间 $O(\log(m+1))$ 。注意本接口的常数项是 +b, 若题目写 $a*x\equiv c$, 应传 -c; c 为 `LLONG_MIN` 时须避免在 64 位中取负。

```
1 inline pair<long long, long long>
2 linear_congruence(long long a, long long b, long long m)
3 {
4     assert(m > 0);
5     using i128 = __int128_t;
6     a %= m;
7     if (a < 0) a += m;
8     i128 x, y;
9     i128 g = extended_gcd(a, m, x, y);
10    if (i128(b) % g) return {-1, -1};
11    i128 period = m / g;
12    x = x * (-i128(b) / g) % period;
13    if (x < 0) x += period;
14    return {(long long)x, (long long)period};
15 }
```

2.11 中国剩余定理

2.11.1 广义 CRT：逐项合并同余

`crt_merge(r,m,b,n)` 合并 $x\equiv r \pmod m$ 与 $x\equiv b \pmod n$, $m,n>0$, r、m 须为不同变量。从 $r=0$ 、 $m=1$ 开始逐项合并; 成功后 r 为最小非负解、m 为最小正周期。余数支持完整 long long。无解返回 false; 有解但新周期超过 `LLONG_MAX` 抛 `overflow_error`。两种失败都会先规范化 r, m 不变, 失败后不能视为完整解。依赖 `extended_gcd`, 中间量用 int128; $O(\log \min(m,n))$ 时间与递归空间。

```
1 inline bool crt_merge(long long &r, long long &m, long long b, long long n)
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5     assert(m > 0 && n > 0);
6     r %= m;
7     if (r < 0) r += m;
8     b %= n;
9     if (b < 0) b += n;
10    i128 x, y;
11    ll g = (ll)extended_gcd(m, n, x, y);
12    i128 diff = i128(b) - r;
13    if (diff % g) return false;
14    i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
15    if (mod > LLONG_MAX) throw overflow_error("CRT modulus");
16    r = (r + i128(m) * k) % mod;
17    m = (ll)mod;
18    return true;
19 }
```

2.11.2 Garner: 大周期 CRT 与混合进制

`garner_digits(b,moduli)` 返回按输入顺序排列的混合进制各位；`garner(b,moduli,target)` 返回最小非负 CRT 解对 `target` 的余数。两个输入向量等长，长度可为 0；模数为正且两两互素，允许模数 1，余数支持完整 `long long` 范围。`target` 为正 `long long`，与输入模数无需互素。输入不互素时抛 `invalid_argument`，即使方程有解也不适用；此时用扩展 CRT。空输入的规范解取 0。算法从不构造模数乘积，中间运算使用 `int128`。k 个模数耗时 $O(k^2 + k \log M)$ ，空间 $O(k)$ ；M 为最大输入模数。依赖 `mod_inverse` 和 `extended_gcd`。

```

1 // Pairwise coprime positive moduli. Digits use the input order.
2 inline vector<long long> garner_digits(const vector<long long> &b,
3                                     const vector<long long> &moduli)
4 {
5     using ll = long long;
6     using i128 = __int128_t;
7     int n = b.size();
8     assert(moduli.size() == b.size());
9     vector<ll> value(n), coefficient(n, 1), digit(n);
10    for (ll m : moduli) assert(m > 0);
11    for (int i = 0; i < n; i++)
12    {
13        ll m = moduli[i];
14        ll inverse = mod_inverse(coefficient[i], m);
15        if (inverse == -1) throw invalid_argument("Garner moduli are not coprime");
16        i128 delta = (i128(b[i]) - value[i]) % m;
17        if (delta < 0) delta += m;
18        digit[i] = delta * inverse % m;
19        for (int j = i + 1; j < n; j++)
20        {
21            value[j] = (value[j] + i128(coefficient[j]) * digit[i]) % moduli[j];
22            coefficient[j] = i128(coefficient[j]) * m % moduli[j];
23        }
24    }
25    return digit;
26 }

```

规范解对目标数取模

```

28 // Return the least nonnegative CRT solution modulo target, not all solutions.
29 inline long long
30 garner(const vector<long long> &b, const vector<long long> &moduli, long long target)
31 {
32     assert(target > 0);
33     auto digit = garner_digits(b, moduli);
34     long long answer = 0;
35     for (int i = (int)digit.size() - 1; i >= 0; i--)
36         answer = (__int128_t(answer) * moduli[i] + digit[i]) % target;
37     return answer;
38 }

```

2.12 卢卡斯定理

2.12.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p ，choose(n,k) 返回 $C(n,k)$ 模 p ； n 、 k 为完整 unsigned long long 范围， $k > n$ 返回 0， $C(0,0)=1$ 。init(p) 可更换模数并重新预处理；不检测素性，要求调用者保证 p 为素数且 $p \geq 2$ 。预处理时间 $O(p + \log p)$ ，动态版空间 $O(p)$ ；单次查询 $O(1 + \log_p(n+1))$ 。适合小素数模数，通常 $p \leq 10^6$ ；大素数不能因查询下标小就忽略这里整段 $O(p)$ 的预处理。fac、ifac 下标为 $0..p-1$ ，模数和表只读；中间乘法用 long long。合数模数请使用 exLucas。

```

1 struct Lucas
2 {
3     int mod;
4     vector<int> fac, ifac;
5
6     Lucas(int p) { init(p); }
7
8     void init(int p)
9     {
10         assert(p >= 2);
11         mod = p;
12         fac.resize(p);
13         ifac.resize(p);
14         fac[0] = 1;
15         for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
16         long long a = fac[p - 1], r = 1;
17         for (int e = p - 2; e; e >>= 1)
18         {
19             if (e & 1) r = r * a % p;
20             a = a * a % p;
21         }
22         ifac[p - 1] = r;
23         for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
24     }
25
26     int choose(unsigned long long n, unsigned long long k) const
27     {
28         if (k > n) return 0;
29         long long answer = 1;
30         while (n || k)
31         {
32             int a = n % mod, b = k % mod;
33             if (b > a) return 0;
34             answer = answer * fac[a] % mod;
35             answer = answer * ifac[b] % mod;
36             answer = answer * ifac[a - b] % mod;
37             n /= mod;
38             k /= mod;
39         }
40         return answer;
41     }
42 };

```

2.12.2 合数模数组合数 (扩展 Lucas)

ExLucas(mod) 预处理固定模数, choose(n,k) 返回 $C(n,k)$ 模 mod; n、k 为 unsigned long long, $k > n$ 返回 0, mod=1 返回 0。适合 $\text{mod} \leq 10^6$ 的合数或素数模数, 不能用于 10^9 级素数的整段预处理。分解 mod 为互素素数幂 $q=p^e$, 预处理每段内不含 p 因子的前缀积; unit(n,f) 去除 n! 中全部 p 因子, valuation 统计指数。分子分母指数之差不少于 e 时该项为零, 否则只对互素单位部分求逆, 最后 CRT 合并。预处理 $O(\sqrt{\text{mod}} + \sum q)$, 空间 $O(\sum (q+1))$; 每次查询 $O(\sum (\log_p(n+1) \cdot \log(n+1) + \log q))$, 求和遍历不同素因子。所有乘法用 long long; 同一对象可重复查询, factors、fac 与辅助函数视为内部数据。

```

1 struct ExLucas
2 {
3     using ull = unsigned long long;
4     using ll = long long;
5
6     struct Factor
7     {
8         int p, q, exponent, start, crt;
9     };
10
11     int mod;
12     vector<int> fac;
13     vector<Factor> factors;
14
15     static ll power(ll a, ull b, int m)
16     {
17         ll result = 1;
18         while (b)
19         {
20             if (b & 1) result = result * a % m;
21             a = a * a % m;
22             b >>= 1;
23         }
24         return result;
25     }
26
27     ExLucas(int m) : mod(m)
28     {
29         assert(m >= 1);
30         int x = m;
31         for (int p = 2; p <= x / p; p++)
32         {
33             if (x % p) continue;
34             int q = 1, e = 0;
35             do
36             {
37                 x /= p;
38                 q *= p;
39                 e++;
40             } while (x % p == 0);
41             add_factor(p, q, e);
42         }
43         if (x > 1) add_factor(x, x, 1);
44     }
45
46     void add_factor(int p, int q, int e)
47     {

```

```

48     int start = fac.size();
49     fac.push_back(1);
50     for (int i = 1; i < q; i++) fac.push_back(1ll(fac.back()) * (i % p ? i : 1) % q);
51     fac.push_back(fac.back());
52     int phi = q - q / p;
53     int crt = 1ll(mod / q) * power(mod / q % q, phi - 1, q) % mod;
54     factors.push_back({p, q, e, start, crt});
55 }
56
57 static ull valuation(ull n, int p)
58 {
59     ull result = 0;
60     while (n)
61     {
62         n /= p;
63         result += n;
64     }
65     return result;
66 }
67
68 ll unit(ull n, const Factor &f) const
69 {
70     ll result = 1;
71     while (n)
72     {
73         result = result * power(fac[f.start + f.q], n / f.q, f.q) % f.q;
74         result = result * fac[f.start + n % f.q] % f.q;
75         n /= f.p;
76     }
77     return result;
78 }

```

```

80 int choose(ull n, ull k) const
81 {
82     if (k > n || mod == 1) return 0;
83     ll answer = 0;
84     for (const auto &f : factors)
85     {
86         ull e = valuation(n, f.p) - valuation(k, f.p) - valuation(n - k, f.p);
87         if (e >= ull(f.exponent)) continue;
88         ll denominator = unit(k, f) * unit(n - k, f) % f.q;
89         ll residue =
90             unit(n, f) * power(denominator, f.q - f.q / f.p - 1, f.q) % f.q;
91         residue = residue * power(f.p, e, f.q) % f.q;
92         answer = (answer + residue * f.crt) % mod;
93     }
94     return answer;
95 }
96 };

```

2.13 二次剩余

2.13.1 素数模平方根 (Cipolla)

`mod_sqrt(n,p)` 返回 `vector<long long>`, 按升序列出全部不同根; 无解返回空向量, 零或模数 2 返回单根。要求 p 为素数, $2 \leq p \leq \text{INT_MAX}$, n 支持完整 `long long` 范围并在入口归一化。先用 Euler 判别; $p \equiv 3 \pmod{4}$ 时直接快速幂, 其余使用 Cipolla 二次扩域。函数内保存固定种子的伪随机生成器; 随机性影响寻找非剩余的尝试次数, 不影响结果, 通常期望 $O(\log p)$ 时间、 $O(1)$ 辅助空间。乘法用 `long long`, 扩域实部的三因子乘积中间先取模; 指数中的 $p+1$ 先转 `long long`, 避免 `INT_MAX` 溢出。此版本不处理合数模数及 64 位大素数, 也不用于浮点开方。

```

1 inline vector<long long> mod_sqrt(long long n, int p)
2 {
3     using ll = long long;
4     assert(p >= 2);
5     n = (n % p + p) % p;
6     if (n == 0 || p == 2) return {n};
7     auto power = [p](ll a, ll b)
8     {
9         ll result = 1;
10        while (b)
11        {
12            if (b & 1) result = result * a % p;
13            a = a * a % p;
14            b >>= 1;
15        }
16        return result;
17    };
18    if (power(n, (p - 1) / 2) != 1) return {};
19    ll answer;
20    if (p % 4 == 3)
21        answer = power(n, (ll(p) + 1) / 4);
22    else
23    {
24        static mt19937_64 rng(712367);
25        ll a, w;
26        do
27        {
28            a = rng() % p;
29            w = (a * a % p - n + p) % p;
30        } while (power(w, (p - 1) / 2) != p - 1);
31        using P = pair<ll, ll>;
32        auto mul = [p, w](P x, P y)
33        {
34            ll real = (x.first * y.first + x.second * y.second % p * w) % p;
35            ll imag = (x.first * y.second + x.second * y.first) % p;
36            return P{real, imag};
37        };
38        P result{1, 0}, base{a, 1};
39        ll exponent = (ll(p) + 1) / 2;
40        while (exponent)
41        {
42            if (exponent & 1) result = mul(result, base);

```

```

43     base = mul(base, base);
44     exponent >>= 1;
45 }
46 answer = result.first;
47 }
48 ll other = p - answer;
49 if (answer > other) swap(answer, other);
50 return {answer, other};
51 }

```

2.14 阶 & 原根

2.14.1 乘法阶与原根

构造 `PrimitiveRoot(n)`，要求 $2 \leq n \leq \text{INT_MAX}$ ；`init(n)` 可重建。`order(a)` 返回 a 模 n 的乘法阶， a 不可逆时返回 0；`is_root(a)` 判断该剩余类是否为原根， a 可为完整 `long long`。`minimum()` 返回 $1..n-1$ 内最小原根，无原根返回 -1；`all()` 返回升序的全部原根，无原根返回空向量。`mod`、`phi`、`exists` 只读；`factors` 为 `phi` 的不同素因子，辅助 `factor` 为试除分解。依赖 `Mod64::power`。初始化 $O(\sqrt{n})$ ，求阶 $O(\log^2 n)$ ，原根判定 $O((1+\omega(\phi))\log n)$ 。若最小原根为 G ，顺序枚举最坏检查 G 个候选，不能假定 G 总是常数。枚举全部另需 $O(\phi \log \phi + c \log c)$ 时间、 $O(c)$ 空间， c 为原根数；大模数只求最小原根或阶时，不应调用 `all`。

```

1 struct PrimitiveRoot
2 {
3     int mod, phi;
4     bool exists;
5     vector<int> factors;
6
7     static vector<int> factor(int n)
8     {
9         vector<int> result;
10        for (int p = 2; p <= n / p; p++)
11            if (n % p == 0)
12            {
13                result.push_back(p);
14                while (n % p == 0) n /= p;
15            }
16        if (n > 1) result.push_back(n);
17        return result;
18    }
19
20    PrimitiveRoot(int n) { init(n); }

```

```
22 void init(int n)
23 {
24     assert(n >= 2);
25     mod = n;
26     phi = n;
27     for (int p : factor(n)) phi -= phi / p;
28     int odd = n;
29     if (odd % 2 == 0) odd /= 2;
30     exists = n == 2 || n == 4 || (odd % 2 && factor(odd).size() == 1);
31     factors = factor(phi);
32 }
33
34 int order(long long a) const
35 {
36     a %= mod;
37     if (a < 0) a += mod;
38     if (gcd(a, (long long)mod) != 1) return 0;
39     int result = phi;
40     for (int p : factors)
41         while (result % p == 0 && Mod64::power(a, result / p, mod) == 1)
42             result /= p;
43     return result;
44 }
```

```
46     bool is_root(long long a) const
47     {
48         if (!exists) return false;
49         a %= mod;
50         if (a < 0) a += mod;
51         if (gcd(a, (long long)mod) != 1) return false;
52         for (int p : factors)
53             if (Mod64::power(a, phi / p, mod) == 1) return false;
54         return true;
55     }
56
57     int minimum() const
58     {
59         if (!exists) return -1;
60         for (int g = 1; g < mod; g++)
61             if (is_root(g)) return g;
62         return -1;
63     }
64
65     vector<int> all() const
66     {
67         int g = minimum();
68         vector<int> roots;
69         if (g == -1) return roots;
70         long long value = 1;
71         for (int k = 1; k <= phi; k++)
72         {
73             value = value * g % mod;
74             if (gcd(k, phi) == 1) roots.push_back(value);
75         }
76         sort(roots.begin(), roots.end());
77         return roots;
78     }
79 };
```

2.15 离散对数

2.15.1 扩展 BSGS

依赖 number_theory.hpp 的求逆与快速幂。solve(a,b,m) 返回最小 $x \geq 0$ 使 $a^x \equiv b \pmod{m}$ ，无解返回 -1； $1 \leq m \leq 10^{12}$ ，a、b 可为完整 long long 范围，入口归一化，约定 $0^0=1$ 。先剥离 gcd 并逐次检查较小指数，再在互质剩余问题上 BSGS。baby 表对同余幂保留最小指数 j，按递增大步 i 检查区间，因此首个命中即最小解。单次期望时间 $O(\sqrt{m} + \log m)$ ，空间 $O(\sqrt{m})$ ，哈希表先 reserve；基于 unordered_map 的界为期望界。每次调用独立建表；底数 1 及零指数解直接返回。

```

1 struct DiscreteLog
2 {
3     using ll = long long;
4
5     // 1<=m<=1e12; Expected O(sqrt(m)) time, O(sqrt(m)) storage. Smallest x>=0, or -1.
6     static ll solve(ll a, ll b, ll m)
7     {
8         assert(m >= 1 && m <= 1000000000000LL);
9         a = (a % m + m) % m;
10        b = (b % m + m) % m;
11        if (m == 1 || b == 1) return 0;
12        if (a == 1) return -1;
13        ll offset = 0, k = 1;
14        for (ll g; (g = gcd(a, m)) > 1;)
15        {
16            if (b == k) return offset;
17            if (b % g) return -1;
18            b /= g;
19            m /= g;
20            k = (__int128)k * (a / g) % m;
21            ++offset;
22        }
23        ll target = (__int128)b * mod_inverse(k, m) % m;
24        ll step = sqrtl(m) + 1;
25        unordered_map<ll, ll> baby;
26        baby.reserve(step);
27        ll cur = 1 % m;
28        for (ll j = 0; j < step; j++)
29        {
30            if (!baby.count(cur)) baby[cur] = j;
31            cur = (__int128)cur * a % m;
32        }
33        ll inv = mod_inverse(Mod64::power(a, step, m), m);
34        cur = target;
35        for (ll i = 0; i <= step; i++)
36        {
37            auto it = baby.find(cur);
38            if (it != baby.end()) return offset + i * step + it->second;
39            cur = (__int128)cur * inv % m;
40        }
41        return -1;
42    }
43 };

```

2.16 高次剩余 & 单位根

2.16.1 素数模高次剩余：全部根的紧凑表示

$\text{solve}(a, k, p, g)$ 求 $x^k \equiv a \pmod{p}$; p 必须是 2 到 $1e12$ 的素数, g 是模 p 原根, k 为正 unsigned long long, a 支持完整 signed long long。无解返回 nullopt; 否则得到 first、ratio、count, 全部解为 $\text{first} * \text{ratio}^i \pmod{p}$, $0 \leq i < \text{count}$, 无重复; first 不保证最小。 $a \equiv 0$ 时仅有根 0, 返回 $\{0, 1, 1\}$ 。枚举时每次乘 ratio, 乘法先转 int128; 无需展开全部根即可读取解数。给定原根后, 期望 $O(\sqrt{p})$ 时间与空间, 依赖 DiscreteLog、linear_congruence 及模快速幂。不验证 p 的素性或 g 的阶; 合数模数不在此接口范围内。

```

1 struct KthResidue
2 {
3     using ll = long long;
4     ll first, ratio, count;
5
6     // Prime p <= 1e12; g is a primitive root of p; k > 0.
7     // All roots: first * ratio^i mod p, 0 <= i < count.
8     static optional<KthResidue> solve(ll a, unsigned long long k, ll p, ll g)
9     {
10         assert(p >= 2 && p <= 1000000000000LL && g >= 1 && g < p && k > 0);
11         a %= p;
12         if (a < 0) a += p;
13         if (a == 0) return KthResidue{0, 1, 1};
14         ll logarithm = DiscreteLog::solve(g, a, p);
15         if (logarithm < 0) return nullopt;
16         auto [exponent, period] = linear_congruence(k % (p - 1), -logarithm, p - 1);
17         if (exponent < 0) return nullopt;
18         ll first = Mod64::power(g, exponent, p);
19         ll ratio = Mod64::power(g, period, p);
20         return KthResidue{first, ratio, (p - 1) / period};
21     }
22 };

```

2.16.2 素数幂模高次剩余：含非互素被开方数

$\text{solve}(a,k,p,e,g)$ 求 $x^k \equiv a \pmod{p^e}$ 。p 是素数， $e \geq 1$ ， $p^e \leq 1e12$ ，k 为正 unsigned long long；a 为 signed long long。奇素数须提供模 p^e 的原根 g；p=2 时不使用 g。无解返回 nullopt，否则 size() 返回解数，get(i,j,lift) 返回指定根，范围是 count[0]、count[1]、lifts，全部组合无重复且无遗漏。字段 first、ratio[2]、step 描述乘法根族与加法提升； $\text{step} \cdot \text{lifts} = p^e$ 。原根与素性由调用者保证。期望 $O(\sqrt{p^e})$ 时间和空间；单次 get 为 $O(\log(p^e))$ ，不强制展开全部根。依赖离散对数、线性同余及模幂。

```

1 struct PrimePowerRoots
2 {
3     using ll = long long;
4     ll first = 0, step = 1, lifts = 1;
5     array<ll, 2> ratio{1, 1}, count{1, 1};
6
7     ll size() const { return count[0] * count[1] * lifts; }
8
9     ll get(ll i, ll j, ll lift) const
10    {
11        assert(i >= 0 && i < count[0] && j >= 0 && j < count[1]);
12        assert(lift >= 0 && lift < lifts);
13        ll x = (__int128)first * Mod64::power(ratio[0], i, step) % step;
14        x = (__int128)x * Mod64::power(ratio[1], j, step) % step;
15        return x + lift * step;
16    }
17
18    // p prime, 1 <= e, p^e <= 1e12, k > 0.
19    // For odd p, g must be a primitive root modulo p^e. For p=2, g is unused.

```

接上页同一结构体（第三页继续同一函数）:

```
20 static optional<PrimePowerRoots>
21 solve(ll a, unsigned long long k, ll p, int e, ll g = 0)
22 {
23     assert(p >= 2 && p <= 10000000000000LL && e >= 1 && e <= 39 && k > 0);
24     array<ll, 40> power{1};
25     for (int i = 1; i <= e; i++)
26     {
27         assert(power[i - 1] <= 10000000000000LL / p);
28         power[i] = power[i - 1] * p;
29     }
30     ll mod = power[e];
31     a %= mod;
32     if (a < 0) a += mod;
33     PrimePowerRoots answer;
34     if (a == 0)
35     {
36         answer.step = power[(e - 1) / k + 1];
37         answer.lifts = mod / answer.step;
38         return answer;
39     }
40     int t = 0;
41     while (a % p == 0)
42     {
43         a /= p;
44         ++t;
45     }
46     if (t % k) return nullopt;
```

接上页同一结构体（第三页继续同一函数）：

```

47     ll scale = power[t / k], q = power[e - t];
48     answer.first = 1;
49     answer.step = scale * q;
50     answer.lifts = mod / answer.step;
51     auto cyclic = [&](ll base, ll target, ll order, int slot)
52     {
53         ll logarithm = DiscreteLog::solve(base, target, q);
54         if (logarithm < 0) return false;
55         auto [x, period] = linear_congruence(k % order, -logarithm, order);
56         if (x < 0) return false;
57         answer.first = (__int128)answer.first * Mod64::power(base, x, q) % q;
58         answer.ratio[slot] = Mod64::power(base, period, q);
59         answer.count[slot] = order / period;
60         return true;
61     };
62     if (p != 2)
63     {
64         assert(g >= 1 && g < mod);
65         if (!cyclic(g % q, a, q / p * (p - 1), 0)) return nullopt;
66     }
67     else if (q >= 4)
68     {
69         int sign = a % 4 == 3;
70         auto [x, period] = linear_congruence(k % 2, -sign, 2);
71         if (x < 0) return nullopt;
72         answer.first = x ? q - 1 : 1;
73         answer.ratio[0] = period == 1 ? q - 1 : 1;
74         answer.count[0] = 2 / period;
75         if (q >= 8 && !cyclic(5, sign ? q - a : a, q / 4, 1)) return nullopt;
76     }
77     answer.first *= scale;
78     return answer;
79 }
80 };

```

2.16.3 合数求根准备：分解模数与素数幂原根

`root_factors(M, rho)` 为 $1 \leq M \leq 1e12$ 生成完整的 (p, e, g) 列表，可直接传入 `CompositeRoots`。rho 是可复用的 PollardRho 对象。按素数递增返回； $p=2$ 时 $g=0$ ，其余 g 为模 p^e 的最小正原根。先分解 M ，再分解各个欧拉函数值，逐个检查原根候选的阶。 $M=1$ 返回空列表。分解使用随机重试；原根搜索耗时取决于候选数，不包含 BSGS。

```

1 inline vector<tuple<long long, int, long long>> root_factors(long long mod,
2                                     PollardRho &rho)
3 {
4     assert(mod >= 1 && mod <= 10000000000000LL);
5     auto primes = rho.factor(mod);
6     vector<tuple<long long, int, long long>> answer;
7     for (int i = 0; i < (int)primes.size();)
8     {
9         long long p = primes[i], power = 1;
10        int e = 0;
11        while (i < (int)primes.size() && primes[i] == (unsigned long long)p)
12        {
13            power *= p;
14            ++e;
15            ++i;
16        }
17        long long g = 0;
18        if (p != 2)
19        {
20            long long phi = power / p * (p - 1);
21            auto factors = rho.factor(phi);
22            factors.erase(unique(factors.begin(), factors.end()), factors.end());
23            auto primitive = [&](long long candidate)
24            {
25                if (candidate % p == 0) return false;
26                for (auto q : factors)
27                    if (Mod64::power(candidate, phi / q, power) == 1) return false;
28                return true;
29            };
30            g = 2;
31            while (!primitive(g)) ++g;
32        }
33        answer.push_back({p, e, g});
34    }
35    return answer;
36 }

```

2.16.4 一般合数模高次剩余：CRT 组合与按索引取根

`solve(a,k,factors)` 求 $x^k \equiv a \pmod{M}$, `factors` 是完整且素数互异的 (p,e,g) 列表, M 为所有 p^e 的乘积, $M \leq 1e12$, k 为正 unsigned long long, a 为 signed long long。奇素数的 g 是模 p^e 原根; $p=2$ 时填 0。无解返回 `nullopt`, 否则 `mod=M`, `total` 为解数, `get(index)` 在 $0 \leq \text{index} < \text{total}$ 上取出每个根恰好一次, 不按数值排序。空因子列表表示模 1, 唯一标准代表为 0。依赖 `PrimePowerRoots` 和模逆元; 列表接口不做分解; 重载 `solve(a,k,M,rho)` 自动分解并寻找原根, `rho` 为 PollardRho。重复同模数宜复用因子列表。预处理的主要成本是各素数幂的 BSGS; `get` 为 $O(F \log M)$, F 为不同素因子数。无需展开全部根, `parts` 和 `weight` 保存局部根族及 CRT 系数。

```

1 struct CompositeRoots
2 {
3     using ll = long long;
4     ll mod = 1, total = 1;
5     vector<PrimePowerRoots> parts;
6     vector<ll> weight;
7
8     ll get(ll index) const
9     {
10         assert(index >= 0 && index < total);
11         ll answer = 0;
12         for (int t = 0; t < (int)parts.size(); t++)
13         {
14             const auto &p = parts[t];
15             ll local = index % p.size();
16             index /= p.size();
17             ll lift = local % p.lifts;
18             local /= p.lifts;
19             ll j = local % p.count[1], i = local / p.count[1];
20             ll root = p.get(i, j, lift);
21             answer = (answer + (__int128)root * weight[t]) % mod;
22         }
23         return answer;
24     }
25
26     // Factors are distinct primes with positive exponents: (p, e, g).
27     // Odd-prime g is a primitive root modulo p^e; for p=2 use g=0.

```

局部求根与 CRT 系数（接上页同一结构体）：

```

28     static optional<CompositeRoots>
29     solve(ll a, unsigned long long k, const vector<tuple<ll, int, ll>> &factors)
30     {
31         assert(k > 0);
32         CompositeRoots answer;
33         for (auto [p, e, g] : factors)
34         {
35             auto roots = PrimePowerRoots::solve(a, k, p, e, g);
36             if (!roots) return nullopt;
37             ll m = roots->step * roots->lifts;
38             assert(gcd(answer.mod, m) == 1);
39             assert(answer.mod <= 1000000000000LL / m);
40             answer.mod *= m;
41             answer.total += roots->size();
42             answer.parts.push_back(*roots);
43         }
44         for (const auto &p : answer.parts)
45         {
46             ll m = p.step * p.lifts, rest = answer.mod / m;
47             ll inverse = mod_inverse(rest % m, m);
48             answer.weight.push_back((__int128)rest * inverse % answer.mod);
49         }
50         return answer;
51     }
52
53     static optional<CompositeRoots>
54     solve(ll a, unsigned long long k, ll mod, PollardRho &rho)
55     {
56         return solve(a, k, root_factors(mod, rho));
57     }
58 };

```

2.17 莫比乌斯反演

2.17.1 矩形 GCD 计数

构造 CoprimePairs(n) 预处理 mu 前缀和, $0 \leq n < \text{INT_MAX}$, $O(n)$ 时间与空间; 依赖 LinearSieve。count(a,b) 求 $[1,a] \times [1,b]$ 的互质数对, a,b 为非负 int, 要求 $\min(a,b) \leq n$, 任一为 0 返回 0。rectangle(a,b,c,d,k) 求 $[a,b] \times [c,d]$ 中 $\gcd(x,y)=k$ 的数对, 要求 $1 \leq a \leq b$, $1 \leq c \leq d$, $k \geq 1$, $\min(b/k, d/k) \leq n$; 闭区间容斥。返回 long long, 中间分块累加用 int128。单次前缀查询 $O(\sqrt{a} + \sqrt{b})$, 额外空间 $O(1)$ 。sum 为 mu 前缀和, 只读。

```

1 struct CoprimePairs
2 {
3     vector<int> sum;
4
5     CoprimePairs(int n) : sum(LinearSieve(n).mu)
6     {
7         for (int i = 1; i <= n; i++) sum[i] += sum[i - 1];
8     }
9
10    long long count(int a, int b) const
11    {
12        assert(a >= 0 && b >= 0 && min(a, b) < (int)sum.size());
13        __int128_t answer = 0;
14        for (int l = 1, r; l <= min(a, b); l = r + 1)
15        {
16            int x = a / l;
17            int y = b / l;
18            r = min(a / x, b / y);
19            answer += __int128_t(sum[r] - sum[l - 1]) * x * y;
20        }
21        return (long long)answer;
22    }
23
24    long long rectangle(int a, int b, int c, int d, int k) const
25    {
26        assert(1 <= a && a <= b && 1 <= c && c <= d && k >= 1);
27        return count(b / k, d / k) - count((a - 1) / k, d / k) -
28            count(b / k, (c - 1) / k) + count((a - 1) / k, (c - 1) / k);
29    }
30 };

```

2.18 狄利克雷双曲线法 & 杜教筛

2.18.1 杜教筛

构造预筛上限 $1 \leq B < \text{INT_MAX}$ ，内存须可用； $\text{mertens}(n)$ 返回 μ 的前缀和， $\text{totient_sum}(n)$ 返回 φ 的前缀和 (int128)。约定 $0 \leq n \leq 10^{11}$ ， $n=0$ 返回 0。按整除分块递归并写入 mmu/mphi 缓存；重复查询复用结果，清空缓存可重算，不可修改 $\text{pmu}/\text{pphi}/\text{limit}$ 。单次最大参数 N 的常用均摊分析为 $O(B+N/\sqrt{B})$ （哈希按期望常数计算）， B 约 $N^{2/3}$ 时为 $O(N^{2/3})$ ；小 B 不构成大输入性能保证。多次不同 N 可能新增缓存，空间随实际缓存增长；递归深度 $O(\log N)$ 。

```

1 struct DuJiao
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int limit;
6     vector<ll> pmu, pphi;
7     unordered_map<ll, ll> mmu;
8     unordered_map<ll, I> mphi;
9
10    DuJiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
11    {
12        assert(limit >= 1);
13        LinearSieve s(limit);
14        for (int i = 1; i <= limit; i++)
15        {
16            pmu[i] = pmu[i - 1] + s.mu[i];
17            pphi[i] = pphi[i - 1] + s.phi[i];
18        }
19    }
20
21    ll mertens(ll n)
22    {
23        if (n <= limit) return pmu[n];
24        if (mmu.count(n)) return mmu[n];
25        ll ans = 1;
26        for (ll l = 2, r; l <= n; l = r + 1)
27        {
28            r = n / (n / l);
29            ans -= (r - l + 1) * mertens(n / l);
30        }
31        return mmu[n] = ans;
32    }
33
34    I totient_sum(ll n)
35    {
36        if (n <= limit) return pphi[n];
37        if (mphi.count(n)) return mphi[n];
38        I ans = I(n) * (n + 1) / 2;
39        for (ll l = 2, r; l <= n; l = r + 1)
40        {
41            r = n / (n / l);
42            ans -= I(r - l + 1) * totient_sum(n / l);
43        }
44        return mphi[n] = ans;
45    }
46 };

```

2.19 类欧几里德算法

2.19.1 有符号整除和：类欧几里德

`floor_sum(n,m,a,b)` 返回 $i=0..n-1$ 的 $\text{floor}((ai+b)/m)$ 之和，结果为 `int128`。要求 $0 \leq n \leq 10^9$ 、 $1 \leq m \leq 10^9$ ，`a`、`b` 支持完整 `long long`；`n=0` 返回 0。先按数学向下取整规范化负系数，再迭代交换分母和余数；不能把 C++ 负数截零除法当作 `floor`。 $O(\log m)$ 时间、 $O(1)$ 空间，不依赖完整数论模块。

```

1 inline __int128_t floor_sum(long long n, long long m, long long a, long long b)
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5     assert(n >= 0 && m > 0);
6     i128 ans = 0;
7     auto norm = [&](ll &v)
8     {
9         ll q = v / m;
10        if (v % m < 0) --q;
11        v = (ll)(i128(v) - i128(q) * m);
12        return q;
13    };
14    ans += i128(norm(a)) * n * (n - 1) / 2;
15    ans += i128(norm(b)) * n;
16    while (true)
17    {
18        ans += i128(n) * (n - 1) / 2 * (a / m);
19        a %= m;
20        ans += i128(n) * (b / m);
21        b %= m;
22        i128 y = i128(a) * n + b;
23        if (y < m) break;
24        n = (ll)(y / m);
25        b = (ll)(y % m);
26        swap(a, m);
27    }
28    return ans;
29 }
```

2.19.2 整除和、带权和与平方和

$\text{floor_moments}(n, m, a, b)$ 对 $i=0..n-1$, 令 $y_i = \text{floor}((a*i+b)/m)$, 返回 $\{\sum y_i, \sum i*y_i, \sum y_i^2\}$, 元素为 $\text{ModInt}\langle 998244353 \rangle$, 取整数用 $.v$ 。 $0 \leq n \leq 1000000001$, $1 \leq m \leq 10^9$; a, b 支持完整 long long 。 $n=0$ 返回全零。时间与递归空间 $O(\log(m+1))$; 模数固定, 499122177 与 166374059 分别是 2、6 的逆元。P5170 输入上限为闭区间, 驱动传 $n+1$, 输出索引顺序 0、2、1。

```

1 inline array<ModInt<998244353>, 3>
2 floor_moments(long long n, long long m, long long a, long long b)
3 {
4     using Z = ModInt<998244353>;
5     assert(n >= 0 && n <= 1000000001 && m >= 1 && m <= 1000000000);
6     if (!n) return {0, 0, 0};
7     auto normalize = [&](long long &v)
8     {
9         long long q = v / m;
10        if (v % m < 0) q--;
11        v = (__int128_t(v) - __int128_t(q) * m);
12        return Z(q);
13    };
14    Z x = normalize(a);
15    Z y = normalize(b);
16    Z count = n;
17    Z s1 = count * Z(n - 1) * Z(499122177);
18    Z s2 = count * Z(n - 1) * Z(2 * n - 1) * Z(166374059);
19    Z f = 0, g = 0, h = 0;
20    if (a)
21    {
22        long long height = (a * (n - 1) + b) / m;
23        auto [u, v, w] = floor_moments(height, a, m, m + a - b - 1);
24        Z t = height;
25        f = count * t - u;
26        g = t * s1 - (w - u) * Z(499122177);
27        h = count * t * t - Z(2) * v - u;
28    }
29    return {f + x * s1 + y * count,
30            g + x * s2 + y * s1,
31            h + x * x * s2 + y * y * count + Z(2) * x * y * s1 + Z(2) * x * g +
32            Z(2) * y * f};
33 }
```

第三章 多项式与生成函数

3.1 快速数论变换

3.1.1 可指定素数与原根的 NTT 卷积

声明 `NttConvolution<mod, primitive>`, `primitive` 默认 3。`mod` 必须为 `int` 范围内的素数, `primitive` 是模 `mod` 原根; 不在接口内验证素性或阶。`max_size` 是 `mod-1` 的最大二次幂因子。`ntt(a,invert)` 原地变换, 长度须为正二次幂且不超过 `max_size`; `invert=true` 为逆变换。`multiply(a,b)` 返回普通卷积, 任一为空则为空; 非空时长度和减一不超过 `max_size`。元素类型 `Z`, 数组类型 `Poly`。 $O(n \log n)$ 时间、 $O(n)$ 空间。998244353 和 167772161 均可用原根 3。

```
1 template <int mod, int primitive = 3> struct NttConvolution
2 {
3     static_assert(mod >= 2 && primitive >= 1 && primitive < mod);
4     static constexpr int max_size = (mod - 1) & -(mod - 1);
5     using Z = ModInt<mod>;
6     using Poly = vector<Z>;
7
8     // mod is prime, primitive is a primitive root modulo mod.
9     static void ntt(Poly &a, bool invert = false)
10    {
11        int n = a.size();
12        assert(n > 0 && (n & (n - 1)) == 0 && n <= max_size);
13        for (int i = 1, j = 0; i < n; i++)
14        {
15            int bit = n >> 1;
16            for (; j & bit; bit >>= 1) j ^= bit;
17            j ^= bit;
18            if (i < j) swap(a[i], a[j]);
19        }
```

蝶形变换与卷积（接上页同一结构体与函数）：

```

20     for (int half = 1; half < n; half *= 2)
21     {
22         int len = half * 2;
23         Z step = Z(primitive).pow((mod - 1) / len);
24         if (invert) step = step.inv();
25         for (int i = 0; i < n; i += len)
26         {
27             Z w = 1;
28             for (int j = 0; j < half; j++)
29             {
30                 Z u = a[i + j], v = a[i + j + half] * w;
31                 a[i + j] = u + v;
32                 a[i + j + half] = u - v;
33                 w = w * step;
34             }
35         }
36     }
37     if (invert)
38     {
39         Z inverse = Z(n).inv();
40         for (auto &x : a) x = x * inverse;
41     }
42 }
43
44 static Poly multiply(Poly a, Poly b)
45 {
46     if (a.empty() || b.empty()) return {};
47     assert(a.size() + b.size() - 1 <= max_size);
48     int size = a.size() + b.size() - 1, n = 1;
49     while (n < size) n *= 2;
50     a.resize(n);
51     b.resize(n);
52     ntt(a);
53     ntt(b);
54     for (int i = 0; i < n; i++) a[i] = a[i] * b[i];
55     ntt(a, true);
56     a.resize(size);
57     return a;
58 }
59 };

```

3.1.2 精确有符号整数卷积

`convolution_i64(a,b)` 返回 signed64 整数卷积，空输入返回空。非空结果长度 $\leq 2^{24}$ ，所有真实系数必须在 signed64 范围内。依赖 NttConvolution，三个素数模卷积后用 int128 CRT 重构中心代表。断言不能检测任意越界结果的模数别名。O(L log L) 时间、O(L) 空间，L 为补齐的二次幂长度。负数与极值已通过 `cpp_int` 参实验证；P3803 最大次数驱动本地通过，线上性能待验证。

```

1 template <int p>
2 vector<int> convolution_residue(const vector<long long> &a, const vector<long long> &b)
3 {
4     using N = NttConvolution<p>;
5     typename N::Poly x(a.begin(), a.end()), y(b.begin(), b.end());
6     auto c = N::multiply(move(x), move(y));
7     vector<int> r(c.size());
8     for (int i = 0; i < (int)c.size(); i++) r[i] = c[i].v;
9     return r;
10 }
11
12 inline vector<long long> convolution_i64(const vector<long long> &a,
13                                         const vector<long long> &b)
14 {
15     if (a.empty() || b.empty()) return {};
16     assert(a.size() <= (1 << 24) && b.size() <= (1 << 24));
17     assert(a.size() + b.size() - 1 <= (1 << 24));
18     const long long p = 167772161, q = 469762049, r = 1224736769;
19     const long long pq = p * q;
20     const __int128 mod = (__int128)pq * r;
21     static const long long ip = ModInt<q>(p).inv().v;
22     static const long long ipq = ModInt<r>(pq).inv().v;
23     auto x = convolution_residue<p>(a, b);
24     auto y = convolution_residue<q>(a, b);
25     auto z = convolution_residue<r>(a, b);
26     vector<long long> c(x.size());
27     for (int i = 0; i < (int)c.size(); i++)
28     {
29         long long t = (y[i] - (long long)x[i] + q) % q * ip % q;
30         long long v = x[i] + p * t;
31         long long u = (z[i] - v % r + r) % r * ipq % r;
32         __int128 value = v + (__int128)pq * u;
33         if (value > mod / 2) value -= mod;
34         assert(LLONG_MIN <= value && value <= LLONG_MAX);
35         c[i] = (long long)value;
36     }
37     return c;
38 }

```

3.2 快速沃尔什变换

3.2.1 按位 OR、AND、XOR 卷积

SetConvolution<mod=998244353>; Z 为 ModInt<mod>, Poly 为 vector<Z>。数组长度 N 必须是正的 2 次幂且 $\leq \text{INT_MAX}$, 表示固定二进制位数的全集, 长度 1 合法。transform(a,op,invert=false) 原地变换, op 只能为 |、&、^; | 为子集和, & 为超集和, ^ 为 Walsh 变换。multiply(a,b,op) 返回 $\sum a[i]b[j]$ (满足 $i \text{ op } j = s$), a、b 长度必须相同。mod 可为 ≥ 2 的 int 范围合数; XOR 要求 mod 为奇数, 逆 2 用 $(\text{mod}+1)/2$, 不使用费马逆元。 $O(N \log N)$ 时间, 原地变换 $O(1)$ 额外空间, 卷积 $O(N)$ 空间。

```

1 template <int mod = 998244353> struct SetConvolution
2 {
3     static_assert(mod >= 2);
4     using Z = ModInt<mod>;
5     using Poly = vector<Z>;
6
7     static void transform(Poly &a, char op, bool invert = false)
8     {
9         assert(!a.empty() && a.size() <= INT_MAX);
10        int n = a.size();
11        assert((n & (n - 1)) == 0);
12        assert(op == '|' || op == '&' || op == '^');
13        assert(op != '^' || mod % 2 == 1);
14        Z half = (mod + 1LL) / 2;
15        for (int len = 1; len < n; len *= 2)
16            for (int i = 0; i < n; i += len * 2)
17                for (int j = 0; j < len; j++)
18                    {
19                        Z &u = a[i + j], &v = a[i + j + len];
20                        if (op == '|')
21                            v = invert ? v - u : v + u;
22                        else if (op == '&')
23                            u = invert ? u - v : u + v;
24                        else
25                            {
26                                Z x = u, y = v;
27                                u = x + y;
28                                v = x - y;
29                                if (invert)
30                                    {
31                                        u = u * half;
32                                        v = v * half;
33                                    }
34                            }
35                    }
36    }

```

点乘并逆变换（接上页同一结构体）：

```

38 static Poly multiply(Poly a, Poly b, char op)
39 {
40     assert(a.size() == b.size());
41     transform(a, op);
42     transform(b, op);
43     for (int i = 0; i < int(a.size()); i++) a[i] = a[i] * b[i];
44     transform(a, op, true);
45     return a;
46 }
47 };

```

3.2.2 不相交子集卷积

`subset_convolution<mod=998244353>(a,b)` 返回 $c[s] = \sum a[t]b[s \text{ xor } t]$ ，求和枚举 $t \subseteq s$ ，等价于 $i|j=s$ 且 $i \& j=0$ 。依赖 `SetConvolution`，只使用 OR 变换；mod 为 ≥ 2 的 int 范围整数，允许偶数与合数。两个数组长度相同、非空， $N=2^m \leq \text{INT_MAX}$ ；长度 1 的结果为 $a[0]b[0]$ 。按 popcount 分层，层间普通卷积再逆 OR 变换，保留总层数等于目标集合大小的项。 $O(m^2 2^m)$ 时间、 $O(m 2^m)$ 空间；两组分层数组约占 $8(m+1)2^m$ 字节，另有线性辅助数组。m=20 时仅分层数组约 168 MiB，使用前核对内存。

```

1 // c[s] = sum_{t subset s} a[t] * b[s xor t].
2 template <int mod = 998244353>
3 vector<ModInt<mod>> subset_convolution(const vector<ModInt<mod>> &a,
4                                       const vector<ModInt<mod>> &b)
5 {
6     using F = SetConvolution<mod>;
7     using Poly = typename F::Poly;
8     assert(!a.empty() && a.size() == b.size() && a.size() <= INT_MAX);
9     int n = a.size(), m = 0;
10    assert((n & (n - 1)) == 0);
11    while ((1 << m) < n) m++;
12    vector<Poly> f(m + 1, Poly(n)), g(m + 1, Poly(n));
13    vector<int> rank(n);
14    for (int s = 0; s < n; s++)
15    {
16        rank[s] = __builtin_popcount(unsigned(s));
17        f[rank[s]][s] = a[s];
18        g[rank[s]][s] = b[s];
19    }
20    for (int k = 0; k <= m; k++)
21    {
22        F::transform(f[k], '|');
23        F::transform(g[k], '|');
24    }
25    Poly answer(n), h(n);
26    for (int k = 0; k <= m; k++)
27    {

```

```
28     fill(h.begin(), h.end(), 0);
29     for (int i = 0; i <= k; i++)
30         for (int s = 0; s < n; s++) h[s] = h[s] + f[i][s] * g[k - i][s];
31     F::transform(h, '|', true);
32     for (int s = 0; s < n; s++)
33         if (rank[s] == k) answer[s] = h[s];
34 }
35 return answer;
36 }
```

3.3 Chirp Z 变换

3.3.1 Chirp Z: 等比点求值

`chirp_z(f,a,r,m)` 返回 $f(a \cdot r^i)$, $i=0..m-1$; f 为升幂系数数组, a, r 为 $\text{ModInt}\langle \text{mod} \rangle$. $m \geq 0, f.\text{size}() \leq \text{INT_MAX}$; 空 f 表示零多项式, $m=0$ 返回空。约定 $r^0=1$, 因此 $r=0$ 时首项为 $f(a)$ 、其余为 $f(0)$; $a=0$ 或 $r=1$ 也单独处理。普通分支依赖 NTT 素数及原根, 非空长度 n 要求 $n+m-1 \leq \text{NTT}::\text{max_size}$ 。利用三角数指数与循环卷积, 避免平方指数的整数溢出; $O((n+m) \log(n+m))$ 时间、 $O(n+m)$ 空间, 特殊分支 $O(n+m)$ 。默认模板参数 $\text{mod}=998244353$, $\text{primitive}=3$ 。输出顺序允许重复点。

```

1 template <int mod = 998244353, int primitive = 3>
2 vector<ModInt<mod>>
3 chirp_z(const vector<ModInt<mod>> &f, ModInt<mod> a, ModInt<mod> r, int m)
4 {
5     using N = NttConvolution<mod, primitive>;
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     assert(m >= 0 && f.size() <= INT_MAX);
9     if (m == 0 || f.empty()) return Poly(m);
10    if (a.v == 0 || r.v == 0 || r.v == 1)
11    {
12        Poly answer(m, f[0]);
13        Z value = 0;
14        for (int i = int(f.size()) - 1; i >= 0; i--) value = value * a + f[i];
15        answer[0] = value;
16        if (r.v == 1) fill(answer.begin(), answer.end(), value);
17        return answer;
18    }
19    assert(f.size() + m - 1 <= N::max_size);
20    int n = f.size(), size = 1;
21    while (size < n + m - 1) size *= 2;
22    Poly x(size), y(size), inverse(max(n, m));
23    Z power = 1, triangle = 1, ri = r.inv();
24    for (auto &v : inverse)
25    {
26        v = triangle;
27        triangle = triangle * power;
28        power = power * ri;
29    }

```

构造循环卷积并取点值（接上页同一函数）：

```

30     power = 1;
31     for (int j = 0; j < n; j++)
32     {
33         x[n - 1 - j] = f[j] * power * inverse[j];
34         power = power * a;
35     }
36     power = 1;
37     triangle = 1;
38     for (int j = 0; j < n + m - 1; j++)
39     {
40         y[j] = triangle;
41         triangle = triangle * power;
42         power = power * r;
43     }
44     N::ntt(x);
45     N::ntt(y);
46     for (int j = 0; j < size; j++) x[j] = x[j] * y[j];
47     N::ntt(x, true);
48     Poly answer(m);
49     for (int i = 0; i < m; i++) answer[i] = x[n - 1 + i] * inverse[i];
50     return answer;
51 }
```

3.4 多项式牛顿迭代

3.4.1 形式幂级数求逆

`FpsInverse::inverse(a,n)` 返回 a 的逆级数前 n 项，模数固定 998244353。要求 $n>0$ 、 a 非空且 $a[0]\neq 0$ ；不足项视为零，多余项只取需要的前缀。继承 `NttConvolution<998244353>` 的 `Poly`、`Z`、`multiply`，按 Newton 公式 $b \leftarrow b(2-ab)$ 倍增。所有内部卷积结果长度须 $\leq 2^{23}$ ， $n \leq 2^{22}$ 是统一充分界。 $O(n \log n)$ 时间、 $O(n)$ 辅助空间。

```

1 struct FpsInverse : NttConvolution<998244353>
2 {
3     static constexpr int mod = 998244353;
4
5     static Poly inverse(const Poly &a, int n)
6     {
7         assert(n > 0 && !a.empty() && a[0].v);
8         Poly b{a[0].inv()};
9         while ((int)b.size() < n)
10         {
11             int m = min(n, 2 * (int)b.size());
12             Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
13             auto t = multiply(f, b);
14             t.resize(m);
15             for (auto &x : t) x = Z(0) - x;
16             t[0] = t[0] + 2;
17             b = multiply(b, t);
18             b.resize(m);
19         }
20         return b;
21     }
22 };
```

3.5 多项式初等函数

3.5.1 形式幂级数微积分、对数与指数

FpsFunctions 继承 FpsInverse，复用同一 NTT 和求逆。derivative(a) 形式求导；integral(a) 返回常数项为零的积分，要求 $a.size() < 998244353$ 。log(a,n) 要求 $n > 0$ 、a 非空、 $a[0]=1$ ；exp(a,n) 要求 $n > 0$ 、a 为空或 $a[0]=0$ 。返回前 n 项，不足输入项视为零。内部每次卷积结果长度须 $\leq 2^{23}$ ； $n \leq 2^{22}$ 且 $a.size() \leq n$ 是对数/指数的统一充分界。log 会对完整 a 求导，长输入仍须检查容量。微积分 $O(a.size())$ ；在 $a.size() \leq n$ 时 log/exp 为 $O(n \log n)$ 时间和 $O(n)$ 辅助空间。

```

1 struct FpsFunctions : FpsInverse
2 {
3     static Poly derivative(const Poly &a)
4     {
5         Poly b;
6         for (int i = 1; i < (int)a.size(); i++) b.push_back(a[i] * i);
7         return b;
8     }
9
10    static Poly integral(const Poly &a)
11    {
12        Poly b(a.size() + 1);
13        vector<Z> inv(a.size() + 1);
14        if (!a.empty()) inv[1] = 1;
15        for (int i = 2; i <= (int)a.size(); i++)
16            inv[i] = Z(mod - mod / i) * inv[mod % i];
17        for (int i = 0; i < (int)a.size(); i++) b[i + 1] = a[i] * inv[i + 1];
18        return b;
19    }
20
21    static Poly log(const Poly &a, int n)
22    {
23        assert(n > 0 && !a.empty() && a[0].v == 1);
24        auto b = multiply(derivative(a), inverse(a, n));
25        b.resize(n - 1);
26        return integral(b);
27    }

```

```
29 static Poly exp(const Poly &a, int n)
30 {
31     assert(n > 0 && (a.empty() || a[0].v == 0));
32     Poly b{1};
33     while ((int)b.size() < n)
34     {
35         int m = min(n, 2 * (int)b.size());
36         auto t = log(b, m);
37         for (int i = 0; i < m; i++) t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
38         t[0] = t[0] + 1;
39         b = multiply(b, t);
40         b.resize(m);
41     }
42     return b;
43 }
44 };
```

3.6 常系数齐次线性递推

3.6.1 特征多项式求线性递推第 k 项

`recurrence_nth(init,c,n,p)` 使用 0-based 下标, n 为 unsigned long long。init 与 c 长度均为阶数 k ; $a[i] = \sum c[j-1]a[i-j]$, $2 \leq p \leq \text{INT_MAX}$, 数值均在 $[0,p)$, 本函数允许合数模数。空系数约定全零序列。计算 x^n 对特征多项式取余再与初值点乘, $O(k^2 \log(n+1))$ 时间、 $O(k)$ 空间; 不依赖 BM 或其他模板。较大阶数可使用本库 `BostanMori::recurrence`, 须满足其 NTT 素数模数与容量条件; 本函数允许合数模数。

```

1 // init has exactly k entries; c[j-1] multiplies a[n-j]. Modulus need not be prime.
2 int recurrence_nth(const vector<int> &init,
3                   const vector<int> &c,
4                   unsigned long long n,
5                   int p = 998244353)
6 {
7     assert(p >= 2 && init.size() == c.size());
8     int k = c.size();
9     if (!k) return 0;
10    for (int x : init) assert(0 <= x && x < p);
11    for (int x : c) assert(0 <= x && x < p);
12    if (n < (unsigned)k) return init[n];
13    auto multiply = [&](const vector<int> &a, const vector<int> &b)
14    {
15        vector<int> t(2 * k - 1);
16        for (int i = 0; i < k; i++)
17        {
18            for (int j = 0; j < k; j++) t[i + j] = (t[i + j] + 1LL * a[i] * b[j]) % p;
19        }
20        for (int i = 2 * k - 2; i >= k; i--)
21        {
22            for (int j = 1; j <= k; j++)
23                t[i - j] = (t[i - j] + 1LL * t[i] * c[j - 1]) % p;
24        }
25        t.resize(k);
26        return t;
27    };
28    vector<int> answer(k), x(k);
29    answer[0] = 1;
30    if (k == 1)
31        x[0] = c[0];
32    else
33        x[1] = 1;
34    while (n)
35    {
36        if (n & 1) answer = multiply(answer, x);
37        x = multiply(x, x);
38        n >>= 1;
39    }
40    long long value = 0;
41    for (int i = 0; i < k; i++) value = (value + 1LL * answer[i] * init[i]) % p;
42    return value;
43 }

```

3.6.2 快速递推与有理生成函数单项

BostanMori<mod=998244353,primitive=3>, 依赖参数化 NTT; 素数模数与原根条件同 NttConvolution。Poly 为升幂系数 $\text{vector}<Z>$ 。coefficient(p,q,n) 返回 $[x^n]p(x)/q(x)$, q 非空且 $q[0] \neq 0$, p 可空或次数不小于 q; 令 $d=\max(p.size(),q.size())$, 要求 $2d-1 \leq \text{NTT}::\text{max_size}$ 。recurrence(init,c,n) 返回 $a[n]$, 从下标 k 起 $a[i]=c[0]a[i-1]+\dots+c[k-1]a[i-k]$, init、c 长度均为 k; $k=0$ 约定全零。非初值查询要求 $2k+1 \leq \text{NTT}::\text{max_size}$ 。n 支持完整 unsigned long long。每轮两次卷积后抽取奇偶项, $O(d \log d \log(n+1))$ 时间、 $O(d)$ 空间; 递推接口 $d=k+1$ 。

```

1 template <int mod = 998244353, int primitive = 3> struct BostanMori
2 {
3     using N = NttConvolution<mod, primitive>;
4     using Z = typename N::Z;
5     using Poly = typename N::Poly;
6     using U = unsigned long long;
7
8     // Coefficient of  $x^n$  in  $p(x) / q(x)$ , with  $q[0] \neq 0$ .
9     static Z coefficient(Poly p, Poly q, U n)
10    {
11        assert(!q.empty() && q[0].v != 0);
12        assert(max(p.size(), q.size()) <= (N::max_size + 1ULL) / 2);
13        while (n && !p.empty())
14        {
15            Poly r = q;
16            for (int i = 1; i < int(r.size()); i += 2) r[i] = Z(0) - r[i];
17            Poly a = N::multiply(p, r);
18            Poly b = N::multiply(q, r);
19            p.clear();
20            q.clear();
21            for (int i = n & 1; i < int(a.size()); i += 2) p.push_back(a[i]);
22            for (int i = 0; i < int(b.size()); i += 2) q.push_back(b[i]);
23            n >>= 1;
24        }
25        return p.empty() ? Z(0) : p[0] / q[0];
26    }

```

从初值构造递推生成函数（接上页同一结构体）：

```

28 // a[i] = c[0]*a[i-1] + ... + c[k-1]*a[i-k], for i >= k.
29 static Z recurrence(const Poly &init, const Poly &c, U n)
30 {
31     assert(init.size() == c.size());
32     int k = c.size();
33     if (k == 0) return 0;
34     if (n < init.size()) return init[n];
35     assert(k + 1ULL <= (N::max_size + 1ULL) / 2);
36     Poly q(k + 1);
37     q[0] = 1;
38     for (int i = 0; i < k; i++) q[i + 1] = Z(0) - c[i];
39     Poly p = N::multiply(init, q);
40     p.resize(k);
41     return coefficient(p, q, n);
42 }
43 };

```

3.7 多项式平移 | 连续点值平移

3.7.1 多项式系数平移

`polynomial_shift<mod=998244353,primitive=3>(f,c)` 返回 $f(x+c)$ 的升幂系数，结果长度始终与 f 相同，保留末尾零；空输入返回空。参数 c 使用 `ModInt<mod>`。依赖参数化 NTT， mod 为素数且 primitive 为原根；非空长度 n 要求 $2n-1 \leq \text{NTT::max_size}$ ，因此阶乘均可逆。逆序的 $f[i]i!$ 与 $c^i/i!$ 卷积后提取对应项， $O(n \log n)$ 时间、 $O(n)$ 空间。 $c=0$ 合法。

```

1 template <int mod = 998244353, int primitive = 3>
2 vector<ModInt<mod>> polynomial_shift(const vector<ModInt<mod>> &f, ModInt<mod> c)
3 {
4     using N = NttConvolution<mod, primitive>;
5     using Z = typename N::Z;
6     using Poly = typename N::Poly;
7     if (f.empty()) return {};
8     assert(f.size() <= (N::max_size + 1ULL) / 2);
9     int n = f.size();
10    Poly fact(n), inv(n), a(n), b(n);
11    fact[0] = 1;
12    for (int i = 1; i < n; i++) fact[i] = fact[i - 1] * i;
13    inv[n - 1] = fact[n - 1].inv();
14    for (int i = n - 1; i > 0; i--) inv[i - 1] = inv[i] * i;
15    Z power = 1;
16    for (int i = 0; i < n; i++)
17    {
18        a[n - 1 - i] = f[i] * fact[i];
19        b[i] = power * inv[i];
20        power = power * c;
21    }
22    auto product = N::multiply(a, b);
23    for (int i = 0; i < n; i++) a[i] = product[n - 1 - i] * inv[i];
24    return a;
25 }

```

第四章 组合数学

4.1 排列组合

4.1.1 阶乘组合数

`Binomial<prime>(n=0)` 预处理 $0..n$, 要求 `prime` 为素数且 $0 \leq n < \text{prime}$ 。`init(n)` 只扩大当前上限, 较小或相同的 n 不改变表; `choose(n,k)` 返回 $C(n,k)$, `permute(n,k)` 返回 $n!/(n-k)!$ 。查询前显式预处理到 n , 查询本身不扩表。 $n < 0$, $k < 0$ 或 $k > n$ 返回 0, $C(0,0)=P(0,0)=1$ 。`fac`、`ifac` 为可直接读取的阶乘与逆阶乘表, 增长可能使旧引用失效。单次查询 $O(1)$; 从 `old` 扩到 n 新增 $O(n-\text{old}+\log \text{prime})$ 次域运算, 重分配还可能复制旧表。首次预处理 $O(n+\log \text{prime})$, 空间 $O(n)$ 。超过模数不能使用阶乘除法: 小素数用 `Lucas`, 小合数用 `exLucas`。

```
1 template <int mod> struct Binomial
2 {
3     using Z = ModInt<mod>;
4     vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6     Binomial(int n = 0) { init(n); }
7
8     void init(int n)
9     {
10         assert(0 <= n && n < mod);
11         int old = (int)fac.size() - 1;
12         if (n <= old) return;
13         fac.resize(n + 1);
14         ifac.resize(n + 1);
15         for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
16         ifac[n] = fac[n].inv();
17         for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
18     }
19 }
```

```
20 Z choose(int n, int k) const
21 {
22     if (k < 0 || k > n) return 0;
23     assert(n < (int)fac.size());
24     return fac[n] * ifac[k] * ifac[n - k];
25 }
26
27 Z permute(int n, int k) const
28 {
29     if (k < 0 || k > n) return 0;
30     assert(n < (int)fac.size());
31     return fac[n] * ifac[n - k];
32 }
33 };
```

4.2 斯特林数

4.2.1 第二类斯特林数：同一行前缀

`stirling_second_row<mod,primitive>(n,m)` 返回 $S(n,0..m)$ ，默认 `mod=998244353`、`primitive=3`。模数必须是素数，`primitive` 是原根。`n` 支持完整 `unsigned long long`；`2m+1` 不能超过 NTT 的 `max_size`。 $S(n,k)$ 为无标号非空集合划分数； $n=0$ 时仅第零项为 1， $k>n$ 为零。一次卷积配合逆阶乘， $O(m \log m + m \log \text{mod})$ 时间、 $O(m)$ 空间。依赖 `NttConvolution`；模 167772161 可直接使用模板参数 `<167772161>`。

```

1 template <int mod = 998244353, int primitive = 3>
2 inline vector<ModInt<mod>> stirling_second_row(unsigned long long n, int m)
3 {
4     using N = NttConvolution<mod, primitive>;
5     assert(m >= 0 && m <= (N::max_size - 1) / 2);
6     using Z = typename N::Z;
7     typename N::Poly a(m + 1), b(m + 1);
8     b[0] = 1;
9     for (int i = 1; i <= m; i++) b[i] = b[i - 1] * i;
10    b[m] = b[m].inv();
11    for (int i = m; i > 0; i--) b[i - 1] = b[i] * i;
12    for (int i = 0; i <= m; i++)
13    {
14        Z value = i ? Z(i).pow(n % (mod - 1)) : Z(n == 0);
15        a[i] = value * b[i];
16        if (i % 2) b[i] = Z(0) - b[i];
17    }
18    auto answer = N::multiply(a, b);
19    answer.resize(m + 1);
20    return answer;
21 }
```

4.2.2 无符号第一类斯特林数：同一行

`stirling_first_row<mod,primitive>(n)` 返回无符号 $c(n,0..n)$ ，默认模 998244353、原根 3。 $c(n,k)$ 为 n 元排列恰有 k 个循环的个数。 $n \geq 0$, $n+1 \leq \text{NTT 的 max_size}$; $n=0$ 返回 $\{1\}$ 。倍增配合多项式平移计算 $x(x+1)\cdots(x+n-1)$, $O(n \log n)$ 时间、 $O(n)$ 空间。带符号版本再乘 $(-1)^{(n-k)}$ 。依赖 `NttConvolution`；模 167772161 可用 `<167772161>`。

```

1 template <int mod = 998244353, int primitive = 3>
2 inline vector<ModInt<mod>> stirling_first_row(int n)
3 {
4     using N = NttConvolution<mod, primitive>;
5     assert(n >= 0 && n < N::max_size);
6     using Z = typename N::Z;
7     using Poly = typename N::Poly;
8     Poly fact(n + 1), inv(n + 1);
9     fact[0] = 1;
10    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i;
11    inv[n] = fact[n].inv();
12    for (int i = n; i > 0; i--) inv[i - 1] = inv[i] * i;
13    function<Poly(int)> solve = [&](int len) -> Poly
14    {
15        if (len == 0) return {1};
16        int m = len / 2;
17        Poly f = solve(m);
18        Poly a(m + 1), b(m + 1);
19        Z power = 1;
20        for (int i = 0; i <= m; i++)
21        {
22            a[m - i] = f[i] * fact[i];
23            b[i] = power * inv[i];
24            power = power * m;
25        }
26        auto c = N::multiply(a, b);
27        for (int i = 0; i <= m; i++) a[i] = c[m - i] * inv[i];
28        f = N::multiply(f, a);
29        if (len % 2)
30        {
31            f.push_back(0);
32            for (int i = len; i > 0; i--) f[i] = f[i - 1] + f[i] * (len - 1);
33            f[0] = f[0] * (len - 1);
34        }
35        return f;
36    };
37    return solve(n);
38 }

```

4.3 分拆数

4.3.1 整数分拆与重数上限

$\text{Partitions}(N, \text{mod})$ 求无序正整数分拆, $0 \leq N < \text{INT_MAX}$, $1 \leq \text{mod} \leq \text{INT_MAX}$, $p[n]$ 为不限制重数的分拆数; $p[0] = 1 \% \text{mod}$ 。预处理 $O(N\sqrt{N})$ 时间、 $O(N)$ 空间; 模数不要求素数。 $\text{limited}(n, k)$ 限制每个正整数最多出现 k 次, $0 \leq n \leq N$, $0 \leq k \leq \text{INT_MAX}$; $k=0$ 时仅空分拆存在, $k=1$ 为互异分拆。单次 $O(1 + \sqrt{(n/(k+1))})$, 额外空间 $O(1)$ 。 p/mod 只读; 中间五边形数与累加采用 long long 。

```

1 struct Partitions
2 {
3     int mod;
4     vector<int> p;
5
6     Partitions(int n, int mod) : mod(mod), p(n + 1)
7     {
8         assert(n >= 0 && mod >= 1);
9         p[0] = 1 % mod;
10        for (int i = 1; i <= n; i++)
11        {
12            long long value = 0;
13            for (long long j = 1; j * (3 * j - 1) / 2 <= i; j++)
14            {
15                long long a = j * (3 * j - 1) / 2;
16                long long b = j * (3 * j + 1) / 2;
17                int sign = j % 2 ? 1 : -1;
18                value += 1LL * sign * p[i - a];
19                if (b <= i) value += 1LL * sign * p[i - b];
20            }
21            p[i] = (value % mod + mod) % mod;
22        }
23    }
24
25    int limited(int n, int k) const
26    {
27        assert(0 <= n && n < (int)p.size() && k >= 0);
28        long long step = 1LL + k;
29        long long value = p[n];
30        for (long long j = 1; j * (3 * j - 1) / 2 <= n / step; j++)
31        {
32            long long a = step * (j * (3 * j - 1) / 2);
33            long long b = step * (j * (3 * j + 1) / 2);
34            int sign = j % 2 ? -1 : 1;
35            value += 1LL * sign * p[n - a];
36            if (b <= n) value += 1LL * sign * p[n - b];
37        }
38        return (value % mod + mod) % mod;
39    }
40 };

```

第五章 线性代数

5.1 矩阵

5.1.1 模矩阵乘法与幂

`ModMatrix<p>::multiply(a,b)` 计算 $n \times k$ 与 $k \times m$ 的矩阵积，要求非空且每行尺寸一致。`power(a,e)` 接收非空 $n \times n$ 方阵和 `unsigned long long` 指数；`e=0` 返回单位阵。只使用加乘，`p` 为 ≥ 2 的 `int` 模数，允许合数。矩阵乘法 $O(nkm)$ ，矩阵幂 $O(n^3 \log(e+1))$ ，额外空间 $O(n^2)$ ；复用 `ModInt`。

```
1 template <int mod> struct ModMatrix
2 {
3     using Z = ModInt<mod>;
4     using Matrix = vector<vector<Z>>>;
5
6     static Matrix multiply(const Matrix &a, const Matrix &b)
7     {
8         assert(!a.empty() && !b.empty());
9         int n = a.size(), m = b[0].size(), k = b.size();
10        assert((int)a[0].size() == k);
11        Matrix c(n, vector<Z>(m));
12        for (int i = 0; i < n; i++)
13            for (int t = 0; t < k; t++)
14                for (int j = 0; j < m; j++) c[i][j] = c[i][j] + a[i][t] * b[t][j];
15        return c;
16    }
17
18    static Matrix power(Matrix a, unsigned long long e)
19    {
20        int n = a.size();
21        assert(n > 0 && (int)a[0].size() == n);
22        Matrix r(n, vector<Z>(n));
23        for (int i = 0; i < n; i++) r[i][i] = 1;
24        for (; e; e >>= 1, a = multiply(a, a))
25            if (e & 1) r = multiply(r, a);
26        return r;
27    }
28 };
```

5.1.2 Max-plus 矩阵幂

$a[i][j]$ 为转移收益，不存在的边为 neg ； $power(k)$ 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^{119} 。 $O(n^3 \log k)$ ；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```

1 struct MaxPlusMatrix
2 {
3     using I = __int128_t;
4     static constexpr I neg = -(I(1) << 120);
5     int n;
6     vector<vector<I>> a;
7
8     MaxPlusMatrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
9     {
10         if (identity)
11             for (int i = 0; i < n; i++) a[i][i] = 0;
12     }
13
14     MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
15     {
16         assert(n == b.n);
17         MaxPlusMatrix c(n);
18         for (int i = 0; i < n; i++)
19             for (int k = 0; k < n; k++)
20                 if (a[i][k] != neg)
21                     for (int j = 0; j < n; j++)
22                         if (b.a[k][j] != neg)
23                             c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
24         return c;
25     }
26
27     MaxPlusMatrix power(unsigned long long e) const
28     {
29         MaxPlusMatrix b = *this, r(n, true);
30         for (; e >>= 1, b = b * b)
31             if (e & 1) r = r * b;
32         return r;
33     }
34 };

```

5.2 行列式

5.2.1 素数模行列式

`det_prime<p>(a)` 输入 $n \times n$ 方阵, 返回行列式模 p ; p 必须是素数。允许奇异矩阵和零维矩阵, `det(empty)=1`。按行交换改变符号, 主元求逆消元。 $O(n^3)$ 时间、传值副本 $O(n^2)$ 空间, 可传 `move(a)`。合数模数改用 `determinant_mod`。

```

1 template <int mod> ModInt<mod> det_prime(vector<vector<ModInt<mod>>> a)
2 {
3     using Z = ModInt<mod>;
4     int n = (int)a.size();
5     Z ans = 1;
6     for (int i = 0; i < n; i++)
7     {
8         assert((int)a[i].size() == n);
9         int p = i;
10        while (p < n && !a[p][i].v) ++p;
11        if (p == n) return 0;
12        if (p != i)
13        {
14            swap(a[p], a[i]);
15            ans = Z(0) - ans;
16        }
17        ans = ans * a[i][i];
18        Z inv = a[i][i].inv();
19        for (int j = i + 1; j < n; j++)
20        {
21            Z f = a[j][i] * inv;
22            for (int k = i; k < n; k++) a[j][k] = a[j][k] - f * a[i][k];
23        }
24    }
25    return ans;
26 }
```

5.2.2 任意正模数行列式：欧几里得消元

`determinant_mod(a,mod)` 返回 $\det(a)$ 的最小非负余数。 a 为 `long long` 方阵，元素支持完整 signed 64 位； $1 \leq \text{mod} \leq \text{LLONG_MAX}$ ，可以是合数。不使用模逆元，允许不可逆主元。矩阵按值传入，调用后原矩阵不变；空矩阵返回 $1 \% \text{mod}$ ， $\text{mod}=1$ 时返回 0。换行改变符号，最终乘上三角矩阵的对角元素。所有乘法先转 `int128`。 $O(n^3 \log \text{mod})$ 时间、 $O(n^2)$ 空间；素数模通常优先用已有高斯行列式。

```

1 inline long long determinant_mod(vector<vector<long long>> a, long long mod)
2 {
3     assert(mod >= 1);
4     int n = a.size();
5     for (auto &row : a)
6     {
7         assert((int)row.size() == n);
8         for (auto &x : row)
9         {
10             x %= mod;
11             if (x < 0) x += mod;
12         }
13     }
14     long long answer = 1 % mod;
15     for (int i = 0; i < n; i++)
16     {
17         for (int j = i + 1; j < n; j++)
18             while (a[j][i])
19             {
20                 long long quotient = a[i][i] / a[j][i];
21                 for (int k = i; k < n; k++)
22                 {
23                     __int128 value = (__int128)a[i][k] - (__int128)quotient * a[j][k];
24                     value %= mod;
25                     if (value < 0) value += mod;
26                     a[i][k] = (long long)value;
27                 }
28                 swap(a[i], a[j]);
29                 answer = answer ? mod - answer : 0;
30             }
31         if (a[i][i] == 0) return 0;
32         answer = (__int128)answer * a[i][i] % mod;
33     }
34     return answer;
35 }

```

5.3 线性基

5.3.1 64 位异或线性基

值类型 `U` 为 `unsigned long long`，完整支持 64 位。默认空基；`init/Init` 清空。`insert/Insert` 返回是否增秩；`contains/Contains` 判断可表示性（允许空子集，所以 0 总可表示）。`query/Query(x=0)` 返回 `x` 与所张成空间异或后的最大无符号值。`kth/Kth(k,nonempty=true)` 返回第 `k` 小的不同异或值，1-based，参数 `k` 为 `__uint128_t`，非法序号返回 `nullopt`。默认只允许非空输入子集，0 仅在 `dependent` 为真时出现；传 `false` 则包括空子集所得的 0。`minimum/Minimum` 同此约定，空输入的非空最小值不存在。`merge/Merge` 将另一基所代表的输入多重集并入；重复元素和自合并可新增异或为 0 的非空方案。`rank` 是维数，`dependent` 表示存在非空零异或子集。插入、成员判断、最大值为 $O(64)$ ；合并 $O(64^2)$ 。`kth` 会在需要时 `rebuild/Rebuild` 规范化缓存，首次有效查询 $O(64^2)$ ，后续 $O(64)$ ；仅新增独立向量使缓存失效。所有成员仅供只读，不要直接改数组或标志。

```

1 struct XorBasis
2 {
3     using U = unsigned long long;
4     using Wide = __uint128_t;
5     U a[64]{};
6     vector<U> p;
7     int rank = 0;
8     bool dependent = false, dirty = false;
9
10    void init()
11    {
12        fill(a, a + 64, 0);
13        p.clear();
14        rank = 0;
15        dependent = false;
16        dirty = false;
17    }
18
19    bool insert(U x)
20    {
21        for (int i = 63; i >= 0; i--)
22            if (x >> i & 1)
23            {
24                if (!a[i])
25                {
26                    a[i] = x;
27                    rank++;
28                    dirty = true;
29                    return true;
30                }
31                x ^= a[i];
32            }
33        dependent = true;
34        return false;
35    }

```

```
37     bool contains(U x) const
38     {
39         for (int i = 63; i >= 0; i--)
40             if (x >> i & 1) x ^= a[i];
41         return x == 0;
42     }
43
44     U query(U x = 0) const
45     {
46         for (int i = 63; i >= 0; i--) x = max(x, x ^ a[i]);
47         return x;
48     }
49
50     void rebuild()
51     {
52         vector<U> b(a, a + 64);
53         for (int i = 0; i < 64; i++)
54             if (b[i])
55                 for (int j = i + 1; j < 64; j++)
56                     if (b[j] >> i & 1) b[j] ^= b[i];
57         p.clear();
58         for (U x : b)
59             if (x) p.push_back(x);
60         dirty = false;
61     }
```

```

63 // k is 1-based; nonempty excludes the empty input subset, not value zero.
64 optional<U> kth(Wide k, bool nonempty = true)
65 {
66     if (!k) return nullopt;
67     Wide index = k;
68     if (!nonempty || dependent) index--;
69     if (index >= (Wide(1) << rank)) return nullopt;
70     if (dirty) rebuild();
71     U answer = 0;
72     for (int i = 0; i < rank; i++)
73         if (index >> i & 1) answer ^= p[i];
74     return answer;
75 }
76
77 optional<U> minimum(bool nonempty = true) { return kth(1, nonempty); }
78
79 void merge(const XorBasis &other)
80 {
81     dependent |= other.dependent;
82     for (U x : other.a)
83         if (x) insert(x);
84 }
85 };

```

5.3.2 带位置限制的异或线性基

默认构造为空，init/Init 可清空。insert/Insert(x,at) 插入 unsigned long long 值及非负 int 标签，支持完整 64 位，标签可重复、乱序。contains/Contains(x,lower=0) 判断 x 能否由标签至少为 lower 的输入异或得到；query/Query(lower=0,x=0) 返回 x 与这些输入子集异或后的最大值；rank/Rank(lower=0) 返回对应空间维数。lower 为非负 long long，可大于所有 int 标签，空集仍可产生 0。全部操作 $O(64)$ ，空间 $O(64)$ 。消元时同一主元优先保留标签较大的向量，较旧向量继续消元；按阈值筛选后仍是正确线性基。标签是组合可用范围的界，不是选点方案；数组仅供只读。区间 [l,r] 应按右端点排序询问，扫描并仅插入到 r，再查询 lower=l；接口本身没有上界，插入超过 r 后不能只用 lower 恢复旧区间。若需保留历史，可复制每个前缀的对象，但空间为 $O(64n)$ 。不提供非空零异或判定或第 k 小；普通线性基的 dependent 不能直接移植到所有阈值。

```

1 struct PositionBasis
2 {
3     using U = unsigned long long;
4     U a[64];
5     int pos[64];
6
7     PositionBasis() { init(); }
8
9     void init()
10    {
11        fill(a, a + 64, 0);
12        fill(pos, pos + 64, -1);
13    }

```

```

15 void insert(U x, int at)
16 {
17     assert(at >= 0);
18     for (int i = 63; i >= 0; i--)
19         if (x >> i & 1)
20             {
21                 if (!a[i])
22                 {
23                     a[i] = x;
24                     pos[i] = at;
25                     return;
26                 }
27                 if (at > pos[i])
28                 {
29                     swap(a[i], x);
30                     swap(pos[i], at);
31                 }
32                 x ^= a[i];
33             }
34 }
35
36 bool contains(U x, long long lower = 0) const
37 {
38     assert(lower >= 0);
39     for (int i = 63; i >= 0; i--)
40         if ((x >> i & 1) && pos[i] >= lower) x ^= a[i];
41     return x == 0;
42 }
43
44 U query(long long lower = 0, U x = 0) const
45 {
46     assert(lower >= 0);
47     for (int i = 63; i >= 0; i--)
48         if (pos[i] >= lower) x = max(x, x ^ a[i]);
49     return x;
50 }
51
52 int rank(long long lower = 0) const
53 {
54     assert(lower >= 0);
55     int answer = 0;
56     for (int i = 0; i < 64; i++) answer += pos[i] >= lower;
57     return answer;
58 }
59 };

```

5.3.3 异或线性空间求交

`basis_intersection(a,b)` 返回 `span(a)` 与 `span(b)` 的交空间的一组独立生成元，结果类型 `XorBasis`。支持完整 64 位无符号值，不修改输入，也不要求先 `rebuild`。使用第一个空间的主元，消去第二个空间的向量，同时记录来自第二个空间的部分；消元为零时，该部分属于交空间。时间 $O(64^2)$ ，额外空间 $O(64)$ 。输出按独立生成元解释：`dependent` 为 `false`，输入的重复次数及非空零异或关系不传播；表示的是子空间，不是两组原始输入元素的集合交。枚举整个交空间应调用 `result.kth(k,false)`，包含零；零空间返回空基，`rank=0`、`contains(0)=true`。

```

1 inline XorBasis basis_intersection(const XorBasis &a, const XorBasis &b)
2 {
3     using U = unsigned long long;
4     U pivot[64], from_b[64]{};
5     copy(a.a, a.a + 64, pivot);
6     XorBasis result;
7     for (U value : b.a)
8     {
9         U x = value, part = value;
10        for (int i = 63; i >= 0 && x; i--)
11            if (x >> i & 1)
12            {
13                if (!pivot[i])
14                {
15                    pivot[i] = x;
16                    from_b[i] = part;
17                    break;
18                }
19                x ^= pivot[i];
20                part ^= from_b[i];
21            }
22        if (!x && part) result.insert(part);
23    }
24    return result;
25 }
```

5.3.4 Zassenhaus：同时求和空间与交空间

`basis_sum_intersection(a,b)` 返回和空间基、交空间基这一对结果。和空间由两组基共同张成，一般不是两个空间的集合并；两结果都是独立生成元，其 `dependent` 均为 `false`。将 `a` 的向量 `x` 编成 (x,x) ，`b` 的向量 `y` 编成 $(y,0)$ ，用 `__uint128_t` 储存高低各 64 位并按高位到低位消元。高半部非零的行投影构成和空间的基，高半部为零的行取低半部构成交空间的基。必须先提升到 128 位再左移 64；不能用 64 位变量移位 64。时间 $O(128^2)$ ，额外空间 $O(128)$ 。结果满足 $\dim(A+B)+\dim(A \cap B)=\dim(A)+\dim(B)$ ，但维数恒等式只是检查条件，不能单独证明返回向量正确。包含空空间、相同空间和输入别名。

```

1 // Return independent generators of (sum space, intersection space).
2 inline pair<XorBasis, XorBasis> basis_sum_intersection(const XorBasis &a,
3                                                         const XorBasis &b)
4 {
5     using W = __uint128_t;
6     W pivot[128]{};
7     auto insert = [&](W x)
8     {
9         for (int i = 127; i >= 0 && x; i--)
10             if (x >> i & 1)
11             {
12                 if (!pivot[i])
13                 {
14                     pivot[i] = x;
15                     return;
16                 }
17                 x ^= pivot[i];
18             }
19     };
20     for (auto x : a.a) insert((W(x) << 64) | x);
21     for (auto x : b.a) insert(W(x) << 64);
22     XorBasis sum, intersection;
23     for (int i = 0; i < 128; i++)
24         if (pivot[i])
25         {
26             if (i < 64)
27                 intersection.insert((unsigned long long)pivot[i]);
28             else
29                 sum.insert((unsigned long long)(pivot[i] >> 64));
30         }
31     return {sum, intersection};
32 }

```

5.3.5 无向图最大异或行走

`XorWalk(n)` 建图，点号 $1..n$ ，边权为 `unsigned long long`，支持完整 64 位。`add(u,v,w)` 加无向边，允许重边、自环和零权；`init(n)` 清空图。`build(root=1)` 用递归 DFS 预处理 `root` 的连通块，`query(u,v)` 返回该块内 `u` 到 `v` 的最大边权异或和；任一点未被本次 `build` 覆盖则返回 `nullopt`，即使这两点在另一连通块相连。加入新边后必须重新 `build`；可更换根重复构建。空图可初始化，但不能 `build`。这里的行走允许重复点、重复边；不是最大异或简单路径。`u=v` 也可能通过绕环得到正值。DFS 距离 `dist` 给出一条树路径的异或值，每条已访问边形成 `dist[u] xor dist[v] xor w` 的闭合行走值，加入线性基；两点答案为 `cycles.query(dist[u] xor dist[v])`。`build` 后 `vis`、`dist` 可只读；`cycles` 表示环异或空间，不保留具体环或行走方案，不能将其非空子集标志解释为行走非空性。建图 $O(n+m)$ ，`build` 为 $O(n+64m)$ ，每次 `query` 为 $O(64)$ ，动态空间 $O(n+m+64)$ 。

```

1 struct XorWalk

```

```
2 {
3     using U = unsigned long long;
4     int n;
5     bool built = false;
6     vector<vector<pair<int, U>>> g;
7     vector<int> vis;
8     vector<U> dist;
9     XorBasis cycles;
10
11     XorWalk(int n) { init(n); }
12
13     void init(int size)
14     {
15         assert(size >= 0);
16         n = size;
17         g.clear();
18         g.resize(size_t(n) + 1);
19         vis.assign(size_t(n) + 1, 0);
20         dist.assign(size_t(n) + 1, 0);
21         cycles.init();
22         built = false;
23     }
```

```

25 void add(int u, int v, U w)
26 {
27     assert(1 <= u && u <= n && 1 <= v && v <= n);
28     g[u].push_back({v, w});
29     g[v].push_back({u, w});
30     built = false;
31 }
32
33 void dfs(int u)
34 {
35     vis[u] = 1;
36     for (auto [v, w] : g[u])
37     {
38         if (!vis[v])
39         {
40             dist[v] = dist[u] ^ w;
41             dfs(v);
42         }
43         else
44             cycles.insert(dist[u] ^ dist[v] ^ w);
45     }
46 }
47
48 void build(int root = 1)
49 {
50     assert(1 <= root && root <= n);
51     fill(vis.begin(), vis.end(), 0);
52     fill(dist.begin(), dist.end(), 0);
53     cycles.init();
54     dfs(root);
55     built = true;
56 }
57
58 optional<U> query(int u, int v) const
59 {
60     assert(built && 1 <= u && u <= n && 1 <= v && v <= n);
61     if (!vis[u] || !vis[v]) return nullopt;
62     return cycles.query(dist[u] ^ dist[v]);
63 }
64 };

```

第六章 数值算法

6.1 插值

6.1.1 拉格朗日插值与连续点优化

先声明 Lagrange. init(xs,ys,p) 使用任意横坐标, xs、ys 为等长 0-based vector<long long>; consecutive(ys,p) 专用于横坐标 0..n-1。两者均覆盖旧数据。要求 p 为素数, $2 \leq p \leq \text{INT_MAX}$, 点数 $n \leq p$, 任意横坐标取模后两两不同; 不是仅要求原整数不同。query(k) 返回次数小于 n 的插值多项式在 k 处的值, 允许 k、xs、ys 为完整 long long 范围, 内部先归一化; 空数据约定零多项式。通用预处理 $O(n^2 + n \log p)$, 连续点预处理 $O(n + \log p)$, 每次查询 $O(n)$, 空间 $O(n)$ 。查询利用前缀积与滚动后缀积, 无需求逆, 因此 k 与样本点模 p 相等时也正确。若求 $1^d + \dots + k^d$, 要求 $0 \leq d \leq p-2$, 先计算 0..d+1 的前缀和作为 ys, 再用 consecutive; 用快速幂生成这些样本另花 $O(d \log(d+1))$ 。仅处理素数域中的精确插值, 不适用于浮点曲线拟合或重复节点的导数约束。

```
1 struct Lagrange
2 {
3     using ll = long long;
4     int n, mod;
5     vector<ll> x, w;
6
7     ll norm(ll a) const
8     {
9         a %= mod;
10        if (a < 0) a += mod;
11        return a;
12    }
13
14    ll power(ll a, int b) const
15    {
16        ll result = 1;
17        while (b)
18        {
19            if (b & 1) result = result * a % mod;
20            a = a * a % mod;
21            b >>= 1;
22        }
23        return result;
24    }
25
26    void init(const vector<ll> &xs, const vector<ll> &ys, int p)
27    {
28        n = ys.size();
29        mod = p;
30        assert(p >= 2 && n <= p && xs.size() == ys.size());
31        x.resize(n);
32        w.resize(n);
33        for (int i = 0; i < n; i++) x[i] = norm(xs[i]);
34        for (int i = 0; i < n; i++)
35        {
36            ll denominator = 1;
37            for (int j = 0; j < n; j++)
38                if (i != j) denominator = denominator * norm(x[i] - x[j]) % mod;
```

```

39         assert(denominator != 0);
40         w[i] = norm(ys[i]) * power(denominator, mod - 2) % mod;
41     }
42 }
43
44 void consecutive(const vector<ll> &ys, int p)
45 {
46     n = ys.size();
47     mod = p;
48     assert(p >= 2 && n <= p);
49     x.resize(n);
50     w.resize(n);
51     if (!n) return;
52     vector<ll> invfac(n, 1);
53     ll factorial = 1;
54     for (int i = 1; i < n; i++) factorial = factorial * i % mod;
55     invfac[n - 1] = power(factorial, mod - 2);
56     for (int i = n - 1; i > 0; i--) invfac[i - 1] = invfac[i] * i % mod;
57     for (int i = 0; i < n; i++)
58     {
59         x[i] = i;
60         w[i] = norm(ys[i]) * invfac[i] % mod * invfac[n - 1 - i] % mod;
61         if ((n - 1 - i) & 1) w[i] = norm(-w[i]);
62     }
63 }
64
65 ll query(ll k) const
66 {
67     k = norm(k);
68     vector<ll> pre(n + 1, 1);
69     for (int i = 0; i < n; i++) pre[i + 1] = pre[i] * norm(k - x[i]) % mod;
70     ll answer = 0, suffix = 1;
71     for (int i = n - 1; i >= 0; i--)
72     {
73         answer = (answer + w[i] * pre[i] % mod * suffix) % mod;
74         suffix = suffix * norm(k - x[i]) % mod;
75     }
76     return answer;
77 }
78 };

```

6.2 高斯消元

6.2.1 素数模高斯消元与全部解

`GaussMod<p>::solve(a,n)` 接收 m 行 $n+1$ 列增广矩阵, p 必须是素数, $n \geq 0$ 。返回 consistent、系数矩阵秩 rank、特解 particular 和零空间基 kernel; 解为特解加基向量的任意线性组合。无解时解数组为空; 空方程组需显式给出变量数。 $O(m \cdot n \cdot \min(m,n) + n^2)$ 时间, 矩阵及解空间输出 $O(mn + n^2)$ 空间。只依赖 `ModInt`, 不含行列式或矩阵树。

```

1 template <int mod> struct GaussMod
2 {
3     using Z = ModInt<mod>;
4     using Matrix = vector<vector<Z>>>;
5
6     struct Solution
7     {
8         bool consistent;
9         int rank;
10        vector<Z> particular;
11        Matrix kernel;
12    };
13
14    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
15    static Solution solve(Matrix a, int n)
16    {
17        int m = (int)a.size(), row = 0;
18        vector<int> where(n, -1);
19        for (const auto &v : a) assert((int)v.size() == n + 1);
20        for (int col = 0; col < n && row < m; col++)
21        {
22            int p = row;
23            while (p < m && !a[p][col].v) ++p;
24            if (p == m) continue;
25            swap(a[p], a[row]);
26            Z inv = a[row][col].inv();
27            for (int j = col; j <= n; j++) a[row][j] = a[row][j] * inv;
28            for (int i = 0; i < m; i++)
29                if (i != row && a[i][col].v)
30                {
31                    Z f = a[i][col];
32                    for (int j = col; j <= n; j++) a[i][j] = a[i][j] - f * a[row][j];
33                }
34            where[col] = row++;
35        }
36        for (int i = row; i < m; i++)
37            if (a[i][n].v) return {false, row, {}, {}};

```

```
38     Solution ans{true, row, vector<Z>(n), {}};
39     for (int j = 0; j < n; j++)
40         if (where[j] != -1) ans.particular[j] = a[where[j]][n];
41     for (int j = 0; j < n; j++)
42         if (where[j] == -1)
43         {
44             vector<Z> v(n);
45             v[j] = 1;
46             for (int k = 0; k < n; k++)
47                 if (where[k] != -1) v[k] = Z(0) - a[where[k]][j];
48             ans.kernel.push_back(v);
49         }
50     return ans;
51 }
52 };
```

第七章 Berlekamp–Massey 算法

7.1 Berlekamp–Massey：最短线性递推

`berlekamp_massey(s,p)` 要求 p 为素数, $2 \leq p \leq \text{INT_MAX}$, 输入均在 $[0,p)$; 返回 c , 使 $s[i] = \sum c[j-1]s[i-j]$ ($j=1..k, i \geq k$)。空序列或全零序列返回空系数。有限前缀可能对应多种未来, 不能无条件外推; 若已知原序列满足阶数至多 d 的齐次线性递推, 提供至少 $2d$ 项才能恢复可外推的递推。模运算次数 $O(Nk+N \log p)$, 后项来自逐次求逆; 空间 $O(N)$ 。不适用于合数模数或浮点拟合。

```
1 // Prime p; normalized input. s[i] = sum(c[j-1] * s[i-j]), i >= c.size().
2 vector<int> berlekamp_massey(const vector<int> &s, int p = 998244353)
3 {
4     assert(p >= 2);
5     vector<int> c{1}, b{1};
6     int len = 0, shift = 1, last = 1;
7     for (int i = 0; i < (int)s.size(); i++)
8     {
9         assert(0 <= s[i] && s[i] < p);
10        long long d = s[i];
11        for (int j = 1; j <= len; j++) d = (d + 1LL * c[j] * s[i - j]) % p;
12        if (d == 0)
13        {
14            shift++;
15            continue;
16        }
17        long long inv = 1, x = last;
18        for (int e = p - 2; e; e >>= 1)
19        {
20            if (e & 1) inv = inv * x % p;
21            x = x * x % p;
22        }
23        long long factor = d * inv % p;
24        auto old = c;
25        if (c.size() < b.size() + shift) c.resize(b.size() + shift);
26        for (int j = 0; j < (int)b.size(); j++)
27            c[j + shift] = (c[j + shift] - factor * b[j] % p + p) % p;
28        if (2 * len <= i)
29        {
30            len = i + 1 - len;
31            b = old;
32            last = d;
33            shift = 1;
34        }
35        else
36            shift++;
37    }
38    c.resize(len + 1);
39    c.erase(c.begin());
40    for (int &x : c) x = x == 0 ? 0 : p - x;
41    return c;
42 }
```

第八章 配套知识与验证范围

8.1 整除、卷积与筛法

对正整数定义狄利克雷卷积

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

单位元为 $\varepsilon(n) = [n = 1]$, 常函数 $1(n) = 1$, 恒等函数 $\text{id}(n) = n$. 关键恒等式是 $\mu * 1 = \varepsilon$, $\varphi * 1 = \text{id}$. 若 $F(n) = \sum_{d|n} f(d)$, 则 $f(n) = \sum_{d|n} \mu(d)F(n/d)$.

$$\sum_{i=1}^a \sum_{j=1}^b [\gcd(i, j) = 1] = \sum_{d=1}^{\min(a, b)} \mu(d) \lfloor a/d \rfloor \lfloor b/d \rfloor.$$

整除分块: 当左端点为 l 、商为 $q = \lfloor n/l \rfloor$, 右端点为 $r = \lfloor n/q \rfloor$. 同时包含多个商时, 取各自右端点的最小值。杜教筛由卷积恒等式获得

$$M(n) = 1 - \sum_{l=2}^n M(\lfloor n/l \rfloor), \quad \Phi(n) = \frac{n(n+1)}{2} - \sum_{l=2}^n \Phi(\lfloor n/l \rfloor),$$

求和按等商区间合并; 第二式求和中没有额外乘以 l 。

8.2 杜教筛: 预筛与缓存都不可省略

前节的两条卷积恒等式给出 DuJiao 的递推, 见代码中的 `mertens` 与 `totient_sum`。把商相同的一段合并, 系数为区间长度, 递归参数严格变小; 0 直接查表返回 0。超过预筛上限的结果在返回时写入缓存, 后续相同参数直接复用。WIDA 收集的 `sumMu` 示例查询了 `fMu`, 但未写入新结果; 本库已有实现包含写回, 不能照搬遗漏缓存的版本。

设最大查询为 N , 预筛上限为 B 。单次查询的常用上界为 $O(B + N/\sqrt{B})$, 其中哈希访问按期望常数处理; 取 B 约 $N^{2/3}$ 得到 $O(N^{2/3})$ 。多个不同查询可能继续增加缓存, 不能把单次空间估计当成任意多询问的上界。保留缓存可复用, 主动清空仅影响重算开销; 预筛表与上限应视为只读。

接口范围为 $0 \leq n \leq 10^{11}$, 返回精确整数; 欧拉函数前缀和用 128 位。WIDA 的 Z 模数结果可在精确莫比乌斯前缀和返回后取模得到。负值先求余, 仅在余数为负时加模数, 避免无条件加模数的溢出风险。

两套实现以独立试除构建到 200000 的前缀参考, 检查四种预筛上限、零输入、乱序与重复查询、缓存写回及清空重建。上限 10^{11} 使用 10^6 与 2×10^6 两种预筛规模核对双版本结果; 这一项属于边界运行与一致性证据, 不是独立答案对拍。普通与 ASan/UBSan 均通过, 暂无该接口的在线 AC。

8.3 64 位判素与质因数分解

本库判素采用七个固定底数的 Miller-Rabin, 覆盖无符号 64 位整数:

2, 325, 9375, 28178, 450775, 9780504, 1795265022.

将 $n-1 = d2^s$ 、 d 为奇数, 检查 a^d 及后续平方链。小于 2 的输入不视为素数; 底数模 n 为 0 时跳过该底数, 不可误判为合数。模乘用无符号 128 位容纳完整的两个 64 位数之积。固定底数的适用范围依据公开 SPRP 记录, 不外推到任意精度整数。

<https://miller-rabin.appspot.com/>

Pollard-Rho 用 $f(x) = x^2 + c \pmod n$ 迭代, 并求 $\gcd(|x-y|, n)$ 。只有 $1 < d < n$ 的因子才返回; 一次尝试超过轮数上限或得到 $d = n$, 就换参数重试。其中模平方后的加法也在 128 位中执行, 避免 n 接近 2^{64} 时加 c 溢

出。该过程保证返回真因子，但随机尝试次数没有确定的最坏上界。递归分解两侧，直到判为素数，最后排序并保留重复因子。

调用 `factor(1)` 得到空数组；对 $n \geq 2$ ，返回单元素说明 n 为素数。POJ 1811 所需的最小质因子可取排序结果首项；P4718 所需的最大质因子可取末项。不要对空数组取首尾，也不要绕过 `factor` 把素数传给内部 `rho`。

双版本对百万范围以独立埃氏筛核验判素，对 $1 \dots 10000$ 以试除核验完整因子列表。另验证三个随机种子下的大质数平方、重复因子、强伪素数和 $2^{64} - 1$ ；期望因子的素性均以试除独立验证。 $2^{61} - 1$ 使用完整分解 $n - 1$ 的 Lucas 素性证书检查；模乘与模幂另以任意精度运算对拍。完整 POJ 1811 与 P4718 驱动分别验证 350 组输入，检查素数标签与最小、最大质因子。P4718 压力批次含 300 个接近 10^{18} 的平衡半素数，本机普通编译运行约 1.99、1.83 秒；机器差异与随机路径都会影响耗时，本地运行时间不直接代表线上性能。两套代码随后通过 P4718 全部 20 个测试点，最长单点显示约 2.11、2.12 秒，低于当前页面的 4 秒限制。该 AC 检查素数标签与最大质因子，范围为 $2 \leq n \leq 10^{18}$ 。完整因子重数、最小质因子、 $n = 1$ 与更大的无符号 64 位输入仍依据本地验证；普通与 ASan/UBSan 均通过，不构成随机运行时间保证。

来源：kuangbin 2.6；固定底数记录见上方链接。

8.4 单个数的欧拉函数

实现见第 7 页。对 $n \geq 1$ ，由容斥得到

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right),$$

其中只枚举不同质因子，故质数幂 p^k 的贡献为 $p^{k-1}(p-1)$ 。每次更新按先除后乘的顺序： $\text{ans} \leftarrow (\text{ans}/p)(p-1)$ 。当前尚未处理的质因子仍整除答案，而且答案只会减小，乘法结果不会超过原始 n 。

无分解器参数的入口采用试除，循环上界用 $p \leq n/p$ ，剩下的 $n > 1$ 再处理一次。这能避免 p^2 溢出，但不消除 $O(\sqrt{n})$ 的耗时；不适合试除 64 位大素数。传入现有 PollardRho 对象时先分解，再在排序后的因子列表上去重更新；不重复构造分解器，后续调用沿用其随机状态。该入口没有确定的最坏运行时间。 $\varphi(1) = 1$ ，两个入口均接受完整的正无符号 64 位输入；0 不在定义域内。

本地双版本验证包含 $1 \dots 20000$ 的线性筛交叉检查、小范围直接计数互质整数，以及重复质因子、大质数平方、 $2^{64} - 1$ 和已独立认证的大素数。普通与 ASan/UBSan 测试均通过；P4718 的 AC 只验证底层分解用途，不是此欧拉函数的新 AC。来源：kuangbin 2.7.1、2.7.3；WIDA 的单个数欧拉函数。

8.5 线性筛：三张表的约定

LinearSieve 同时构造最小质因子、欧拉函数和莫比乌斯函数。内层遇到第一个整除 i 的素数后停止，因此每个合数 j 只由 $p = \text{lp}(j)$ 与 $i = j/p$ 生成一次，时间为 $O(n)$ 。若 $p \mid i$ ，则 $\varphi(ip) = p\varphi(i)$ 、 $\mu(ip) = 0$ ；否则二者分别为 $(p-1)\varphi(i)$ 和 $-\mu(i)$ 。

判素使用 $x \geq 2$ 且 $\text{lp}(x) = x$ ，不能漏掉前半条件。分解表内正整数时，反复取 $p = \text{lp}(x)$ 、令 $x \leftarrow x/p$ ，直到 $x = 1$ ；重复因子会重复输出。0 不可分解，1 的质因子列表为空。上限要求 $0 \leq n < \text{INT_MAX}$ ，避免构造时 $n+1$ 溢出，并保证内存可用。筛乘积前先检查 $p \leq n/i$ ，不直接计算可能溢出的 ip 。

双版本验证至 $n = 10^7$ ，包括小前缀、试除最小质因子、因子还原和逐项约数和证书：

$$\sum_{d|x} \varphi(d) = x, \quad \sum_{d|x} \mu(d) = [x = 1].$$

这些恒等式在 $d = x$ 项系数为 1，可按 x 递增唯一确定两张表。普通与 ASan/UBSan 均通过；不据此宣称莫比乌斯反演的所有应用已验证。

来源：kuangbin 2.1.1、2.1.2、2.7.2、2.7.4、2.13.1；WIDA 的全部欧拉函数表。

WIDA 的“使用莫比乌斯反演求解欧拉函数”先存储全部约数，再使用

$$\varphi(n) = n - \sum_{d|n, d < n} \varphi(d).$$

这与 $\varphi * 1 = \text{id}$ 等价；再卷积 μ 可得 $\varphi(n) = \sum_{d|n} \mu(d)(n/d)$ 。本库用线性筛提供同一张表，无需额外保存约数列表。百万范围另分别运行直接卷积和真约数扣减，逐项核对两套筛表；两种方法都不复用线性筛的递推分支。

8.6 区间筛：只保存目标区间

实现见第 9 页。设 $0 \leq L \leq R \leq \text{INT_MAX}$ ，先筛出不超过 $\lfloor \sqrt{R} \rfloor$ 的素数，再对每个素数 p 从

$$\max(p^2, \lceil L/p \rceil p)$$

开始标记区间内的倍数。任何合数都有不超过自身平方根的素因子，所以不会遗漏；从 p^2 开始也不会误删 p 自身。输出跳过 0 和 1，其余未标记数恰为素数。底筛与区间标记共用同一个外层素数循环，但使用两张独立的表。

令 $W = R - L + 1$ ，空间为 $O(\sqrt{R} + W)$ ，包含返回的素数数组；时间为 $O((\sqrt{R} + W) \log \log(R + 3))$ 。函数每次调用都重建底筛，适合题目中的少量区间查询。端点虽然是 `int`，向上取整的加法、 p^2 、标记变量和输出遍历变量均用 `long long`；否则 $R = \text{INT_MAX}$ 时最后一次递增也会溢出。区间长度仍受可用内存约束，不适合一次保存整个 32 位值域。

POJ 2689 与 UVa 10140 的驱动读到 EOF，扫描相邻素数差。最近、最远都只在严格更优时更新，距离并列保留最先出现的一对；少于两个素数时输出题目指定句子。两套实现以试除和 Miller–Rabin 判素进行独立对拍，并检查平方边界、单点区间及右端点为 `INT_MAX` 的百万长度区间；完整驱动验证 208 组输入，含官方样例。普通测试与 ASan/UBSan 均通过；尚无本库在线 AC。

来源：kuangbin 2.1.3；<https://onlinejudge.org/external/101/10140.pdf>。

8.7 矩形 GCD 计数：缩放后容斥

实现见第 32 页。由 $[g = 1] = \sum_{d|g} \mu(d)$ ，前缀矩形互质数对数为

$$F(A, B) = \sum_{i=1}^{\min(A, B)} \mu(i) \left\lfloor \frac{A}{i} \right\rfloor \left\lfloor \frac{B}{i} \right\rfloor.$$

两个商同时不变的右端点取 $r = \min(A/\lfloor A/l \rfloor, B/\lfloor B/l \rfloor)$ ，用莫比乌斯前缀和合并 $[l, r]$ 。不能只按一个商分块。只需筛到 $\min(A, B)$ ，不要求较长的一边也在预处理范围内。 $A = 0$ 或 $B = 0$ 时循环为空，返回 0；正整数条件不包含坐标轴。

矩形内 $\gcd(x, y) = k$ 等价于 x, y 均为 k 的倍数，且缩放后互质。因此答案为四个前缀矩形的容斥：

$$\begin{aligned} & F(\lfloor b/k \rfloor, \lfloor d/k \rfloor) - F(\lfloor (a-1)/k \rfloor, \lfloor d/k \rfloor) \\ & - F(\lfloor b/k \rfloor, \lfloor (c-1)/k \rfloor) + F(\lfloor (a-1)/k \rfloor, \lfloor (c-1)/k \rfloor). \end{aligned}$$

这里必须先减 1 再除以 k ；零商无需特殊补偿。输入端点为正 `int`，最终答案不超过矩形面积，能用有符号 64 位保存。分块累加使用 128 位，避免依赖正负中间项的额外界。

双版本以小矩形逐对求 $\gcd$ 为独立参考，另验证百万正方形的 $F(N, N) = 2 \sum_{i=1}^N \varphi(i) - 1$ ，以及按短边剩余类直接计数的 `INT_MAX` 长条。P2522 双版本完整程序验证五万次查询，包括缩放、窄边和无解；普通与 ASan/UBSan 均通过。尚无本库在线 AC。

来源：kuangbin 2.13.2 (BZOJ 2301)；<https://www.luogu.com.cn/problem/P2522>。

8.8 整数分拆：重数限制的差一问题

实现见第 52 页。分拆不区分加数顺序，部分均为正整数。无约束生成函数 $P(z) = \prod_{i \geq 1} (1 - z^i)^{-1}$ ，空分拆给出 $p(0) = 1$ 。五边形数定理为

$$\prod_{i \geq 1} (1 - z^i) = 1 + \sum_{j \geq 1} (-1)^j \left(z^{j(3j-1)/2} + z^{j(3j+1)/2} \right).$$

与 $P(z)$ 相乘后比较系数, 得到 $p(n)$ 的递推, 符号按 $+, +, -, -, +, +, \dots$ 成对出现。每项只依赖更小的下标, 每个 n 有 $O(\sqrt{n})$ 个非零偏移。实现先用 64 位累加, 再归一化到模数范围; 无需逆元, 合数模数与模数 1 均可使用。每个正整数最多出现 k 次的生成函数是

$$\prod_{i \geq 1} (1 + z^i + \dots + z^{ki}) = P(z) \prod_{i \geq 1} (1 - z^{(k+1)i}).$$

因此用已有 p 表与放大 $k+1$ 倍的五边形偏移相乘, 符号为 $-, -, +, +, \dots$ 。本库 `limited(n,k)` 的上限包含 k ; $k=0$ 时非零 n 无解, $k=1$ 为互异分拆。kuangbin 的注释写“不超过 k 个”, 但其 `solve(n,k)` 使用放大 k 倍的公式, 实际表示每种数少于 k 次, 对应本库 `limited(n,k-1)` ($k \geq 1$)。HDU 原题页面本次无法读取, 不能由这条注释推断原题措辞。

两套实现以任意精度完全背包验证到 1000, 以逐种部分、逐个重数枚举验证有界情况到 100, 覆盖素数与合数模数、0/1 重数限制和大于总和的限制; 另检查 $N = 100000$ 的边界。普通与 ASan/UBSan 均通过, 暂无在线 AC。

来源: kuangbin 2.11; <https://oi-wiki.org/math/combinatorics/partition/>。

8.9 幂的约数和: 不除以 $p-1$

实现见第 2 页。若 $a = \prod p_i^{e_i}$, 则 a^b 的约数和是 $\prod_i (1 + p_i + \dots + p_i^{e_i b})$ 。几何级数的除法公式在模意义下要求分母可逆; 合数模数或 $p \equiv 1$ 时不能直接套用。

令 $G(a, n)$ 同时返回 $P = a^n$ 与 $S = \sum_{i=0}^{n-1} a^i$ 。先递归求一半项数 $h = \lfloor n/2 \rfloor$ 的 (P, S) ; 偶数项的结果为 $(P^2, S(1+P))$, 奇数项再补最后的 $a^{2h} = P^2$, 得到 $(P^2 a, S(1+P) + P^2)$, 全过程取模。 $n=0$ 的结果为 $(1, 0)$, 因此底数为 0 时, 非空级数的和仍为 1。本接口参数是项数, 不是最高指数; 约数和必须传 $e_i b + 1$ 。

把幂与和一起返回, 可使每层只做常数运算, 总时间 $O(\log(n+1))$ 。长度允许无符号 128 位, 避免 $e_i b$ 在 64 位中溢出; 模数及底数允许完整无符号 64 位。所有模乘与必要的加法在 128 位中执行, 任意正模数均可用。底数 $a \geq 1$, $b=0$ 或 $a=1$ 都返回 $1 \bmod m$; 0 的约数和不在接口定义域内。

双版本以直接枚举约数、逐项几何求和和任意精度矩阵幂独立核验, 包括模数 1、合数模数、完整 128 位项数, 以及指数乘重数超过 64 位的案例。P1593 完整程序各验证 126 组输入, 包含 $p \equiv 0, 1 \pmod{9901}$ 和最大输入; 普通与 ASan/UBSan 均通过, 在线 AC 待补。

来源: kuangbin 2.12; <https://www.luogu.com.cn/problem/P1593>。

8.10 类欧几里德: 有符号整除求和

`floor_sum(n,m,a,b)` 返回

$$F(n, m, a, b) = \sum_{i=0}^{n-1} \left\lfloor \frac{ai + b}{m} \right\rfloor.$$

范围约定 $0 \leq n \leq 10^9$ 、 $1 \leq m \leq 10^9$, a, b 支持完整有符号 64 位, 返回 128 位。 n 是项数; 若题目下标为 $0 \dots N$, 需传 $N+1$ 并检查仍在约定范围内。

先令 $a = q_a m + r_a$ 、 $b = q_b m + r_b$, 其中余数属于 $[0, m)$, 答案先加上 $q_a n(n-1)/2 + q_b n$ 。C++ 的负整数除法向零截断, 因此余数为负时需要把商减 1, 再用 128 位乘法计算规范余数。这一步不能简单用两次取模替代后丢弃的贡献。

对规范化后的 $0 \leq a, b < m$, 设 $y = an + b$ 。若 $y < m$, 余下贡献为 0; 否则交换格点计数的求和方向可得

$$F(n, m, a, b) = F(\lfloor y/m \rfloor, a, m, y \bmod m).$$

下一轮先提取新的整商, 再继续交换。进入交换分支时 $a > 0$, 不会产生零分母。模数按欧几里得过程减小, 时间 $O(\log(m+1))$, 额外空间 $O(1)$ 。

双版本以任意精度向下取整逐项求和对拍, 覆盖负系数、零项数和整除边界; 十亿项的常数、连续整数、反向连续整数及整倍数斜率使用闭式公式独立核验。同时检查模数 1 下两个系数的 64 位极值。普通与 ASan/UBSan 均通过, 暂无在线 AC。本节只验证基本整除和; 下节单独给出带下标权与平方和的接口及证据。

8.11 类欧几里德的三个求和量

实现见第 35 页。令 $y_i = \lfloor (ai + b)/m \rfloor$ ，维护 $f = \sum y_i$ 、 $g = \sum iy_i$ 、 $h = \sum y_i^2$ ，下标均为 $0 \leq i < n$ 。结果固定模 998244353，不要更换模数却保留 2、6 的逆元常数。

先把 $a = xm + a'$ 、 $b = ym + b'$ 归一化；设规范化问题返回 (f_0, g_0, h_0) ，并记 $S_1 = \sum i$ 、 $S_2 = \sum i^2$ ，则

$$\begin{aligned} f &= f_0 + xS_1 + yn, \\ g &= g_0 + xS_2 + yS_1, \\ h &= h_0 + x^2S_2 + y^2n + 2xyS_1 + 2xg_0 + 2yf_0. \end{aligned}$$

负系数的商仍须向下取整。规范化后的 $a' = 0$ 时三个剩余量均为 0。

对 $0 < a' < m$ ，令 $t = \lfloor (a'(n-1) + b')/m \rfloor$ 。高度 $j = 1 \dots t$ 首次出现的下标为 $r_j = \lceil (mj - b')/a' \rceil$ 。以 $q = j-1$ 编号，递归参数为 $(t, a', m, m + a' - b' - 1)$ ；设返回 (u, v, w) ，即 $u = \sum r_j$ 、 $v = \sum (j-1)r_j$ 、 $w = \sum r_j^2$ ，则

$$f_0 = nt - u, \quad g_0 = tS_1 - (w - u)/2, \quad h_0 = nt^2 - 2v - u.$$

这来自按高度计算每一层的贡献，其中 $y_i^2 = \sum_{j=1}^{y_i} (2j-1)$ 。分母每次严格减小，所以只有 $O(\log(m+1))$ 层递归。

P5170 的上界包含 n ，驱动传入 $n+1$ ；题目输出顺序为 f, h, g ，与本接口不同。核心函数支持十亿加一项，并对完整有符号 64 位系数做任意精度逐项对拍；大项数用一次多项式的和与平方和闭式检查。两套完整程序各验证十万组输入，检查最大闭区间端点与全部三个输出。普通与 ASan/UBSan 均通过，在线 AC 待补；任意更高次矩尚不在这个接口内。

补充的大参数测试包含 3000 组随机参数、64 位系数端点和斐波那契型欧几里得链。反转 $i \mapsto n-1-i$ 应保持 f, h 不变，并使 g 变为 $(n-1)f - g$ 。将每项替换为 $-y_i - 1$ ，三个结果应变为 $-f - n$ 、 $-g - S_1$ 、 $h + 2f + n$ 。相邻前缀还应满足增量 (y_n, ny_n, y_n^2) ，其中新增项以任意精度计算。第一结果另与独立接口 `floor_sum` 交叉核对。这些恒等式测试是大参数一致性证据，与前述逐项独立答案对拍区分记录；普通与 ASan/UBSan 均通过。

来源与题目：<https://www.luogu.com.cn/problem/P5170>。

8.12 模运算的适用条件

模逆元存在当且仅当 $\gcd(a, m) = 1$ 。费马逆元 a^{p-2} 仅适用于素数模数及 $a \neq 0$ 。对正模数 m, n ，同余式 $x \equiv a \pmod{m}$ 、 $x \equiv b \pmod{n}$ 可合并当且仅当 $\gcd(m, n) \mid (b - a)$ ；新模数为 $\text{lcm}(m, n)$ 。

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p} \quad (p \text{ 为素数, } n_i, k_i \text{ 是 } p \text{ 进制位}).$$

Lucas 不能直接套在合数模数。阶乘与逆阶乘预处理要求上限小于模数；超过模数会出现阶乘为零。扩展欧拉降幂中，非互质情况不能直接将指数对 $\varphi(m)$ 取模；需要判断原指数是否达到足够阈值。大整数指数可以在读入时同时维护余数与是否超过阈值。

8.13 任意数组的批量逆元

设 $s_i = \prod_{j < i} a_j \pmod{m}$ ， $s_0 = 1$ 。仅对总乘积 s_n 求一次逆元，随后从右向左处理。若当前 $t = s_{i+1}^{-1}$ ，则

$$a_i^{-1} = ts_i \pmod{m}, \quad s_i^{-1} = ta_i \pmod{m}.$$

先保存原 a_i 再覆盖输出，避免误用已写入的逆元更新 t 。总乘积与 m 互素当且仅当每一项均与 m 互素，因此一次扩展欧几里得也能判断整个批次是否可行；模数无需为素数。实现见第 14 页。

接口返回可选向量：无解是空的可选值，空输入是有效的空向量，两者不同。模数为 1 时按接口约定返回全零。调用者可以移动不再使用的输入数组，避免额外复制。P5431 的驱动程序计算 $\sum_{i=1}^n k^i/a_i$ ，注意权重从 k^1 开始。五百万项规模采用缓冲的字符读取，并移动输入以控制峰值内存。双版本已做乘法证书和非互素拒绝的穷举检查，以及 64 位极值、空数组、模数 1 的检查；完整驱动另用逐项逆元和几何级数进行对拍。P5431 双版本均通过

全部 5 个测试点；动态版记录 297561197，传统版记录 297561804。最慢单点分别为 386、384 毫秒，峰值内存均为 76.80 MB。合数模数、不可逆输入和完整 64 位范围以上本地测试验证。

该方法可处理任意可逆数组；连续整数的专用递推见下一节。

来源：<https://oi-wiki.org/math/number-theory/inverse/>。

8.14 连续整数的线性逆元表

素数模数 p 下，对 $1 < i < p$ ，写 $p = qi + r$ ，其中 $q = \lfloor p/i \rfloor$ ， $r = p \bmod i$ 。 r 非零，且 $r < i$ 。将等式模 p 后乘以 $i^{-1}r^{-1}$ ，得到

$$i^{-1} \equiv -\lfloor p/i \rfloor (p \bmod i)^{-1} \pmod{p}.$$

从 $1^{-1} = 1$ 开始递推，右侧总是已算过的较小下标，故总时间为 $O(N)$ 。实现见第 14 页；第 0 项只作占位，允许 $N = 0$ 。 $N \geq p$ 时区间包含无逆元的 p ，不能套用该表。

递推还依赖 r 可逆，不能仅凭 i 与合数模数互素就照搬。例如 $i = 3, m = 8$ 时 $r = 2$ 不可逆。kuangbin 的“简洁写法”递归式也有这一限制，本书以一般扩展欧几里得接口代替其不成立的合数推广。若 $\gcd(a, m) = 1$ ，扩展欧几里得给出 $ax + my = 1$ ，归一化 x 即为逆元；若不互素，则整数方程无解。素数下也可用 a^{p-2} 求逆元；kuangbin 标为“利用欧拉函数”的代码实际使用这一费马形式。

P3811 双版本完整程序逐行输出逆元，已在本地对三百万行结果逐项检验 $ix_i \equiv 1$ 。P3811 双版本均通过全部 6 个测试点，单点最长分别为 221 ms 与 219 ms。单个逆元另用小模数穷举与费马乘法证书交叉验证；这部分不由 P3811 的 AC 覆盖。

来源：<https://oi-wiki.org/math/number-theory/inverse/>；WIDA 数论逆元节与 kuangbin 2.4。

8.15 一次不定方程：特解与正整数解范围

实现见第 12 页。对不全为零的 a, b ，令 $g = \gcd(|a|, |b|) > 0$ 。递归扩展欧几里得先求 $|a|u + |b|v = g$ ；当且仅当 $g \mid c$ 时有整数解。把系数乘以 c/g ，再恢复 a, b 的符号，就得到 $ax_0 + by_0 = c$ 。两个特解相减，再利用 $a/g, b/g$ 互素，可知所有解恰为

$$x = x_0 + \frac{b}{g}t, \quad y = y_0 - \frac{a}{g}t, \quad t \in \mathbb{Z}.$$

若 $a = b = 0$ ，则 $c = 0$ 时所有整数对都是解，否则无解；此时不能使用除以 g 的通解式。

底层 `exgcd` 的本库调用约定为非负系数，上限 2^{63} ；返回非负的 g ， $(0, 0)$ 约定返回 0。包装函数先转为 128 位再取相反数，因而可以处理 `LLONG_MIN`。缩放后的特解也用 128 位保存；不要提前转回 `long long`。即使特解可表示，直接验证 $ax + by$ 的两个乘积仍可能超过 128 位，本地验证使用任意精度整数。

对 P5656 的正系数 a, b, c ，设 $d_x = b/g, d_y = a/g$ ，先将 x_0 调整到 $x_{\min} \in [1, d_x]$ ，并计算与它配对的 $y_{\max} = (c - ax_{\min})/b$ 。若 $y_{\max} > 0$ ，正整数解个数为

$$N = \left\lfloor \frac{y_{\max} - 1}{d_y} \right\rfloor + 1, \quad x_{\max} = x_{\min} + (N - 1)d_x, \quad y_{\min} = y_{\max} - (N - 1)d_y.$$

否则不存在同时为正的解，但 x 和 y 仍各自有最小正值；分别按周期归一化即可，它们通常不是同一组解。正值归一化使用 $(v \% d + d - 1) \% d + 1$ ，不要把余数 0 当作正整数。驱动中 $a, b, c \leq 10^9$ ，归一化后的乘积不超过 10^{18} ，故该部分可用 64 位。

双版本以小范围有符号输入穷举、任意精度等式证书、64 位端点和斐波那契递归链验证。完整 P5656 驱动另验证 200000 组输入，包含正解数量与四个极值、无解和无正解分支；普通与 ASan/UBSan 测试均通过，在线 AC 待补。

来源：kuangbin 2.3；WIDA 数论打印稿的 `exgcd` 与不定方程例题。

<https://oi-wiki.org/math/number-theory/bezouts/>。

8.16 线性同余：最小解与最小周期

实现见第 15 页，接口约定为 $ax + b \equiv 0 \pmod{m}$ 。先把 a 归一化到 $[0, m)$ ，令 $g = \gcd(a, m)$ 。有解当且仅当 $g \mid b$ ；若 $au + mv = g$ ，则一个特解为 $u(-b/g)$ ，所有解为 $x = x_0 + (m/g)t$ 。归一化到 $[0, m/g)$ 后得到最小非负解。除以 g 后 a/g 与 m/g 互素，因而 m/g 是最小正周期。当 $a = 0$ 且 $m \mid b$ 时所有整数均为解，返回 $(0, 1)$ ；否则无解。模数 $m = 1$ 始终返回 $(0, 1)$ 。

输入常数项支持 `LLONG_MIN`，内部用 `-i128(b)`，不能先执行 64 位的 `-b` 再转换。若题目给出 $ax \equiv c \pmod{m}$ ，可先将 c 归一化到 $[0, m)$ 再取负传入，避免 $c = \text{LLONG_MIN}$ 时在调用点溢出。本接口依赖的 `exgcd` 与前节相同，不必重复抄写。

P1516 中相遇条件为 $(m - n)t + (x - y) \equiv 0 \pmod{L}$ 。传入速度差与起点差，第一返回值就是最早非负相遇时间；失败时输出 `Impossible`。不能擅自把负的速度差取绝对值，也不能把两者同余的初始位置强制判为需要跳跃。

小模数测试逐个枚举全部剩余类，检查解集、最小解和最小周期；大数用任意精度等式证书验证。完整驱动以直接模拟、交换青蛙和构造大周期最早相遇时间验证 242 组输入，包括等速和初始位置同余。两套代码均通过普通与 ASan/UBSan 测试，在线 AC 待补。

来源：WIDA 收集的 `jiangly exgcd` 文件内 `sol`；<https://www.luogu.com.cn/problem/P1516>。

8.17 扩展 CRT：逐式合并

kuangbin 2.5 的 `solve` 对应单次合并，MLES 对应下面的逐式循环。实现见第 15 页的 `crt_merge`，初始取 $r = 0, m = 1$ ，依次合并 $x \equiv b \pmod{n}$ 。空方程组保持 $(0, 1)$ ，表示任意整数；一旦无解即停止，不能忽略失败继续合并。先把余数归一化。设 $g = \gcd(m, n)$ ，由扩展欧几里得得到 $mu + nv = g$ 。写 $x = r + mt$ ，原问题化为 $mt \equiv b - r \pmod{n}$ 。若 $g \nmid (b - r)$ 则无解，否则

$$q = n/g, \quad t \equiv \frac{b-r}{g}u \pmod{q}, \quad 0 \leq t < q.$$

新周期为 mq ，新余数为 $(r + mt) \bmod (mq)$ 。所有可行 t 相差 q ，故新余数加新周期的任意整数倍恰好是全部解；逐式合并时这一不变量保持成立。模数不必互素，模数 1 也可直接参与。

参考实现使用 `int` 保存合并状态，本库扩展为 64 位并显式报告周期越界；避免可能溢出的 `abs(a-a0)`。

代码要求正模数且最终周期不超过有符号 64 位上限；中间乘法与扩展欧几里得系数使用 128 位整数。无解返回假；有解而周期越界抛出异常。失败时旧周期不变，旧余数已归一化。

k 条方程的时间为 $O(k \log M)$ ，其中 M 是输入及中间模数的上界；递归欧几里得使用 $O(\log M)$ 栈空间。

P4777 的完整程序按“模数、余数”读入，题目保证有解且周期不超过 10^{18} 。本地验证包含双版本小模数穷举、任意精度整数对拍、有符号极值、无解与越界，以及十万条同余式的完整程序测试。另穷举至多三个小模数方程组成的所有有序系统，检查空系统与后续矛盾。双版本均已通过 P4777 全部 15 个测试点；动态版记录 297521102，传统版记录 297521173。

kuangbin 的“解同余方程组”是模 7 的线性方程组消元应用，见后面的工厂工期一节。来源：<https://oi-wiki.org/math/number-theory/crt/>；WIDA 数论模板的 `exCRT` 节。

8.18 Garner：总周期可以超过整数范围

设正模数 m_i 两两互素， $P = \prod_i m_i$ 。经典构造为 $x = \sum_i b_i P_i (P_i^{-1} \bmod m_i) \bmod P$ ，其中 $P_i = P/m_i$ 。模 m_i 时其他项都为零，本项为 b_i ；两解之差被所有 m_i 整除，因而被 P 整除。但直接写此式需要存储整个 P ，并防止乘法溢出。

Garner 改用混合进制

$$x = d_0 + d_1 m_0 + d_2 m_0 m_1 + \cdots, \quad 0 \leq d_i < m_i.$$

在确定第 i 位前，对每个后续模数维护已确定部分的余数 v_j ，以及当前位权 $c_j = \prod_{t < i} m_t \bmod m_j$ 。于是

$$d_i = (b_i - v_i) c_i^{-1} \bmod m_i.$$

把 $c_j d_i$ 加到 v_j , 再将 c_j 乘 m_i 即可进入下一位。互素性保证逆元存在; 模数 1 的位固定为 0, 可直接保留。位数范围保证重建的 x 恰在 $[0, P)$, 因此就是最小非负解。实现见第 16 页。

若只需 $x \bmod T$, 从高位向低位做 Horner 计算: 每步将当前值乘对应模数, 加上该位, 再对 T 取模。 T 无需与 m_i 互素, 因为这里不对 T 求逆。这返回的是规范解 x 的余数; 一般解 $x + kP$ 对 T 的余数可能不同。例如模 3 余 2、模 5 余 3, 规范解是 8; 对 7 取模为 1, 但另一个解 23 的余数是 2。若用多模 NTT 恢复整数系数, 仍须证明真实系数位于所选规范区间, 不能在乘积不够大时直接把 CRT 的代表元当成原整数。

双版本本地验证使用任意精度整数的经典 CRT 求和作为独立对拍, 检查混合进制的完整重建、负余数、有符号极值、非互素输入拒绝和 1000 个模数。P1495 完整程序按题目保证的 $P \leq 10^{18}$ 输出规范解, 双版本均通过全部 11 个测试点; 动态版记录 297521594, 传统版记录 297521658。超出该题范围的能力以上述本地测试验证。来源: <https://oi-wiki.org/math/number-theory/crt/>。

8.19 素数模平方根

奇素数 p 下, 对 $n \not\equiv 0 \pmod{p}$, Euler 判别式 $n^{(p-1)/2} \equiv 1 \pmod{p}$ 当且仅当存在平方根, 否则为 -1 。当 $p \equiv 3 \pmod{4}$ 且有解时, $r = n^{(p+1)/4}$ 为一根, 另一根是 $p - r$ 。

Cipolla 选择 a 使 $w = a^2 - n$ 是非平方, 令 $\omega^2 = w$ 。在扩域中取 $\alpha = a + \omega$, 有 $\alpha^p = a - \omega$, 故 $\alpha^{p+1} = n$ 。令 $\beta = \alpha^{(p+1)/2}$, 则 $\beta^2 = n$ 且 $\beta^{p-1} = n^{(p-1)/2} = 1$, 所以 β 落在原素数域中。实现见第 20 页; 零和模数 2 单独处理。模数为合数时不能照搬此判别和扩域构造。

8.20 乘法阶与原根

对 $\gcd(a, n) = 1$, 阶是最小的正整数 d 使 $a^d \equiv 1 \pmod{n}$, 且 $d \mid \varphi(n)$ 。从 $d = \varphi(n)$ 开始, 对每个素因子 q , 只要 $a^{d/q} \equiv 1$ 就继续除去 q , 所得即为实际阶。对于不可逆的 a , 接口返回 0, 表示此处没有定义乘法阶。

原根要求阶达到 $\varphi(n)$ 。只需对其每个不同素因子 q 检查 $a^{\varphi(n)/q} \not\equiv 1$: 若实际阶是真因子, 商中必含某个这样的 q 。必须先检查 $\gcd(a, n) = 1$; 例如模 9 的候选 3 虽通过上述非 1 判定, 却不是原根。 $n \geq 2$ 时, 原根恰存在于 $2, 4, p^k, 2p^k$, 其中 p 是奇素数。原根存在且取一个原根 g 后, g^t 的阶是 $\varphi(n)/\gcd(t, \varphi(n))$, 所以枚举与 $\varphi(n)$ 互素的指数即可得到全部原根, 共 $\varphi(\varphi(n))$ 个; 这些数按大小输出时还需排序。

实现见第 21 页。本地验证穷举小模数的幂循环, 与双版本的阶、最小值和完整集合逐项比较; 另验证大素数、奇素数幂、无原根合数和有符号输入边界。P6091 的完整程序输出排序后第 $d, 2d, \dots$ 个原根, 零个输出时仍保留空行。双版本均通过全部 11 个测试点; 动态版记录 297558740, 传统版记录 297559082。求阶、最小原根的最小性及更大输入范围仍以上述本地证据验证。若 $d = \text{ord}_n(a)$, 则 $\text{ord}_n(a^k) = d/\gcd(d, k)$ 。两个可逆元素乘积的阶整除各自阶的最小公倍数; 两阶互素时, 乘积的阶等于两阶之积。来源: <https://oi-wiki.org/math/number-theory/primitive-root/>; kuangbin 2.17。

8.21 Carmichael: 最小公共指数

$\lambda(n)$ 是模 n 的所有可逆元素之阶的最小公倍数, 约定 $\lambda(1) = 1$ 。它整除 $\varphi(n)$, 但可以严格更小, 例如 $\lambda(8) = 2$ 而 $\varphi(8) = 4$ 。按互素的素数幂分解 $n = \prod_i p_i^{e_i}$ 后, 由 CRT, 对每个分量同时成立的幂等式也对 n 成立, 因此

$$\lambda(n) = \text{lcm}_i \lambda(p_i^{e_i}), \quad \lambda(p^e) = \begin{cases} 2^{e-2}, & p = 2, e \geq 3, \\ p^{e-1}(p-1), & \text{其他情形} \end{cases}$$

奇素数幂的单位群有原根, 公共指数即其大小。模 2^e 的奇数可表示为 $\pm 5^k$, 当 $e \geq 3$ 时 5 的阶为 2^{e-2} , 这给出二幂情形的上界和达到上界的元素。实现见第 12 页, 试除版本适用于有符号 32 位正模数。

对可逆底数可把指数对 $\lambda(n)$ 取模; 非互素底数仍须另行处理。例如模 8 下 $2^2 = 4$, 但 $2^{2 \bmod \lambda(8)} = 1$, 直接降幂会错误。另有 $\lambda(\text{lcm}(a, b)) = \text{lcm}(\lambda(a), \lambda(b))$, 这也提供了独立的组合检查条件。

合数 n 是 Carmichael 数当且仅当 $\lambda(n) \mid (n-1)$ 。等价的 Korselt 判别是： n 无平方因子，且每个素因子 p 都满足 $(p-1) \mid (n-1)$ 。素数也满足前一整除式，判定时必须排除素数。这些合数对所有互素底数都通过 Fermat 判定，不能用单纯的 Fermat 检验代替素性判定。无平方因子再配合 CRT，也使所有整数 a 满足 $a^n \equiv a \pmod{n}$ 。

双版本以小模数的全部实际阶之最小公倍数对拍，另检验大二幂、奇素数幂、32 位边界及公共指数证书。

UVa10006 完整程序对全部合法输入进行本地 Korselt 对拍；在线 AC 待补。

来源：<https://oi-wiki.org/math/number-theory/primitive-root/>；

应用题：<https://onlinejudge.org/external/100/10006.pdf>。

8.22 Lucas：按素数进制逐位计算

在特征为素数 p 的域中， $(1+x)^p = 1+x^p$ 。若 $n = \sum_i n_i p^i$ ，则

$$(1+x)^n = \prod_i (1+x^{p^i})^{n_i}.$$

每个因子选择的次数在 $0, \dots, n_i$ 之间，严格小于 p ，因此位数表示唯一。比较 x^k 的系数即得 $\binom{n}{k} = \prod_i \binom{n_i}{k_i} \pmod{p}$ ；若某位 $k_i > n_i$ ，答案立即为零。仅预处理 $0!, \dots, (p-1)!$ 及其逆元，这些数均非零。 $p!$ 为零，不能把阶乘表延长后继续用除法公式。实现见第 17 页。

预处理后每位只做常数乘法，适合固定小素数模数下的多次查询。接口接收 n, k ，不接收“ n, m 表示选 n 个”的隐含约定；P3807 的驱动程序显式调用 $\text{choose}(n+m, n)$ 。题目范围内求和安全，完整 64 位输入自行求和时仍要检查溢出。本地验证包含 Pascal 三角、精确大整数、小模数重建和最大无符号 64 位下标；P3807 双版本均已通过全部 25 个测试点；动态版记录 297556209，传统版记录 297556596。完整 64 位参数及额外边界以上述本地对拍验证。来源：<https://oi-wiki.org/math/number-theory/lucas/>。

8.23 合数模数组合数与插值

对素数幂 $q = p^e$ ，记 $v_p(n!) = \sum_{j \geq 1} \lfloor n/p^j \rfloor$ ， $U_p(n) = n!/p^{v_p(n!)}$ 。设 $t = v_p(n!) - v_p(k!) - v_p((n-k)!)$ 。当 $t \geq e$ 时组合数模 q 为零；否则

$$\binom{n}{k} \equiv U_p(n) U_p(k)^{-1} U_p(n-k)^{-1} p^t \pmod{q}.$$

这里仅对与 p 互素的单位部分求逆。各素数幂上的结果通过 CRT 合并，对应第 18 页的实现。

在素数域中， n 个模 p 两两不同的横坐标确定唯一的次数小于 n 的多项式。定义 $w_i = y_i / \prod_{j \neq i} (x_i - x_j)$ ，则

$$f(k) = \sum_{i=0}^{n-1} w_i \prod_{j < i} (k - x_j) \prod_{j > i} (k - x_j).$$

该式不除以 $k - x_i$ ，查询点命中样本点时仍然成立。当 $x_i = i$ 时，分母为 $i!(n-1-i)!(-1)^{n-1-i}$ ，要求 $n \leq p$ 。实现见第 65 页。对整数幂前缀和 $S_d(k) = \sum_{i=1}^k i^d$ ，当 $0 \leq d \leq p-2$ 时，它在模 p 意义下可用次数至多 $d+1$ 的多项式表示；取 $0, 1, \dots, d+1$ 的样本即可。不能只因某序列前几项能插值，就断言后续仍由该多项式给出。

8.24 阶乘表：组合数与排列数

当素数模数为 p 且 $0 \leq k \leq n < p$ 时，阶乘均非零，因此

$$C(n, k) = n! (k!)^{-1} ((n-k)!)^{-1}, \quad P(n, k) = n! ((n-k)!)^{-1} \pmod{p}.$$

将表从旧上限扩展到 N ，先正向补齐阶乘，只对 $N!$ 求一次逆元，再用 $(i-1)!^{-1} = i (i!)^{-1}$ 反向补齐新逆阶乘项即可。不需要每一项分别快速幂求逆。较小上限的初始化请求保留已有表。查询前显式扩表，之后查询为常数时间；内部两表重分配时旧元素引用不能继续使用。实现见第 49 页。

WIDA 的“组合数 (Lucas)”文件实际提供的是普通阶乘、逆阶乘、组合数和排列数，没有逐位 Lucas 递推。本书以此节覆盖该文件的数学功能，用显式扩表取代查询时自动扩表，适用范围仍须限制在模数以内。验证使用独立 Pascal 三角和下降阶乘，另检查百万项表及扩表前后的原查询结果。

8.25 组合计数

容斥: $|\bigcup A_i| = \sum_{\emptyset \neq S} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$ 。二项式反演: 若 $b_n = \sum_{k=0}^n \binom{n}{k} a_k$, 则 $a_n = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} b_k$ 。

$$\sum_k \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n}, \quad C_n = \binom{2n}{n} - \binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免直接除以 $n+1$, 在模数下尤其有用, 但组合数本身仍须正确求模。第二类 Stirling 数满足

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}, \quad S(0,0) = 1.$$

第一类无符号 Stirling 数把系数 k 换为 $n-1$ 。

球与盒	空盒允许	空盒禁止
不同球, 不同盒	k^n	$k!S(n,k)$
相同球, 不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球, 相同盒	$\sum_{j=0}^k S(n,j)$	$S(n,k)$

上表默认 $k \geq 1$; $n=0, k=0$ 等边界应按原问题约定单独处理。

8.26 群作用计数

有限群 G 作用在有限集合 X 上时, 轨道数为

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

对位置置换的 q 色染色, $|\text{Fix}(g)| = q^{c(g)}$, 其中 $c(g)$ 为循环数。环形项链只商去旋转; 手链还商去反射, 二者群不同, 不能共用分母。模数整除 $|G|$ 时不能使用模逆元, 须先在合适的整数模数下求和再做精确除法。

8.27 生成函数与递推

普通生成函数 $A(x) = \sum a_n x^n$ 的乘法对应普通卷积。指数生成函数 $A(x) = \sum a_n x^n / n!$ 的乘法对应 $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$ 。形式幂级数求逆要求常数项可逆; $\log A$ 取 $A(0) = 1$, $\exp A$ 取 $A(0) = 0$ 。有限域中积分涉及 $1/i$, 必须避免 i 被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2m}}, \quad (\log A)' = A'/A.$$

BM 从样本得到最短线性递推。要对未来项使用, 必须另外证明序列确实具有不超过相应阶数的线性递推; 少量数值吻合不是证明。矩阵转移、有限状态计数常可提供阶数上界。

8.28 线性代数与生成树

域上的线性方程组一致时, 解集为一个特解加零空间, 零空间维数为未知数个数减秩。浮点消元中的秩依赖误差阈值; 模素数消元是精确域运算; 合数模数下不可随意除以非零元。无向图 Laplacian 的对角元为度数, 非对角元为负的边重数。删去同一行列后的行列式为生成树数。自环不参与生成树; 重边需要累加。只有一个顶点时空余子式行列式约定为 1。有向图需要重新明确入度/出度 Laplacian 与根向内/向外树的方向, 不能直接复用无向封装。

8.29 模线性方程组：工厂工期

POJ2947 (问题 UVa1564) 给出每位工人制造的产品类型和起止星期。设第 j 种产品需 x_j 天，第 i 条记录中出现 a_{ij} 次，以星期一为 0、星期日为 6，则

$$\sum_j a_{ij} x_j \equiv \text{finish}_i - \text{start}_i + 1 \pmod{7}.$$

起止两天均计入；同一种产品重复出现必须累加系数。跨星期无需特判，模数类型负责归一化负数。每个余数在 $[3, 9]$ 中恰有一个代表，因此模方程的解数就是原问题的解数。

直接使用第 67 页的模高斯消元，模数取 7。增广矩阵有 m 行、 $n+1$ 列，最后一列是右端项。先判断 `consistent`：假时输出无解；为真且秩小于 n 时输出多解；否则读取特解，余数小于 3 的分量加 7。不能先以秩不足判断多解，因为零系数行仍可能有非零右端项。本题所有非零系数均可逆；合数模数不具备这一性质，不能直接套用此消元。

完整双版本程序位于 `verify/poj/2947.*.cpp`。验证包含所有起止星期组合、产品重复 7 次导致系数归零、矛盾的零行、独立枚举所有 $3 \sim 9$ 天赋值，以及用可逆行变换构造的 300 阶稠密系统。该应用验证覆盖分类及唯一解输出，不代替零空间基等其他接口的验证。来源：<https://onlinejudge.org/external/15/1564.pdf>；kuangbin 2.10.2。

8.30 概率、期望与博弈

期望线性性不要求独立： $\mathbb{E}[\sum X_i] = \sum \mathbb{E}[X_i]$ 。方差可加需要协方差为零；独立是充分条件。非负整数变量 $\mathbb{E}[X] = \sum_{k \geq 1} \Pr(X \geq k)$ 。吸收过程的期望方程 $E_i = 1 + \sum_j p_{ij} E_j$ 需要检查是否几乎必然吸收及期望是否有限；不能对闭合非吸收分量直接求唯一有限解。有限无环公平组合游戏在正常玩法下

$$\text{SG}(u) = \text{mex}\{\text{SG}(v) : u \rightarrow v\}.$$

独立子游戏的 SG 值异或为零当且仅当先手必败。反常玩法、含环游戏、双方可走步不同的游戏不能直接套这个结论。

8.31 线性递推：系数方向与外推条件

约定 $a_i = \sum_{j=1}^k c_{j-1} a_{i-j}$ ，初值是 $a_0, \dots, a_{k-1}$ 。对应特征多项式为

$$F(x) = x^k - c_0 x^{k-1} - c_1 x^{k-2} - \dots - c_{k-1}.$$

若 $x^n \bmod F(x) = \sum_{i=0}^{k-1} r_i x^i$ ，则 $a_n = \sum_{i=0}^{k-1} r_i a_i$ 。代码由高次向低次消去多项式项，第 j 个系数写入原次数减 j 的位置，避免倒置系数。例如 Fibonacci 初值为 $[0, 1]$ ，递推系数为 $[1, 1]$ 。

BM 在域上求有限前缀的最短递推。本书实现要求素数模数；一般的模合数环不能保证非零偏差存在逆元。仅凭若干样本无法证明真实序列以后一直遵循同一个规律。若事先知道序列满足阶数至多 d 的齐次线性递推，至少给出前 $2d$ 项，再用恢复出的系数求后续项。系数向量末尾的零不能擅自删除：它可能对应非平稳的初始段，改变阶数会改变递推开始适用的位置。

8.32 素数模高次剩余与单位根

实现见第 25 页；依赖本书离散对数、线性同余和快速幂。设 p 为素数， g 是原根， $k > 0$ 。 $a = 0$ 时根只有零；否则写 $a = g^b$ 、 $x = g^y$ ，方程转为

$$ky \equiv b \pmod{p-1}.$$

令 $d = \gcd(k, p-1)$ 。当 $d \nmid b$ 时无解；否则指数解为 $y = y_0 + i(p-1)/d$ ， $0 \leq i < d$ 。因此用三个数存全部根：

$$\text{first} = g^{y_0}, \quad \text{ratio} = g^{(p-1)/d}, \quad \text{count} = d.$$

依次乘 ratio 即可枚举；不保证数值递增，也不保证 first 是最小根。ratio 是阶为 d 的单位根，因而枚举恰有 d 个不同的根。模 $p = 2$ 同样有效；指数同余的模数是 1，唯一非零根为 1。

接口支持 $p \leq 10^{12}$ ，需要调用者提供已验证的素数与原根； k 支持完整 unsigned long long，先对 $p-1$ 取模后解同余。 a 支持完整 signed long long，先正规化。返回空 optional 表示无解，返回 $\{0, 1, 1\}$ 表示唯一零根。给定原根后的期望时间、空间为 $O(\sqrt{p})$ ；枚举全部根另外需要 $O(d)$ 时间。模乘须先扩展到 128 位，不可用 64 位乘积直接取模。

测试穷举小于 100 的每个素数、全部原根及小指数，用任意精度模幂逐个枚举候选根并比较整个集合；覆盖负数、64 位边界、模 2、无解与零根。另检查 998244352 个根的紧凑表示，以及素数 999999999989 上的植根测试；后一个素数与原根的条件由试除及乘法阶证书独立确认。普通与 ASan/UBSan 均通过，尚无专门在线 AC。

这是 [OI Wiki 高次剩余](#) 中离散对数转线性同余方法的素数模实现。素数幂及非互素被开方数见下一节；一般合数模数见后文；改良 Tonelli-Shanks 尚需另补，不能把本节状态解读为该页面全部覆盖。

8.33 素数幂模高次剩余与根的提升

实现见第 26 页。令 $M = p^e$ 。当 $a = 0$ 时，全部根是 $p^{\lceil e/k \rceil}$ 的倍数。否则令 $t = v_p(a) < e$ ，必须有 $k \mid t$ ，否则无解。取 $s = t/k$ ， $q = p^{e-t}$ ，写 $x = p^s y$ ，只需求单位方程

$$y^k \equiv a/p^t \pmod{q}.$$

每个单位根 y 对应原模数下的根

$$x = p^s y + \ell p^s q, \quad 0 \leq \ell < p^{t-s}.$$

这些提升保持互不相同，也解释了非互素被开方数可能产生大量根。

奇素数时，提供模 p^e 的原根，约化后仍是模 q 的原根，沿用离散对数转线性同余的方法。 $q = 2$ 时单位根只有 1； $q \geq 4$ 且为二的幂时，单位表示为 $(-1)^z 5^y$ 。符号指数模 2，5 的指数模 $q/4$ ； $q = 4$ 时只有符号分支。分别解两个指数同余即可，不可假设二的高次幂存在原根。

返回结构体保存 first、两个 ratio、两个 count，以及 step、lifts。全部根由下式给出：

$$(\text{first} \cdot \text{ratio}[0]^i \cdot \text{ratio}[1]^j \bmod \text{step}) + \ell \cdot \text{step}.$$

其中 $0 \leq i < \text{count}[0]$ 、 $0 \leq j < \text{count}[1]$ 、 $0 \leq \ell < \text{lifts}$ 。get 已处理模乘溢出，size 返回三者乘积；不保证根按数值排序。零根情形复用同一表示。

模数上限 10^{12} ， k 支持完整无符号 64 位。只检查范围，不检查 p 的素性和奇素数原根的阶，调用者须保证。默认参数 $g = 0$ 仅用于 $p = 2$ ；奇素数不能漏传原根。时间和空间由 BSGS 决定，期望 $O(\sqrt{q})$ ；单次按索引取根使用快速幂，需 $O(\log M)$ 。

已穷举多个素数的全部不超过 256 的素数幂、所有被开方数与小指数，用任意精度模幂直接枚举所有根，比较整个集合。另测有符号与无符号 64 位边界、 2^{39} 的非平凡离散对数、 3^{20} 的非互素方程，以及数万亿根的紧凑表示。普通与 ASan/UBSan 均通过；尚无专门在线 AC。一般合数的 CRT 组合见下一节；改良 Tonelli-Shanks 仍未完成，OI Wiki 页面继续标记部分覆盖。

8.34 一般合数模高次剩余：CRT 组合

实现见第 30 页。调用者提供完整分解 $M = \prod_{t=0}^{F-1} m_t$ ，其中 $m_t = p_t^{e_t}$ 两两互素，并提供每个奇素数幂的原根。先分别求局部根族；任一局部无解，整体即无解。若局部根数为 c_t ，则整体根数为 $\prod_t c_t$ 。只保存局部根族，不创建这个大小的数组。

对每个因子预处理

$$E_t = \frac{M}{m_t} \left(\left(\frac{M}{m_t} \right)^{-1} \bmod m_t \right) \bmod M.$$

对一组局部根 r_t ，唯一对应的标准代表是

$$x = \sum_t r_t E_t \bmod M.$$

逆元存在是因为因子两两互素。`get(index)` 将索引按各个 c_t 转成混合进制数字，再把局部索引拆成两个乘法根族索引和加法提升索引，最后用上述公式组合。CRT 双射保证不重复、不遗漏，输出顺序不保证递增。

$M \leq 10^{12}$ ， $k > 0$ ；空因子列表表示 $M = 1$ ，返回唯一代表 0。`mod` 保存模数，`total` 保存总解数，`parts` 和 `weight` 保存局部根族与 E_t 。模数乘积检查上界，CRT 的乘积先扩展至 128 位。单次按索引查询为 $O(F \log M)$ ；求解的主要时间和空间为局部 BSGS，因子列表接口不包含整数分解与原根搜索的成本；自动准备重载见下一节。

已对 $1 \leq M \leq 180$ 、所有被开方数与小指数直接枚举所有根，与两套代码的完整输出集合比较，并反转因子顺序重新检查。另覆盖有符号与无符号 64 位边界、混合素数幂上的非互素方程和大量零根，以及十一种不同素因子组成模数时的全部 1024 个平方根。普通与 ASan/UBSan 均通过；尚无专门在线 AC。至此已有已知分解及原根时的 BSGS 求根路线，更快的改良 Tonelli-Shanks 等内容仍未覆盖，OI Wiki 条目保留部分完成状态。

8.35 自动准备模数分解与原根

实现见第 29 页。`root_factors(M, rho)` 把 $M \leq 10^{12}$ 分解为互异素数的幂，返回合数求根需要的 (p, e, g) 列表；模数 1 返回空列表。二的幂不需要原根，固定填零。奇素数幂 $m = p^e$ 的乘法群是循环群，设 $\varphi = m/p \cdot (p-1)$ 。分解 φ 后，候选 g 为原根当且仅当

$$p \nmid g, \quad g^{\varphi/q} \not\equiv 1 \pmod{m} \quad \text{对每个不同素因子 } q \mid \varphi.$$

从 2 起枚举候选，返回最小原根；所有奇素数幂均有原根。这里检查的是模 p^e 的阶，不能仅取一个模 p 的原根而不验证提升。

重载 `CompositeRoots::solve(a, k, M, rho)` 自动调用此准备过程。

随机分解工作区类型为 `PollardRho`。如果需要在同一个模数下处理多个方程，先保存因子列表，调用原有列表接口，避免重复分解和原根搜索。准备成本包括 M 与各 φ 的因数分解，以及逐个候选的模幂检验；不能把这部分直接算作常数时间。

独立试除验证 $1 \dots 500$ 的全部模数与多组接近 10^{12} 的边界值，并用任意精度模幂验证返回原根的阶。小模数进一步检查更小候选不能生成全部单位。直接传模数的两套求根接口，在小模数上与全部候选根的枚举结果比较。普通与 `ASan/UBSan` 均通过；尚无自动准备入口的在线 AC。

8.36 带权矩阵树定理与方向约定

实现见第 95 页。返回的是每棵树的边权乘积，再对所有树求和，并对素数取模。无权计数只需把所有边权设为 1。重边是不同选择，自环不属于任何生成树；单点图的空树乘积为 1。零权和负权都按域上的运算处理，因此结果为零可能来自抵消，不能作为连通性判定。

无向边 uv 权值 w 对拉普拉斯矩阵贡献为 L_{uu}, L_{vv} 各加 w ， L_{uv}, L_{vu} 各减 w 。删除 root 行和列后求行列式，与 root 的选择无关。对于每个点都能沿有向边走到根的树形图，边 $u \rightarrow v$ 贡献 $L_{uu} += w, L_{uv} -= w$ ；删除根行列后得到内向树形图的权值和。外向树形图先反转所有边，再使用同一构造。

例如只有一条边 $0 \rightarrow 1$ 时，内向根 1 的答案为该边权，内向根 0 为零；外向根 0 的答案为该边权。P6178 指定的是以 1 为根的外向树，驱动把点号减一，并选择 `away_from_root`。只直接构造 $(n-1) \times (n-1)$ 余子式，时间 $O(n^3 + m)$ 、额外空间 $O(n^2)$ 。行列式复用本书的素数模高斯消元，不支持将 mod 随意改成合数。

测试独立枚举 $n-1$ 条边的所有选择，检查连通性或根向约束，用任意精度整数计算边权乘积之和，再分别对多个素数取模。两套实现覆盖所有根、三种方向、自环重边、负权及 64 位权值边界，并通过 220 点完全图的加权 Cayley 公式检查。P6178 完整驱动另测 300 点、十万条边以及输入输出约定。普通与 ASan/UBSan 均通过。两套代码已在 P6178 通过全部 11 个测试点：[动态版记录 297619886](#)、[传统版记录 297620263](#)。单点最大用时分别为 64 ms、62 ms，内存峰值为 2.85 MB、2.40 MB。在线测试覆盖无向与根为 1 的外向树；其他根、内向树和负权仍使用本地证据。定理背景可参考 [OI Wiki 矩阵树定理](#)。

8.37 任意正模数行列式

实现见第 56 页。素数模高斯消元可以除以任意非零主元，合数模数没有这个性质。本节只使用倍加行和交换行，不求逆元。先把元素正规化到 $[0, m)$ ，对第 i 列的两个元素 a, b ，用整数商 $q = \lfloor a/b \rfloor$ 执行 $R_i \leftarrow R_i - qR_j$ ，再交换 R_i, R_j 。该列的新数对为 $(b, a \bmod b)$ ，所以循环按欧几里得算法终止。其余元素每步对 m 取模，只改变整数代表，不改变行列式模 m 的值。

倍加行不改变行列式；交换行改变符号。消去所有下三角元素后，乘上对角元素及累计符号即得结果。剩余部分的该列全零时直接返回零，不需要判断某个非零元素是否与模数互素。例如 $\text{diag}(2, 3)$ 在模 8 下的行列式为 6，不能因为第一个主元没有逆元就判定失败。

支持任意正 signed 64 位模数与完整 signed 64 位输入元素。正规化后单个乘积小于 2^{126} ，行更新差值和结果乘积都放入有符号 128 位。空矩阵按约定返回 $1 \bmod m$ ；模 1 的所有返回值为零。时间上界 $O(n^3 \log m)$ ，矩阵副本占 $O(n^2)$ 空间。本节可以用于合数模的拉普拉斯余子式，但之前的 `MatrixTree<mod>` 接口仍使用素数模高斯，不能仅把其模板参数改成合数；合数模使用下一节的独立接口。

测试用排列定义穷举所有元素属于 $\{-1, 0, 1\}$ 的三阶矩阵，并用任意精度 Bareiss 消元验证随机矩阵。覆盖不可逆主元、奇异矩阵、模 1、有符号 64 位边界、Fibonacci 长欧几里得过程，以及 300 阶对角矩阵和完全图余子式的 Cayley 公式。两套版本普通与 ASan/UBSan 均通过；尚无专门在线 AC。行变换依据可参考 [OI Wiki 行列式](#)。

8.38 任意正模数的矩阵树计数

实现见第 96 页。拉普拉斯余子式表示生成树权值和的恒等式本身不要求模数为素数；需要换掉的是依赖模逆元的行列式算法。本节沿用无向、内向与外向三种构造，但使用欧几里得行列式消元。输入边权和正模数均支持 signed 64 位，接口名为 `MatrixTreeMod::count`。

先把边权正规化到 $[0, m)$ ，每次矩阵元素加减仍先扩展至 128 位，再取模存回 64 位。重边的权值和可能超过 64 位范围，不能先累加到普通整数中。只构造删除根后的余子式，并将其存储直接交给行列式函数。单点返回 $1 \bmod m$ ，模 1 返回零。负权与零权仍按代数权值处理，返回零不能直接解释为没有生成树。

独立参考程序逐个枚举边集，用任意精度整数计算权值和。已检查多个合数模数、所有根与三种方向、断图、重边自环及 64 位极值权重；另以大权值完全图的 Cayley 公式验证 160 点规模。普通与 ASan/UBSan 均通过，尚无该接口的在线 AC。此前 P6178 的 AC 属于素数模版本，不自动转移到本接口。

8.39 斯特林数的快速整行计算

第二类实现见第 50 页，第一类实现见第 51 页。本节默认模数 $p = 998244353$ ，允许用模板参数指定素数与原根，复用参数化 NTT。第二类 $S(n, k)$ 计数无标号非空集合划分；第一类 $c(n, k)$ 计数排列的循环数。两者不能互换，第一类本节返回无符号版本。

第二类的容斥公式为

$$S(n, k) = \sum_{i=0}^k \frac{i^n (-1)^{k-i}}{i! (k-i)!}.$$

分别构造 $a_i = i^n/i!$ 和 $b_i = (-1)^i/i!$ ，一次卷积取前 $m+1$ 项即可。 n 支持无符号 64 位； $i > 0$ 时指数对 $p-1$ 取模后快速幂， $i = 0$ 时单独区分 $n = 0$ 和 $n > 0$ ，不可直接套用费马降幂。 $S(0, 0) = 1$ ，正 n 的 $S(n, 0) = 0$ ， $k > n$ 时为零。令 L 为 NTT 长度上限，要求 $2m+1 \leq L$ ；默认模数下即 $m < 2^{22}$ 。

无符号第一类是升阶乘多项式的系数：

$$x(x+1)\cdots(x+n-1) = \sum_{k=0}^n c(n, k)x^k.$$

令 $F_m(x) = x(x+1)\cdots(x+m-1)$ ，倍增式为 $F_{2m}(x) = F_m(x)F_m(x+m)$ ；奇数长度再乘 $x+2m$ 。平移系数满足 $[x^i]F_m(x+m) = \sum_{j=i}^m f_j j! m^{j-i}/(i!(j-i)!)$ ，将 $f_j j!$ 倒序后与 $m^j/j!$ 卷积即可。每层只递归计算一次半长多项式，总时间 $O(n \log n)$ ，空间 $O(n)$ 。要求 $n+1 \leq L$ ，默认模数下即 $n < 2^{23}$ ； $n = 0$ 返回常数多项式 1。如果需要下降阶乘对应的带符号数，将第 k 项乘 $(-1)^{n-k}$ 。

验证使用小规模集合划分、排列循环的直接枚举，以及 300 行的递推表；大指数第二类另用独立转移矩阵快速幂验证。规模检查包含二十万项第二类行与两万项第一类行，用幂函数的下降阶乘展开和升阶乘求值恒等式检查。普通与 ASan/UBSan 均通过。P5395 和 P5408 使用模 167772161，现已支持指定模板参数 167772161。两题均获两套码风在线 AC、各 10 个测试点全过：第二类动态版与 传统版，第一类动态版与 传统版。OJ 覆盖题面模数与行长，其他模数和第二类完整无符号指数仍由本地证据支持。该模数已测试二十万项第二类行与 262143 阶第一类行，普通与消毒器均通过。按列计算及 OI Wiki 的其他斯特林数应用仍待补，[对应条目](#)保留部分覆盖状态。

8.40 参数化 NTT 与模数适配

实现见第 36 页。传入素数 p 和原根 g ；长度 n 必须为二次幂且整除 $p-1$ 。代码自动计算 $L = 2^{v_2(p-1)}$ 作为长度上限，不代表分配这个长度的数组没有成本。两个非空数组的普通卷积需要补齐到不小于长度和减一的二次幂。模 998244353 的 $L = 2^{23}$ ，模 167772161 的 $L = 2^{25}$ ，两者均可用原根 3。不能只改模数而忽略原根和长度条件；接口只检查基本范围，不检查素性或原根的阶。

正变换按 g 的相应单位根计算，逆变换使用逆单位根并乘 n^{-1} 。模数支持 `int` 范围内的素数，模整数的加减和乘积使用 64 位中间量。蝶形层以半长度递增，避免最后一层之后的有符号整数溢出。

独立参考用任意精度整数直接计算 DFT 和朴素卷积，同时验证正逆变换复原。已检查七组素数和原根，包括模 2、3、17、97，斯特林数的默认模数与洛谷题目模数，以及 2013265921、原根 31。这些测试中的素性及原根条件另用试除和乘法阶证书确认。大规模全一数组卷积逐项比较已知三角形系数，检查补齐长度的边界。两套实现普通与 ASan/UBSan 均通过。

8.41 Bostan–Mori 与快速递推单项

代码见第 47 页。对于 $a_i = \sum_{j=1}^k c_{j-1} a_{i-j}$ ，令 $Q(x) = 1 - \sum_{j=1}^k c_{j-1} x^j$ ， $P(x) = (Q(x) \sum_{i=0}^{k-1} a_i x^i) \bmod x^k$ ，则 $[x^n]P/Q = a_n$ 。注意系数按前一项、前两项的顺序给出，初值从 a_0 开始。

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)} = \frac{U_0(x^2) + xU_1(x^2)}{V(x^2)}.$$

分母的奇数次项抵消，因此只保留分子与 n 同奇偶的项，分母保留偶数项，再将 n 右移一位。最终 $n = 0$ 时答案是 $P(0)/Q(0)$ 。实现不需要分子次数小于分母次数，空分子直接返回零；分母常数项必须非零。每轮系数长度不增加，初始最大长度 d 必须满足 $2d - 1$ 不超过 NTT 容量。总时间 $O(d \log d \log(n + 1))$ ，空间 $O(d)$ ； $n = 0$ 直接取常数项。

独立验证包括按定义做形式幂级数除法、友矩阵快速幂，以及 $(1 - x)^{-32000}$ 的二项式系数闭式；覆盖完整无符号 64 位下标、常数分母、不等于 1 的非零常数项、空分子、不真分式、零递推系数和多种 NTT 素数。P4723 驱动读取有符号系数并归一化；两套版本均获在线 AC：动态版与 传统版 各 5 个测试点全部通过，最慢单点分别为 907 ms、905 ms。OJ 覆盖模 998244353、 $n \leq 10^9$ 、 $k \leq 32000$ ；其他模数、完整 64 位下标与通用有理函数接口的扩展范围由本地测试支持。OI Wiki 线性递推 中的连续区间取项与其他快速算法仍待补，不将单项算法视为整页覆盖。

8.42 集合变换与不相交子集卷积

代码见第 39 页与第 40 页。长度 $N = 2^m$ 确定全集的位数，数组第 s 项对应集合 s 。OR 变换计算 $\hat{a}_s = \sum_{t \subseteq s} a_t$ ，AND 变换计算所有超集的和；逆变换将逐位加法改成减法。XOR 变换的系数是 $(-1)^{\text{popcount}(s \& t)}$ ，蝶形将 (u, v) 变为 $(u + v, u - v)$ ，逆变换每层再除以 2。因此 OR、AND 可用于任意模数，XOR 需要 2 可逆；本实现限制为奇数模数，使用 $(\text{mod} + 1)/2$ ，不对合数模数使用费马小定理。三类卷积均是变换后逐项相乘，再逆变换。

$$c_s = \sum_{t \subseteq s} a_t b_{s \setminus t} \quad \text{要求两个集合不相交。}$$

普通 OR 卷积允许重叠，不能直接用于上式。将输入按集合大小分层， $A_{i,s} = [|s| = i]a_s$ ， B 同理。每层 OR 变换后，对层数做卷积： $H_{k,s} = \sum_{i=0}^k \hat{A}_{i,s} \hat{B}_{k-i,s}$ 。对每个 k 逆 OR 变换，得到并集恰为 s 、两集合大小之和为 k 的贡献。取 $k = |s|$ ，由 $|u| + |v| = |u \cup v| + |u \cap v|$ 得到交集为空。时间 $O(m^2 2^m)$ ，空间 $O(m 2^m)$ ；本实现逐层复用结果缓冲区，不用第三组二维数组。 $m = 20$ 时两组分层数组本身占 168 MiB，另计输入和线性缓冲。

验证独立枚举变换系数、所有下标对与子掩码，并检查复合模数与 `INT_MAX` 模数。大规模使用 $a_s = x^{|s|}$ 、 $b_s = y^{|s|}$ 的逐位乘积恒等式：OR 为 $(x + y + xy)^{|s|}$ ，AND 为 $(xy)^{|s|}(1 + x + y)^{m - |s|}$ ，XOR 为 $(x + y)^{|s|}(1 + xy)^{m - |s|}$ ，不相交子集卷积为 $(x + y)^{|s|}$ 。P4717 两套版本各 10 点在线 AC：动态版和 传统版，最慢单点 84 ms、81 ms。

P6097 两套版本各 5 点在线 AC：动态版和 传统版，最大测试点显示 2.70 s、2.69 s，内存 188.83 MB、189.00 MB。两题 OJ 仅验证各自题面模数和长度范围，其他模数及长度 1 仍由本地独立测试覆盖。OI Wiki FWT 的其他运算及更广泛应用仍待覆盖。

8.43 Taylor 平移与等比点求值

代码见第 48 页和第 42 页。多项式使用升幂系数，输入长度 n 表示次数小于 n ，允许末尾零。平移输出的第 i 项满足

$$[x^i]f(x+c) = \frac{1}{i!} \sum_{j=i}^{n-1} f_j j! \frac{c^{j-i}}{(j-i)!}.$$

倒序 $f_j j!$ ，与 $c^j/j!$ 卷积，再取第 $n-1-i$ 项并乘 $1/i!$ 。保留原数组长度；零多项式可以用空数组表示，返回空数组。使用 NTT 素数及原根，要求 $2n-1$ 不超过变换容量；此时所需阶乘非零。时间 $O(n \log n)$ 、空间 $O(n)$ 。这与给出 $f(0), f(1), \dots$ 后平移点值的任务不同，后者尚待补充。

Chirp Z 返回 $f(ar^i)$ ， $0 \leq i < m$ 。设 $T_j = j(j-1)/2$ ，由 $ij = T_{i+j} - T_i - T_j$ 得

$$f(ar^i) = r^{-T_i} \sum_{j=0}^{n-1} f_j a^j r^{-T_j} r^{T_{i+j}}.$$

将第一项数组倒序，与 r^{T_j} 卷积，抽取第 $n-1+i$ 项。三角数幂通过累乘生成，不计算可能溢出的 $j(j-1)$ 。令 L 为不小于 $n+m-1$ 的最小二次幂，直接做模 x^L-1 的循环卷积。普通乘积最高次数为 $2n+m-3$ ，而所需区间任意下标 $t \geq n-1$ 满足 $t+L \geq 2n+m-2$ ，因此不会收到高次项回卷，所取系数仍然精确。要求 $n+m-1$ 不超过 NTT 容量，总时间 $O((n+m) \log(n+m))$ 、空间 $O(n+m)$ 。

约定 $r^0 = 1$ ，故 $r=0$ 时第一项为 $f(a)$ ，余项均为 $f(0)$ ； $a=0$ 时所有点为零， $r=1$ 时所有点相同。这些情况单独计算，避免对零求逆。 $m=0$ 返回空数组，空输入返回 m 个零。同一份算法支持小阶公比、重复点与有限域中多次绕回，输出保持原顺序。

独立验证采用 Pascal 二项式展开和 Horner 求值，涵盖 7 种 NTT 素数、零公比与单位公比、小素数完整容量边界。大规模平移用 $(x+b)^d$ 检查每个系数；等比点求值用几何级数闭式逐点检查。普通与 ASan/UBSan 均通过。Library Checker 的 [Polynomial Taylor Shift](#) 和 [等比点求值](#) 已准备 vector 完整驱动；本地检查各题 524288 的最大规模，尚无在线 AC。参考OI Wiki [平移](#)及 [Chirp Z](#)，更广泛的点值平移和应用另行覆盖。

第一部分

跨类依赖

附录 A 树状数组

A.1 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。T 须能从 0 构造并支持 $+=$ 与减法；可直接使用 `ModInt` 维护模数下的区间和。 kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b; b >>= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```

附录 B 矩阵树定理

B.1 带权矩阵树：无向树与内外向树形图

声明 `MatrixTree<mod>`，调用 `count(n, edges, root, kind)` 返回所有合法树的边权乘积之和，`mod` 必须是素数。`Edge=(u,v,Z)`，点号 `0..n-1`， $n \geq 1$ ；`Kind::undirected` 为无向生成树，`toward_root` 为每点沿边能到根，`away_from_root` 为根沿边能到每点。后两种使用输入边方向，不能混淆。重边按不同边计数，自环忽略；单点答案 1。支持模意义下的零权和负权，结果 0 不一定表示不存在树。无向图删除哪个根行列均可；有向图 `root` 决定答案。依赖 `det_prime` 的素数模行列式， $O(n^3+m)$ 时间、 $O(n^2)$ 额外空间。

```
1 template <int mod> struct MatrixTree
2 {
3     using Z = ModInt<mod>;
4     using Edge = tuple<int, int, Z>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    // Sum of products of edge weights; vertices 0..n-1, prime modulus.
13    static Z count(int n, const vector<Edge> &edges, int root, Kind kind)
14    {
15        assert(n >= 1 && root >= 0 && root < n);
16        vector<vector<Z>> lap(n - 1, vector<Z>(n - 1));
17        auto add = [&](int u, int v, Z w)
18        {
19            if (u == root) return;
20            int i = u - (u > root), j = v - (v > root);
21            lap[i][i] = lap[i][i] + w;
22            if (v != root) lap[i][j] = lap[i][j] - w;
23        };
24        for (auto [u, v, w] : edges)
25        {
26            assert(u >= 0 && u < n && v >= 0 && v < n);
27            if (u == v) continue;
28            if (kind == Kind::away_from_root) swap(u, v);
29            add(u, v, w);
30            if (kind == Kind::undirected) add(v, u, w);
31        }
32        return det_prime<mod>(move(lap));
33    }
34};
```

B.2 任意正模数带权矩阵树

`count(n, edges, root, kind, mod)` 返回树的边权乘积之和对 `mod` 的最小非负余数。`mod` 为任意正 signed 64 位数，边权支持完整 signed 64 位。`Kind` 的 `undirected`、`toward_root`、`away_from_root` 分别表示无向、内向和外向树；点号 `0..n-1`。自环忽略，重边按不同边计数，单点返回 `1%mod`，`mod=1` 返回零。先在 128 位中构造模拉普拉斯余子式，再调用 `determinant_mod`； $O(n^3 \log \text{mod} + m)$ 时间、 $O(n^2)$ 额外空间。不使用模逆元。已知模数是素数时一般优先使用 `MatrixTree<mod>`。

```

1 struct MatrixTreeMod
2 {
3     using ll = long long;
4     using Edge = tuple<int, int, ll>;
5     enum class Kind
6     {
7         undirected,
8         toward_root,
9         away_from_root
10    };
11
12    static ll count(int n, const vector<Edge> &edges, int root, Kind kind, ll mod)
13    {
14        assert(n >= 1 && root >= 0 && root < n && mod >= 1);
15        vector<vector<ll>> lap(n - 1, vector<ll>(n - 1));
16        auto add = [&](int u, int v, ll w)
17        {
18            if (u == root) return;
19            int i = u - (u > root), j = v - (v > root);
20            lap[i][i] = ((__int128)lap[i][i] + w) % mod;
21            if (v != root) lap[i][j] = ((__int128)lap[i][j] + mod - w) % mod;
22        };
23        for (auto [u, v, w] : edges)
24        {
25            assert(u >= 0 && u < n && v >= 0 && v < n);
26            if (u == v) continue;
27            w %= mod;
28            if (w < 0) w += mod;
29            if (kind == Kind::away_from_root) swap(u, v);
30            add(u, v, w);
31            if (kind == Kind::undirected) add(v, u, w);
32        }
33        return determinant_mod(move(lap), mod);
34    }
35 };

```

算法索引

basis_intersection, 61
basis_sum_intersection, 62
batch_inverse, 14
batch_units, 13
Berlekamp-Massey, 79
berlekamp_massey, 69
Bezout identity, 75
Binomial, 49
BostanMori, 47
Burnside, 79

Carmichael, 77
carmichael, 12
Catalan, 79
chirp_z, 42
Cipolla, 77
CompositeRoots, 30
convolution_i64, 38
coprime pairs, 72
CoprimePairs, 32
CRT, 74
crt_merge, 15

det_prime, 55
determinant_mod, 56
Dirichlet convolution, 70
DiscreteLog, 24
divisor sum, 73
divisor_sum_power, 2
Du Jiao sieve, 70
DuJiao, 33

Euler theorem, 74
Euler totient, 71
euler_phi, 7
exCRT, 76
ExLucas, 18
exLucas, 78
expectation, 80
extended_gcd, 11

Fenwick, 94
floor moments, 74
floor sum, 73
floor_moments, 35

floor_sum, 34
FpsFunctions, 44
FpsInverse, 43

Garner, 76
garner, 16
GaussMod, 67
generating functions, 79
geometric series, 73

inclusion-exclusion, 79
integer partitions, 72
inverse_table, 14

Korselt criterion, 77
KthResidue, 25

Lagrange, 65
Lagrange interpolation, 78
linear congruence, 76
linear sieve, 71
linear_congruence, 15
linear_equation, 12
LinearSieve, 8
Lucas, 17, 78
Lucas theorem, 74

Matrix Tree theorem, 79
MatrixTree, 95
MatrixTreeMod, 96
MaxPlusMatrix, 54
Miller–Rabin, 70
mint, 2
Mobius inversion, 70
Mod64, 1
mod_inverse, 13
mod_sqrt, 20
ModInt, 5
ModMatrix, 53
multiplicative order, 77

NttConvolution, 36

Partitions, 52
pentagonal number theorem, 72
Pollard–Rho, 70
PollardRho, 10
Polya, 79
polynomial_shift, 48
PositionBasis, 59

power_sum, 1
Prime64, 6
PrimePowerRoots, 26
PrimitiveRoot, 21

quadratic residue, 77

rank, 79
recurrence_nth, 46
root_factors, 29

segmented sieve, 72
segmented_primes, 9
SetConvolution, 39
Sprague-Grundy, 80
Stirling numbers, 79
stirling_first_row, 51
stirling_second_row, 50
subset_convolution, 40

Widget Factory, 80

XorBasis, 57
XorWalk, 62

二元一次不定方程, 75

区间筛素数, 72
原根, 77

扩展中国剩余定理, 76
批量逆元, 74
排列数, 78
模高斯消元, 80
混合进制, 76

矩形 GCD 计数, 72
类欧几里德, 73
线性同余方程, 76
线性逆元表, 75
线性递推, 80

阶乘表, 78